学校代号 10532 学 号 B2010S0412
分 类 号 TP391 密 级



湖南大學
HUNAN UNIVERSITY

博士学位论文

CPU-GPU 平台上稀疏张量运算库的设计与性能优化研究

学位申请人姓名 王昊天
培 养 单 位 信息科学与工程学院
导师姓名及职称 阳王东 教授
学 科 专 业 计算机科学与技术
研 究 方 向 高性能计算与云计算
论 文 提 交 日 期 2023 年 10 月 11 日

中国知网 https://www.cnki.net

学校代号： 10532

学　　号： B2010S0412

密　　级：

湖南大学博士学位论文

CPU-GPU 平台上稀疏张量运算库的设计与性能优化研究

学位申请人姓名： 王昊天

培　养　单　位： 信息科学与工程学院

导师姓名及职称： 阳王东 教授

专　业　名　称： 计算机科学与技术

论文提交日期： 2023 年 10 月 11 日

论文答辩日期： 2023 年 11 月 29 日

答辩委员会主席： 骆嘉伟 教授

中国知网 https://www.cnki.net

Design and Performance Optimization of Sparse Tensor Computing Library on CPU-GPU Platforms

By

Haotian Wang

B.S. (Nanchang University) 2018

A dissertation submitted in partial satisfaction of the

requirements for the degree of

Doctor of Engineering

in

Computer Science and Technology

in the

Graduate School

of

Hunan University

Supervisor

Professor Wangdong Yang

October, 2023

中国知网 https://www.cnki.net

摘 要

作为高维数据的一种表示方式，稀疏张量在学术界和工业界引起了广泛关注，其高维稀疏的特性为稀疏张量运算的计算和存储带来了巨大挑战。随着高性能计算平台由单一架构向异构架构的演进，高效的并行算法和性能优化方法的设计变得更加复杂。异构计算架构为稀疏张量运算库的设计和性能优化研究提供了新的机遇和挑战。不同类型的硬件核心适用于不同的稀疏张量运算，同时还需要解决体系结构差异和数据传输开销等问题。因此，充分发挥异构平台的优势以实现高效的稀疏张量并行计算和内存管理仍然是一个亟待解决的问题。

本文的研究主要集中在中央处理器-图形处理器（CPU-GPU）平台上的稀疏张量运算库的设计与性能优化方面。首先，对稀疏张量运算的并行算法设计和性能优化所面临的挑战进行了分析，并将常见的稀疏张量运算归纳为四种类型，包括内存限制类型、批量计算类型、高维计算类型以及链乘计算类型。其次，对 CPU-GPU 平台上的并行算法设计和性能优化存在的问题进行了分析，将这些问题总结为两个难题，即高速计算单元导致的内存墙问题和异构硬件之间的性能差异问题。随后，设计了一个异构自适应稀疏张量库（HASTensor），通过构建切片粒度和张量块粒度两种稀疏张量格式类型，以适应数据中复杂的稀疏分布和最新的高速计算硬件。最后，介绍了 HASTensor 中四类典型的稀疏张量运算的并行算法设计和性能分析优化方法，包括内存限制类型的稀疏张量乘以向量（SpTV）、批量计算类型的稀疏张量乘以矩阵（SpTTM）、高维计算类型的稀疏张量缩并（SpTC）以及链乘计算类型的稀疏张量矩阵链乘（SpTMCM）。

具体而言，本文取得的主要创新性研究成果如下：

1. 针对访存受限型稀疏张量运算中的计算效率问题，本文以 SpTV 为例，设计了输入感知的自适应流水线 SpTV（IAP-SpTV）方法，以缓解 GPU 的高速计算带来的内存墙问题。该方法首先采用一种混合切片格式的张量数据表示形式，并研究了基于该格式的流水线 SpTV 算法，以应对非零值的稀疏分布导致的性能下降。其次，构建了一个图卷积神经网络模型，为每个切片选择合适的存储格式。随后，通过建立流水线性能分析模型，实现了传输时间和计算时间的最大程度重叠，进一步缩短整体计算时间，提高了计算效率。最后，在两种不同的 CPU-GPU 平台上进行了一系列实验，结果显示，与现有方法相比，IAP-SpTV 获得了 1.25 倍至 1.58 倍的加速比。

2. 针对批量计算型稀疏张量运算中的批量处理过程，本文以 SpTTM 为例，设计了异构逐切片的 SpTTM（HS-SpTTM）方法，以充分发挥异构平台上各硬件架构的独特特性。该方法首先描述了 HS-SpTTM 在 CPU-GPU 平台上的处理流程，

中国知网 https://www.cnki.net

并给出了不同稀疏格式的 SpTTM 并行执行策略。接着通过对处理器的理论计算能力和任务计算量的估算，以实现在 CPU 和 GPU 之间任务划分的负载平衡。然后讨论了一种用图结构描述张量稀疏结构的方法，并设计了一个用于为张量选择储存格式的图卷积神经网络。最后，在 CPU-GPU 平台上进行了一系列实验，结果显示，与现有方法相比，HS-SpTTM 获得了 1.31 倍至 2.64 倍的加速比。

3. 针对高维计算类型的稀疏张量运算中的高维特征，本文以 SpTC 为例，设计了基于位坐标位图的 SpTC（BCB-SpTC）方法，以应对张量核心的矩形计算所导致的内存墙问题。该方法首先通过采用位压缩和位图技术，设计了一种稀疏张量存储格式，旨在减少高维张量的存储成本，从而显著提升内存访问效率。接着引入了一种冲突解决方法，通过构建任务列表和解除任务间的依赖关系，有效提高了线程块的利用率。然后通过设计高效的张量核心填充策略，以确保在处理各类张量缩并任务时充分利用张量核心的计算能力。最后，在 GPU 上使用一系列真实场景的稀疏张量进行了实验验证，结果显示，与现有方法相比，BCB-SpTC 获得了 1.10 倍至 21.27 倍的加速比。

4. 针对链乘计算类型的稀疏张量运算中计算依赖特性，本文以 SpTMCM 为例，设计了一种基于混合线性格式的张量矩阵链乘运算（HLSTO）方法，以应对 SpTMCM 在 GPU 的新兴张量核心上所面临的挑战。首先，与之前的研究相比，HLSTO 为 SpTMCM 提供了统一的存储格式和优化方法。接着，采用了多维分块的混合张量格式和高效的内存访问模式，以降低连续计算过程中的计算依赖，从而有助于实现 SpTMCM 的高效并行计算。然后，还开发了一种基于张量核心的高效并行算法，以提升内存带宽的利用效率。最后，在 GPU 上使用一系列真实场景的稀疏张量进行了实验验证，结果显示，与现有方法相比，HLSTO 获得了 1.16 倍至 24.12 倍的加速比。

对于上述研究内容，本文在稀疏张量运算库的设计与性能优化研究领域取得了一系列的研究成果，这些成果具有重要的理论价值和广泛的应用前景，而且进一步丰富和完善了该领域的前沿探索工作。

关键词：稀疏张量运算；并行算法设计；CPU-GPU 平台；性能分析；张量核心

中国知网 https://www.cnki.net

Abstract

Sparse tensors, as a representation method for high-dimensional data, have attracted significant attention in both academia and industry. Their high-dimensional sparsity poses substantial challenges for the computation and storage of sparse tensor computing. As high-performance computing platforms evolve from single architecture to heterogeneous architecture, the design of efficient parallel algorithms and performance optimization methods becomes increasingly intricate. Heterogeneous computing architectures offer both new opportunities and challenges for the design and performance optimization research of sparse tensor operation libraries. Different types of hardware cores are suited for various sparse tensor computing, necessitating the resolution of architectural disparities and data transmission overheads. Therefore, fully harnessing the advantages of heterogeneous platforms to achieve efficient parallel computation and memory management for sparse tensors remains an urgent issue.

This paper primarily focuses on the design and performance optimization of sparse tensor operation libraries on Central Processing Unit-Graphics Processing Unit (CPU-GPU) platforms. Firstly, it conducts an analysis of the challenges in parallel algorithm design and performance optimization for sparse tensor computing. It categorizes common sparse tensor computing into four types, including memory-bound, batch computation, high-dimensional computation, and sequential computation. Secondly, it analyzes the issues in parallel algorithm design and performance optimization on CPU-GPU platforms. These issues are summarized into two challenges: the memory wall issue caused by high-speed computing units and the performance disparity issue among heterogeneous hardware. Thirdly, an adaptive heterogeneous sparse tensor library, named HASTensor, is designed. It achieves adaptability to complex sparse distributions in data and the latest high-speed computing hardware by introducing two sparse tensor format types: slice granularity and tensor block granularity. Finally, it presents parallel algorithm design and performance analysis optimization methods for four typical sparse tensor computing in HASTensor. These operations include memory-bound operations like Sparse Tensor times Vector (SpTV), batch processing operations like Sparse Tensor Times Matrix (SpTTM), high-dimensional operations such as Sparse Tensor Contractions (SpTC), and sequential operations exemplified by Sparse Tensor Matrix Chain Multiplications (SpTMCM).

Specifically, this paper presents four key innovative research contributions:

1. To address the computational efficiency issues in memory-bound sparse tensor com-

中国知网 https://www.cnki.net

puting, this paper presents an Input-aware Adaptive Pipeline SpTV (IAP-SpTV) method using SpTV as an example, aiming to alleviate the memory wall problem caused by high-speed GPU computations. Firstly, it employs a hybrid slice format for tensor data representation and investigates a pipeline SpTV algorithm based on this format, addressing the performance degradation resulting from high-dimensional sparse data. Secondly, a graph convolutional neural network model is constructed for selecting an appropriate storage format for each slice. Thirdly, establishing a pipeline performance analysis model, maximizes the overlap between transfer time and computation time, further reducing overall computation time and enhancing computational efficiency. Finally, a series of experiments conducted on two different CPU-GPU platforms reveal that IAP-SpTV achieves acceleration ranging from 1.25 times to 1.58 times compared to existing methods.

2. For batch computation sparse tensor computing, this paper, taking SpTTM as an example, proposes the Heterogeneous Slice-wise SpTTM (HS-SpTTM) method, aiming to leverage the unique characteristics of various hardware architectures on heterogeneous platforms. Firstly, this method outlines the processing workflow of HS-SpTTM on CPU-GPU platforms and provides parallel execution strategies for different storage formats of SpTTM. Secondly, it achieves load balancing between the CPU and GPU by estimating the theoretical computing capabilities of processors and task workloads, enabling task partitioning. Thirdly, it discusses an approach utilizing graph structures to describe tensor sparse patterns and designs a Graph Convolutional Neural Network (SPT-GCN) for selecting appropriate tensor sparse formats. Finally, a series of experiments conducted on two different CPU-GPU platforms reveal that HS-SpTTM achieves acceleration ranging from 1.31 times to 2.64 times compared to existing methods.

3. To address the high-dimensional characteristics in high-dimensional computation sparse tensor computing, this paper, using SpTC as an example, proposes the Bit Coordinate Bitmap Format Based SpTC SpTC (BCB-SpTC) method to combat the memory wall problem induced by rectangular computations in tensor cores. Firstly, it designs a sparse tensor storage format by employing bit compression and bitmap techniques, aiming to reduce storage costs for high-dimensional tensors, significantly enhancing memory access efficiency. Secondly, it introduces a conflict resolution approach by constructing task lists and resolving inter-task dependencies, effectively improving thread block utilization. Thirdly, it designs an efficient tensor core padding strategy to ensure the full utilization of tensor core computing power when handling various tensor contraction tasks. Finally, experiments conducted on GPUs using a series of real-world sparse tensors demonstrate that BCB-SpTC achieves acceleration ranging from 1.10 times to 21.27 times compared to existing methods.

4. For sequential computation sparse tensor computing with computational dependen-

中国知网 https://www.cnki.net

cies, this paper, using SpTMCM as an example, designs a Hybrid Linear Storage Format Based Tensor Matrix Chain Multiplication Operation (HLSTO) method to address the challenges faced by SpTMCM on emerging tensor cores in GPUs. Firstly, in contrast to prior research, HLSTO provides a unified storage format and optimization methods for SpTMCM. Secondly, it employs multi-dimensional blocking in a hybrid tensor format and efficient memory access patterns to reduce computational dependencies during consecutive computations, facilitating efficient parallel computation for SpTMCM. Thirdly, it develops an efficient parallel algorithm based on tensor cores to enhance memory bandwidth utilization. Finally, experiments conducted on GPUs using a series of real-world sparse tensors demonstrate that HLSTO achieves acceleration ranging from 1.16 times to 24.12 times compared to existing methods.

In conclusion, this paper has achieved a series of research outcomes in the field of sparse tensor operation library design and performance optimization. These outcomes hold significant theoretical value and wide-ranging application prospects, further enriching and enhancing the frontiers of research in this domain.

Key Words: Sparse Tensor Computing, Parallel Algorithm Design, CPU-GPU Platform, Performance Analysis, Tensor Core

中国知网 https://www.cnki.net

目 录

第 1 章 绪论.....1
1.1 研究背景与意义.....1
1.2 相关研究工作.....6
1.2.1 经典的稀疏张量运算并行算法设计和性能优化研究.....6
1.2.2 自适应的稀疏张量运算并行算法设计和性能优化研究.....8
1.2.3 研究现状总结与本文的研究目标.....9
1.3 研究内容与主要贡献.....10
1.3.1 基于 CPU-GPU 平台的稀疏张量运算库 HASTensor.....10
1.3.2 面向四类稀疏张量运算的并行算法设计和性能优化.....11
1.4 本文组织结构.....13
第 2 章 张量基础概念和并行编程基础.....15
2.1 张量基础和相关符号.....15
2.1.1 矩阵化.....15
2.1.2 稀疏张量乘以向量.....16
2.1.3 稀疏张量乘以矩阵.....16
2.1.4 稀疏张量乘以稀疏张量.....17
2.1.5 稀疏张量矩阵链乘.....18
2.2 稀疏张量存储格式.....19
2.2.1 稀疏矩阵存储格式的扩展.....20
2.2.2 高维稀疏张量存储格式.....21
2.3 编程基础.....23
2.3.1 NVIDIA 架构介绍.....23
2.3.2 CUDA 编程模型.....25
2.3.3 张量核心编程模型.....26
2.3.4 CPU–GPU 异构并行编程.....27
第 3 章 访存受限型稀疏张量运算的并行算法设计和性能优化.....29
3.1 引言.....29
3.2 HTF 格式及其流水线 SpTV 算法.....31
3.2.1 HTF 存储结构.....31
3.2.2 基于 HTF 格式的流水线 SpTV 算法.....32
3.3 Slice-GCN 格式选择模型.....35
3.3.1 稀疏结构的图表示形式.....35
3.3.2 Slice-GCN 设计.....36
3.3.3 超参数设置.....36

中国知网 https://www.cnki.net

3.4 基于 HTF 的流水线性能分析模型.....37
3.4.1 HTF 格式的性能分析.....37
3.4.2 流水线性能模型.....40
3.5 实验评估.....42
3.5.1 平台和数据集.....43
3.5.2 模型训练结果.....44
3.5.3 最佳流数.....45
3.5.4 性能比较.....45
3.5.5 结果分析.....49
3.6 本章小结.....50
第 4 章 批量计算型稀疏张量运算的并行算法设计和性能优化.....51
4.1 引言.....51
4.2 HS-SpTTM 的概述.....53
4.2.1 整体工作流程.....53
4.2.2 异构并行计算模式.....53
4.3 负载均衡的任务映射策略.....57
4.4 SPT-GCN 格式选择模型.....59
4.4.1 稀疏张量的图表示.....59
4.4.2 图卷积神经网络.....60
4.4.3 SPT-GCN 的结构设计.....61
4.5 实验评估.....62
4.5.1 平台和数据集.....62
4.5.2 性能比较.....64
4.5.3 负载平衡.....65
4.5.4 格式选择的预测精度.....66
4.6 本章小结.....69
第 5 章 高维计算型稀疏张量运算的并行算法设计和性能优化.....70
5.1 引言.....70
5.2 BCB 格式.....71
5.2.1 格式转换.....72
5.2.2 分块技术.....72
5.2.3 位图存储技术.....73
5.2.4 位压缩技术.....73
5.2.5 存储空间分析.....73
5.3 BCB-SpTC 并行算法.....74
5.3.1 框架设计.....74
5.3.2 张量核心上的数据填充策略.....76
5.4 实验评估.....79

中国知网 https://www.cnki.net

5.4.1 实验设置.....79
5.4.2 BCB 格式的内存使用量分析.....80
5.4.3 BCB 格式的效率.....81
5.4.4 与其他实现的比较.....84
5.5 本章小结.....85
第 6 章 链乘计算型稀疏张量运算的并行算法设计和性能优化.....86
6.1 引言.....86
6.2 SpTMCM 的数学符号表示.....88
6.2.1 常见的符号表示.....88
6.2.2 基于矩形计算的符号表示.....89
6.3 混合线性存储格式.....90
6.3.1 格式转换.....91
6.3.2 位优化技术.....92
6.3.3 空间复杂度分析.....93
6.4 基于 HLS 的 SpTMCM 并行算法.....94
6.4.1 使用张量核心加速 SpTMCM.....94
6.4.2 HLS 的张量块阈值分析.....97
6.4.3 任务负载方案.....98
6.5 实验评估.....99
6.5.1 实验设置.....99
6.5.2 HLS 的有效性.....102
6.5.3 与其他实现的比较.....104
6.6 本章小结.....106
结论与展望.....107
参考文献.....110

中国知网 https://www.cnki.net

插图索引

图 1.1 研究背景和张量运算类型的概念图 ····· 2
图 1.2 处理器架构发展趋势的示意图 ····· 4
图 1.3 本文研究内容和稀疏张量运算库 HASTensor 的概述图 ····· 11
图 1.4 本文主要研究内容之间的关系结构图 ····· 13
图 2.1 矩阵格式存储张量切片的示意图 ····· 20
图 2.2 张量存储格式的示意图 ····· 21
图 2.3 GPU 内存层次的示意图 ····· 23
图 2.4 NVIDIA A100 中流多处理器架构的示意图 ····· 24
图 2.5 张量核心上的矩阵乘加示意图 ····· 27
图 3.1 不同 CUDA 流数量对应 SpTV 性能的结果图 ····· 31
图 3.2 HTF 格式储存张量的示意图 ····· 32
图 3.3 矩阵到二部图的转换示意图 ····· 35
图 3.4 Slice-GCN 模型结构示意图 ····· 36
图 3.5 矩阵稀疏度与稠密格式是最佳格式的关系图 ····· 37
图 3.6 流水线计算类型的示意图 ····· 41
图 3.7 模型在训练阶段的损失值、训练集精度和测试集精度的结果图 ····· 44
图 3.8 理论分析流数与实际测量流数的结果图 ····· 45
图 3.9 Mov 数据集上 SpTV 的平均执行时间的结果图 ····· 46
图 3.10 Het 数据集上 SpTV 的平均执行时间的结果图 ····· 46
图 3.11 Ube 数据集上 SpTV 的平均执行时间的结果图 ····· 47
图 3.12 Chi 数据集上 SpTV 的平均执行时间的结果图 ····· 47
图 3.13 Nip 数据集上 SpTV 的平均执行时间的结果图 ····· 48
图 3.14 一次 SpTV 计算的时间开销与模型推断的时间开销的比较结果图 ····· 48
图 3.15 分摊不同类型 SpTV 计算的模型推理时间所需的迭代次数的结果图 ····· 49
图 3.16 HTF 中每种格式的切片集的平均执行时间的结果图 ····· 49
图 3.17 自适应流水线方法中用于不同格式的 CUDA 流数量的结果图 ····· 50
图 4.1 不同格式张量对应的 SpTTM 性能结果图 ····· 52
图 4.2 HS-SpTTM 的工作流程示意图 ····· 53
图 4.3 CPU-GPU 上 HS-pTTM 异构并行计算模式的示意图 ····· 54
图 4.4 将张量转换成二部图的过程示意图 ····· 60
图 4.5 SPT-GCN 模型结构示意图 ····· 61
图 4.6 不同方法实现的 SpTTM 性能比较的结果图 ····· 65

中国知网 https://www.cnki.net

图 4.7 不同方法预测最佳存储格式的准确率及 SpTTM 性能比较的结果图 …… 65
图 4.8 不同格式转换方法的时间开销结果图 …… 66
图 4.9 负载均衡策略对时间开销影响的结果图 …… 66
图 4.10 SPT-GCN 的训练过程的结果图 …… 67
图 4.11 最佳稀疏格式预测的准确性比较的结果图 …… 68
图 4.12 搜索最佳稀疏格式的时间开销的结果图 …… 68
图 5.1 从 COO 到 BCB 的格式转换过程的示意图 …… 72
图 5.2 BCB-SpTC 并行计算框架的示意图 …… 74
图 5.3 张量核心上数据加载方法的示意图 …… 77
图 5.4 张量核心上数据填充方法的示意图 …… 77
图 5.5 收缩两个模态 SpTC 的总执行时间结果图 …… 82
图 5.6 与 tSparse 对比计算阶段时间开销的结果图 …… 83
图 5.7 张量核心上数值误差的结果图 …… 84
图 5.8 比较不同方法实现 SpTC 性能的结果图 …… 85
图 6.1 常用计算方法的示意图 …… 89
图 6.2 基于矩形计算方法的示意图 …… 90
图 6.3 张量从 COO 到 HLS 格式转换过程的示例图 …… 91
图 6.4 两位优化技术的示例图 …… 92
图 6.5 张量核心加速 SpTMCM 的流程图 …… 95
图 6.6 HLS 格式计算任务加载到 GPU 的流多处理器上的过程示意图 …… 98
图 6.7 不同张量块密度对应的结果图 …… 101
图 6.8 A100 上执行融合任务负载方案前后的时间开销比较的结果图 …… 101
图 6.9 A100 上 SpTMCM 内存带宽的结果图 …… 103
图 6.10 A100 上 SpTMCM 的时间开销的结果图 …… 104
图 6.11 SpTMCM 的计算误差的结果图 …… 104
图 6.12 SpMTTKRP 不同方法在 A100 上的性能的结果图 …… 105
图 6.13 SpTTMChain 不同方法在 A100 上的性能的结果图 …… 105
图 6.14 从标准稀疏格式 COO 到其他格式的转换时间的结果图 …… 106

中国知网 https://www.cnki.net

附表索引

表 3.1 SpTV 实验平台配置总结表 ····· 43
表 3.2 SpTV 实验中张量数据集的总结表 ····· 43
表 4.1 SpTTM 实验平台配置总结表 ····· 63
表 4.2 SpTTM 实验中张量数据集的总结表 ····· 63
表 4.3 训练数据集中不同大小张量所占比例的总结表 ····· 67
表 5.1 数据集的内存使用量（MB）的统计表 ····· 81
表 6.1 SpTMCM 实验平台配置总结表 ····· 99
表 6.2 SpTMCM 实验中数据集的总结表 ····· 100
表 6.3 数据集的内存使用情况（MB）的总结表 ····· 102

中国知网 https://www.cnki.net

第 1 章　绪　论

1.1　研究背景与意义

随着现代科学技术的迅猛发展，当前社会已进入信息爆炸时代。海量的数据从各种源头不断涌现，包括社交媒体、移动设备、传感器等，覆盖了医疗、金融、交通、工业制造等多个领域，带来了巨大的信息量。这些数据不再仅仅是数字和字符的简单堆积，而呈现出更复杂、多元化的形态。

在众多领域的应用中，数据逐渐组织成高维、多模态的数据结构，对传统的数据表示和处理方法提出了较大的挑战。例如，在基因组学中，每个个体的基因信息可表示为数千维的数组[1]；在交通领域，交通系统数据包括车辆流量、路况、交通信号等多维信息[2,3]；在自然语言处理中，一篇文本可表示为高维词频向量[4,5]。这种高维数据表示引发了“维度灾难”问题，使得传统的数据处理方法变得效率低下甚至无法处理。

稀疏性是现实数据的重要特征之一，意味着数据集中包含大量的零值或缺失值。即在整个数据集中，只有极少数的非零或有意义的数据点，而其他数据点为空。例如，社交网络中的用户连接可以表示为图结构，其中每个用户对应一个节点，而连接表示用户之间的关系[6,7]。然而，在大多数情况下，不是所有用户都会相互连接，这导致了图结构的稀疏性。在大规模的社交网络中，虽然用户众多，但它们之间的连接分布不均匀，只有极少数用户之间存在紧密的关联，而其他用户可能没有连接。这种稀疏性导致了图的邻接矩阵中的大多数元素都是零，只有极少数元素表示实际的连接，从而形成了稀疏的数据结构[8,9]。

作为多维数据结构，稀疏张量能有效地存储高维数据，并且能够捕捉数据的稀疏性。因此，稀疏张量已经成为处理涉及稀疏和多模态数据复杂关系的重要数据表示方式[10,11]。

然而，随着数据维度和模态多样性的持续增长，张量的规模和复杂性急剧扩大。这一现象不仅对计算和存储构成了较大的挑战，还使传统的数据处理方法难以有效应对高维多模态数据[12]。传统的线性代数方法和数据存储方式在处理稀疏张量时表现出效率低下，需要更多计算资源和时间[13]。此外，传统的密集表示方式在处理这些数据时会浪费大量存储空间和计算资源，而稀疏张量格式能更紧凑地呈现数据，同时保留关键信息[14]。

提升计算性能、优化存储效率并保持结果准确性，已成为当前稀疏张量运算并行算法设计和性能优化的核心目标。在数据分析和人工智能领域，如张量分解[15,16] 和深度学习[17–20] 等应用，广泛采用稀疏张量运算，从而涌现出多种计算

中国知网　https://www.cnki.net

形式，包括稀疏张量乘以向量（Sparse Tensor times Vector，SpTV）[21,22]、稀疏张量乘以矩阵（Sparse Tensor Times Matrix，SpTTM）[23,24]、稀疏张量收缩（Sparse Tensor Contraction，SpTC）[25,26]、稀疏矩阵化张量乘 Khatri-Rao 积（Sparse Matricized Tensor Times Khatri-Rao Product，SpMTTKRP）[27,28] 以及稀疏张量乘矩阵链（Sparse Tensor Times Matrix Chain，SpTTMChain）[29,30]。具体描述如图1.1所示，并且每种稀疏张量运算都伴随着独特的挑战和待解决的技术问题。

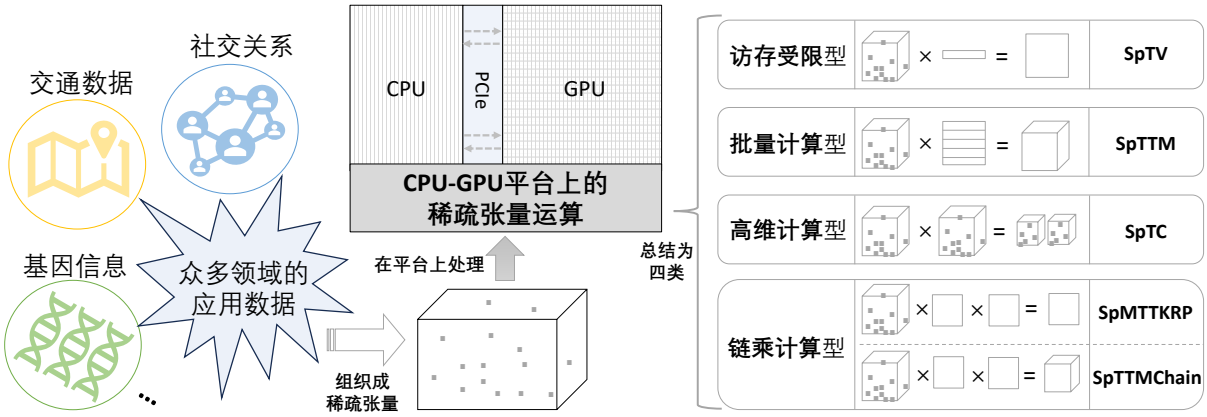


图 1.1　研究背景和张量运算类型的概念图

为了应对这些挑战，本文将稀疏张量运算归纳为以下四种类型进行分析：

1、　访存受限类型

SpTV 被视为访存受限型张量运算的典型案例。在并行计算领域，访存受限问题一直是广泛存在的挑战，对稀疏张量运算的并行算法设计产生了深刻影响。随着数据维度和模态数量的增加，稀疏张量的规模呈指数级增长，因此对内存资源的需求也大幅增加。在处理如此大规模的稀疏张量数据时，内存资源已成为一个至关重要的瓶颈，因为传统的内存存储和访问方式无法有效克服这一挑战[22]。为了应对内存约束的挑战，提高内存访问效率显得至关重要。鉴于稀疏张量中包含大量零元素，妥善处理这些零元素对于提高内存效率至关重要。有效的存储方法设计有助于降低内存开销并提升内存访问效率。然而，在确保数据准确性的前提下实现这一目标仍然是一项充满挑战性的任务[21]。尽管存在多种高效的压缩和稀疏数据表示方法，但在并行计算环境中，必须在计算效率、内存效率和结果准确性之间取得平衡。

总之，访存受限型张量运算所面临的挑战包括全面高效地考虑存储、访问以及处理大规模稀疏张量数据，以满足并行计算任务的需求。因此，这一挑战不仅涉及到计算性能的提升，还涵盖了资源管理、任务调度和数据传输等多个方面。

2、　批量计算类型

作为代表性的批量计算类型的张量运算之一，SpTTM 在并行计算领域具有重要地位。在众多计算任务中，稀疏张量经常呈现出批量计算的特性，即能够同时

中国知网　https://www.cnki.net

处理多个相同类型的计算任务[23]。深度学习领域的批量训练是批量计算的重要应用之一，通常涉及同时处理多个稀疏张量的乘法等运算。高效地设计并行算法，以较好地利用批量计算的特性并提高计算效率，是当前稀疏张量运算研究的一个重要课题[20]。充分发挥批量计算的潜力可以显著减少计算时间和资源的浪费，从而提高计算性能。因此，适应不同批量大小、维度和稀疏性的张量的并行算法设计面临着资源管理和任务调度等方面的挑战[24]。

总之，为了提高计算效率，需要深入研究批量计算类型的张量运算过程，设计高效的并行算法，以充分发挥批量计算的特性。因此，这一挑战涵盖了算法设计、数据结构优化、内存管理等多个方面。

3、 高维计算类型

SpTC 作为典型的高维计算类型的稀疏张量运算，在并行计算领域具有重要地位，并在实际应用中得到广泛使用。处理这类任务需要考虑更多维度和数据分布问题。高维稀疏张量的计算复杂性较高，计算复杂度随维度增加而逐渐增加，需要更多计算资源和内存，可能导致计算时间显著增加，甚至在有限资源下难以完成。因此，有效管理和优化计算复杂度以提高计算效率成为高维计算类型的张量运算中的关键问题[25]。此外，高维稀疏数据存在不均匀分布，即只有少数维度或坐标包含非零值，其他维度可能为空。如何处理这种不均匀性以减少由零值导致的计算资源浪费，也是高维计算类型的挑战之一。传统的稠密存储方式在高维情况下会浪费大量存储空间，而稀疏存储方式则需要高效索引和访问机制。因此，设计高效的存储和访问方案以满足高维计算类型的张量运算需求至关重要[26]。

总之，解决高维计算类型的张量运算挑战需要深入研究高效处理高维稀疏数据，以降低计算复杂度并提高计算效率。因此，这一挑战涉及多个方面，包括高维数据的复杂性、数据不均匀分布以及存储与访问效率等。

4、 链乘计算类型

在实际应用中，链乘计算类型的稀疏张量运算发挥着核心作用，尤其是在涉及多次连续乘法运算的稀疏张量运算场景中，如 SpMTTKRP 和 SpTTMChain。这些运算在本文中被归类为稀疏张量矩阵链乘（Sparse Tensor Matrix Chain Multiplication，SpTMCM）。链乘计算类型的稀疏张量运算通常包含多次乘法运算，相对于其他张量运算，计算量更大。这一特性增加了计算的时间成本，特别是对于大规模的稀疏张量。因此，降低计算时间成本并提高计算速度成为一项重要挑战[28]。链乘计算类型的稀疏张量运算具有严格的计算顺序依赖性，意味着每一步计算的结果都依赖于前一步的结果。这种严格的依赖性无疑增加了计算的复杂性，同时也限制了并行计算的可能性[29]。此外，由于计算的连续性，有效地管理计算资源也至关重要。这包括对内存访问和通信带宽的管理和分配，以及数据流中数据移动路径的设计和实现[27]。

中国知网 https://www.cnki.net

总之，链乘计算类型的挑战需要深入研究高效处理具有计算顺序依赖性的稀疏张量运算。因此，这一挑战不仅涉及计算顺序依赖的处理，还需要考虑资源分配和管理以及数据流等多个方面。

随着计算机技术的不断演进，异构架构的发展在稀疏张量运算领域引发了新的机遇和挑战。如图1.2所示，过去几十年中，计算机处理器主要采用单一的架构，如中央处理器（Central Processing Unit，CPU）。处理器架构经历了从单核心到多核心的演进。然而，随着摩尔定律逐渐失效，传统的单核心处理器已很难继续追求更高性能。因此，多核心处理器逐渐成为主流，这种架构使得在单个芯片上同时执行多个任务成为可能。然而，随着任务的多样性和复杂性的增加，多核心处理器面临着高功耗和高热量等挑战[31,32]。为了进一步提高计算能力和效率，异构计算架构逐渐引起了关注。异构计算指的是在同一个平台中使用不同类型的处理器核心或计算核心，以满足不同类型的计算需求。

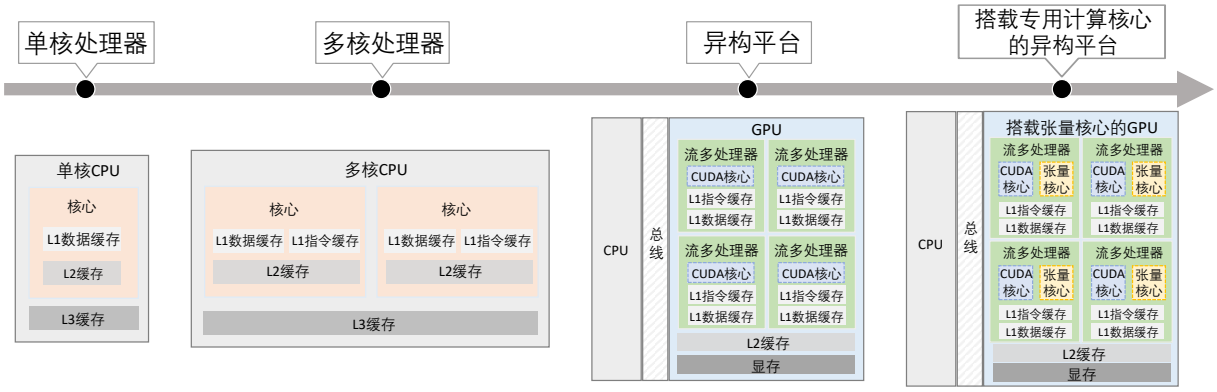


图 1.2　处理器架构发展趋势的示意图

在过去的十年中，图形处理器（Graphics Processing Unit，GPU）异构架构的发展引领了计算性能的飞速提升。NVIDIA 公司于 2007 年推出了第一款统一计算架构（Compute Unified Device Architecture，CUDA）的 GPU，为通用计算提供了强大的并行处理能力。GPU 的引入使得矩阵运算等科学计算任务得以高效地并行执行，为稀疏张量运算的优化铺平了道路。随着深度学习的兴起，GPU 逐渐成为训练神经网络的主要硬件平台，极大地加速了模型训练的速度[33]。2010 年代末至 2020 年代初，异构架构的发展进一步扩展到了深度学习领域。英伟达（NVIDIA）公司推出了 V100 和 A100 等型号的 GPU，其引入了专为深度学习任务优化的张量核心单元（Tensor Core Unit，TCU），使稀疏张量运算的效率有了进一步提升的可能[34]。

然而，异构计算领域在并行算法设计和性能优化方面面临一系列挑战。首先，不同类型的硬件拥有各自的架构和特性，因此需要为每种硬件设计适应性强的优化策略。其次，数据在不同硬件之间的传输和共享会引发数据移动的开销，降低了效率，如何有效减少这些开销成为一个关键问题[35]。此外，不同硬件之间的编程模型和工具链也各自不同，通常在 CPU 上运行的程序使用 GNU 编译器套件

中国知网　https://www.cnki.net

（GNU Compiler Collection，GCC），而在GPU上运行的程序需要使用英伟达CUDA编译器（NVIDIA CUDA Compiler，NVCC）。因此，需要针对异构平台面临的这些问题，设计相应的解决方案，以使开发人员能够更好地利用异构硬件的性能优势[36]。

在中央处理器-图形处理器（Central Processing Unit-Graphics Processing Unit，CPU-GPU）平台上，包含三种不同类型的计算核心，即数量少但功能强大的CPU核心，GPU上数量众多的CUDA核心以及最新引入的张量核心，这些核心协同工作，以充分发挥其计算性能，是CPU-GPU平台上并行算法设计和性能优化的重要目标。在追求这一目标的过程中，主要面临两个核心问题，一是高速计算单元导致的内存墙问题，二是异构硬件之间性能差异问题。内存墙问题是指受限于内存访问速度，可能导致计算单元闲置的情况，从而延长了整体的计算时间。异构硬件之间性能差异问题是指不同计算单元之间存在性能差异，可能导致任务负载不均衡的情况，从而降低了整体的计算性能。

GPU作为高速计算的处理器，在处理计算密集型任务时速度远高于CPU处理器[37]。然而，CPU和GPU异构计算的总时间通常取决于其中最慢的一方，因此，设计有效的任务负载策略至关重要。此外，由于GPU的控制单元远少于计算单元，复杂的计算任务在大多数情况下需要从CPU处理器上开始启动，因此，数据在CPU和GPU之间的传输开销也需要被充分考虑。

张量核心由于其矩形计算的特性，其计算速度远快于CUDA核心，但在处理非矩形计算时，常常会产生大量无效计算。因此，根据计算任务的特性，有针对性地使用这两种核心是十分必要的[38]。由于内存访问速度的限制，很多情况下高速计算单元需要等待数据的加载，即内存墙问题。因此，合理组织计算任务，提高内存访问效率是充分发挥高速计算单元性能的核心策略。

综上所述，在当今大数据和人工智能应用的背景下，稀疏张量运算已广泛应用于多个领域。设计一个通用的稀疏张量运算库成为高性能计算平台的一项重要任务，亟需进行科学研究和工程实践。CPU-GPU平台上的异构计算架构为稀疏张量运算库的并行优化提供了新的机遇和挑战。不同类型的硬件核心可以适用于不同的稀疏张量运算任务，但同时也需要解决高速计算单元导致的内存墙问题和异构硬件之间性能差异问题。在CPU-GPU平台上，如何充分发挥每种计算单元的性能优势，实现高效的内存访问和并行计算，仍然是一项亟待解决的关键问题。因此，CPU-GPU平台上稀疏张量运算库的设计与性能优化研究是一个具有重大科研意义和极高应用价值的研究课题。

中国知网 https://www.cnki.net

1.2 相关研究工作

已有大量研究工作针对不同硬件架构，就稀疏张量运算的并行算法设计和性能优化进行了深入研究。本节将关注这些研究面临的挑战，介绍与稀疏张量运算并行算法设计和性能优化策略相关的研究现状，分析现有工作的不足之处，并概述本文的研究目标。

1.2.1 经典的稀疏张量运算并行算法设计和性能优化研究

在张量被广泛应用的背景下，设计高效的并行算法和性能优化策略，以满足稀疏张量运算的需求，成为一项重要而值得深入探讨的任务。SpTV 作为一个关键的张量运算，当前已存在大量性能优化的研究工作。Chen 等人[21] 在神威太湖之光超级计算机上提出了一个 SpTV 框架，以解决在高性能计算平台上优化 SpTV 中的若干挑战。Xiao 等人[39] 利用稀疏矩阵的统计模型来选择实现并行 SpTV 负载均衡的最佳分割阈值。Yang 等人[22] 制定了一种压缩策略来提高大型稀疏张量的存储效率，并在 CPU-GPU 平台上设计了一种 SpTV 的流水线计算方法。Pawłowski 等人[40] 提出了一种新的数据结构，其中张量被分块并以 Morton 排序的顺序存储块，并基于该数据格式设计了一种张量乘以向量的算法。

SpTTM 作为典型的稀疏张量运算，其性能优化受到密切关注。Smith 等人[41] 提出了一个用于存储稀疏张量的稀疏压缩格式，并基于该格式设计了一种新的 SpTTM 的并行算法。Ma 等人[24] 根据 GPU 的特性，设计了一种优化的 SpTTM 计算方式。该方法避免了张量和矩阵之间的数据转换。Li 等人[23] 在多核架构上构建了一种 SpTTM 算法，并通过并行化、避免锁和挖掘数据局部性的方式进一步优化 SpTTM 算法的性能。

SpTC 作为一个重要的稀疏张量运算，其优化问题一直以来都备受关注。主要的优化思路集中在降低稀疏性对性能的负面影响。为了实现这一目标，近年来提出了许多优化方法。Ozog 等人[42] 研究了在计算化学代码中基于块的 SpTC 的调度算法，并考虑了不同的稀疏程度。Liu 等人[25] 在新的硬件平台上设计了一个 SpTC 算法，它采用了新颖的数据表示形式和结构来解决 SpTC 在新的硬件平台上优化所面临的具体挑战。H’erault 等人[26] 在 Summit 超计算机上提出了一个分布式的块稀疏张量缩并算法。Xiao 等人[43] 在 CPU-GPU 平台上提出了一个高效的按元素计算的 SpTC 框架，解决了索引匹配和积累的问题。Zhao 等人[44] 提出了一个用于 SpTC 计算的代码生成器。它利用稀疏多面体框架中循环嵌套计算的数学表示，扩展多面体模型以支持稀疏张量中出现的计算。

SpMTTKRP 是 CP 分解（Canonical Polyadic Decomposition，CPD）中的核心运算之一，其性能优化一直备受关注和研究。然而，针对稀疏数据而言，传统的并行策略通常会导致严重的负载不均衡，使得每个并行单元处理的数据差异巨大。

中国知网 https://www.cnki.net

为了解决这种负载不均衡问题，Li 等人[45] 提出了一种新颖的张量记忆算法，并专注于一系列的 SpMTTKRP 运算，以寻找在各个模态之间实现计算开销平衡的方法。Hu 等人[46] 利用向量内积来实现 SpMTTKRP。他们设计了一个线程划分策略，最大化 GPU 的计算效率，并通过将线程束执行方法与共享内存相结合，避免了线程束间的负载不均衡。Kurt 等人[47] 为 SpMTTKRP 设计了一种细粒度负载平衡的方案，以支持更高级别的并行。设计一种合适的稀疏格式用于减少内存占用、优化内存访问也是优化 SpMTTKRP 的一个重要方法。Nisa 等人[48] 提出了一种混合模态张量表示方法，将张量的非零值分区为不相交的部分，并将每个部分沿不同的模态压缩以创建纤维。并且将该表示方法应用于优化 SpMTTKRP。Nisa 等人[28] 开发了一个新型高效存储张量表示方法，使得在 GPU 上高性能、负载平衡的执行 MTTKRP 成为可能，并使用这种新的稀疏张量表示实现了 SpMTTKRP 计算。位操作作为常用的数据压缩方法，旨在高效处理频繁访问的数据。Nguyen 等人[49] 设计了一个线性存储格式，通过线性化非零值的索引，将其编码为单个索引，这一方法使得每个索引的非零值只需访问一次，无需像传统的张量模式那样需要多次内存访问。他们成功地将这种格式应用于大规模并行 GPU 架构上，以加速 SpMTTKRP 运算。Hayashi 等人[50] 设计了一个两阶段 SpMTTKRP 算法。该算法首先在张量的一个切片上执行部分的 SpMTTKRP，然后在张量和一个 Khatri-Rao 乘积的矢量化之间执行张量与矢量的运算。Wijeratne 等人[27] 在现场可编辑门阵列（Field Programmable Gate Array，FPGA）上设计了一种硬件算法，以最小化 SpMTTKRP 的执行时间。Nagasaka 等人[51] 设计了一种模态不敏感的快速 SpMTTKRP 算法，其具有可扩展性好、速度快、节省内存的优点。Won 等人[52] 提出了一种在 SpMTTKRP 程序中联合优化给定稀疏模式的格式和调度的新方法。除了以上列举的工作，还有很多 MTTKRP 性能优化的研究工作[53–56]。

SpTTMChain 作为 Tucker 分解算法中的关键内核，其性能优化一直是研究的焦点领域。Kaya 等人[57] 在多核并行平台上为 SpTTMChain 提供了一种高效的并行化方法。Smith 等人[30] 设计了一种基于压缩数据结构的 SpTTMChain 算法。该算法可以在计算过程中识别和修剪许多计算冗余，避免了过多的内存开销。Baert 等人[29] 提出了一种新技术来加速 SpTTMChain，并使用一种稀疏压缩格式存储稀疏张量以减少内存的使用。

寻找稀疏张量运算之间的共性和设计稀疏张量运算的统一形式，也是稀疏张量运算并行算法设计和性能优化的关键研究领域之一。Li 等人[58] 提出了一个块压缩格式，采用稀疏块状模式存储稀疏张量。它将三阶张量分割为大小为 $2 \times 2 \times 2$ 的子张量，并用较少的比特位数表示每个子张量。Liu 等人[59] 为 SpMTTKRP 和 SpTTMChain 设计了统一的并行算法，这些算法对模态变化不敏感。他们捕捉了计算模式中的变化，并使用两个标志数组来表示索引模式中的任何变化。

中国知网 https://www.cnki.net

最近，研究人员已经开始探索张量核心在性能优化方面的潜力。张量核心是一种专用硬件组件，旨在加速 GPU 上的矩形乘法运算。张量核心的发展受到了深度学习的推动，因此如何在各种应用中提高性能已经成为当前研究的重点。此外，张量核心主要用于处理小型而密集的矩阵，因此其适用范围有一定限制。Huang 等人[60] 利用张量核心来优化分层 Tucker 张量学习基元，并将优化后的基元用于优化大规模数据分析的分层 Tucker 张量分解算法。Dakkak 等人[61] 通过张量核心将归约和扫描都表达为矩阵乘法运算，并展示了一种新颖、直接和有效的在张量核心上映射归约和扫描基元的方法。张量核心在张量分析中也被广泛使用。Wang 等人[62] 提出了一个基于张量核心的图神经网络（Graph Neural Network，GNN）加速框架，将稀疏 GNN 计算与张量核心相结合。Zachariadi 等人[38] 设计了一个新的稀疏矩阵乘以矩阵（Sparse General Matrix-Matrix Multiplication，SpGEMM）算法，它采用基于位图的格式来存储矩阵，将输入矩阵分割成小块，并使用张量核心的混合精度模式对小块进行乘法运算。Feng 等人[63] 开发了在张量核心单元上对通用矩阵乘（General Matrix Multiplication，GEMM）进行了仿真，并在不影响精度要求的情况下加速科学计算应用。

1.2.2 自适应的稀疏张量运算并行算法设计和性能优化研究

目前，自适应方法已经成为稀疏张量运算中优化问题的重要方法之一。在这些方法中，自适应选择张量的存储格式是一个备受研究关注的热点。这种方法通常依赖于输入数据的特征，例如数据规模、稀疏度、数据分布等，以自动确定最适合数据的存储格式。Xie 等人[64] 开发了一个深度学习模型，用于稀疏矩阵乘以稀疏矩阵运算，以自动确定任意稀疏矩阵的最佳格式和算法。Zhao 等人[65] 揭示了卷积神经网络作为稀疏格式选择模型的有效性。Sun 等人[66,67] 为 SpMTTKRP 设计了一个用于自动预测输入稀疏张量的最佳存储格式的框架。Tan 等人[68] 开发了一个自动调优系统，为程序员提供了一个统一的接口。该系统利用机器学习模型和可移植的后端库，通过隐式选择和将数据转换成压缩稀疏行的稀疏矩阵格式，快速预测最佳组合。Benatia 等人[69] 使用支持多分类的支持向量机分类器来为 GPU 上的每个输入矩阵选择最佳格式。Sedaghati 等人[70] 研究了 GPU 架构、稀疏矩阵表示和稀疏数据集之间的相互关系，并使用机器学习建立了一个决策模型，在给定目标系统的情况下根据稀疏矩阵的特性自动选择最佳稀疏矩阵格式。Niu 等人[71] 利用稀疏矩阵的二维空间结构，优化了 GPU 上的稀疏矩阵乘以向量（Sparse Matrix-Vector Multiplication，SpMV），并设计了一种选择方法，为每个块找到最佳的格式和 SpMV 实现。Yuan 等人[72] 设计了寄存器分块算法和启发式分块大小选择算法, 将稀疏矩阵分成小的稠密分块并重用保存在寄存器中向量元素, 提高 SpMV 计算内核的性能。Li 等人[73] 为 SpMV 开发了一个自动调优器，结合矩阵特征为给定的稀疏矩阵选择并返回其最优的存储格式。Su 等人[74] 设计了一种

中国知网 https://www.cnki.net

新的稀疏格式，并基于该格式开发了一个 SpMV 框架。该框架能够在运行时分析各种类型的稀疏矩阵，并在不同平台上自适应确定稀疏矩阵的最佳表示。Popoola 等人[75] 提出了稀疏张量格式的形式化定义和一种自动生成格式转换程序的方法。该方法将稀疏张量自适应地转换为适合应用的表达形式。

此外，输入数据自动生成最优的程序代码，以及自动选择最优的计算内核或计算框架等研究方向也是研究自适应方法的有效途径。Dai 等人[76] 提出了一种数据感知的基于 GPU 的启发式核心，并实现了输入的动态自适应优化代码。Kim 等人[77] 展示了一种应用了张量收缩的自适应高性能代码生成器，通过张量收缩中有关数据属性来设计一个有效的代码的生成模式。Du 等人[78] 开发了一个格式选择和内核设计的框架，通过输入稀疏模式和硬件架构自动创建新的机器设计的格式和 SpMV 内核实现。Vuduc 等人[79] 提出了一组底层基元的集合。该基元集合提供了稀疏矩阵上自动调优的计算内核，包括 SpMV 和稀疏三角求解等。它隐藏了为特定用户的稀疏矩阵和机器调优内核实现的性能所需的复杂决策过程。Zheng 等人[80] 设计了一个面向深度学习应用的自适应张量程序生成框架。Duan 等人[81] 提出了一种灵活的模态 Tucker 分解算法，实现了因子矩阵和核心张量求解器的切换，并使用机器学习自适应求解器选择器来自动处理输入数据和硬件的变化。

根据输入数据和硬件架构自适应地为张量运算程序选择优秀的参数配置、并行策略和资源分配策略等也是一个研究热点。Li 等人[82] 展示了一种自适应的张量乘以矩阵（Tensor Times Matrix，TTM）方法，可以选择启发式经验模型来实现 TTM 输入的最优配置。Nguyen 等人[83] 提出了一种自适应阻塞策略，该策略不仅满足了目标 GPU 设备的资源约束，而且加速了数据索引，消除了控制流和内存访问不规则性，减少了内核启动开销，同时保持了较高的压缩率。Chen 等人[84] 提出了一种自适应张量并行框架，可以在不同的互连上自动选择最优并行策略。Kourtis 等人[85] 展示了一种自适应代码生成框架，通过采用运行时代码生成来为每个矩阵构造专用的 SpMV 例程。Gibson 等人[86] 设计了一个用于识别和重用张量程序之间自动调度的新方法。Zheng 等人[87] 设计了一个用于异构系统上张量运算的自动优化框架，并使用启发式方法来优化张量运算程序。此外，自适应优化方法还包括自动调优器等研究内容[88–95]。

1.2.3 研究现状总结与本文的研究目标

总而言之，最新研究工作对稀疏张量运算优化关注点往往是特定类型的稀疏张量运算，缺乏整体性研究与归纳。虽然现有的工作，如并行张量基础设施（A Parallel Tensor Infrastructure!，ParTI!）[①] 和并行稀疏张量工具包（The Surprisingly ParalleL spArse Tensor Toolkit，SPLATT）[②] 等稀疏张量运算库归纳了部分稀疏张量

[①] https://github.com/hpcgarage/ParTI

[②] https://github.com/ShadenSmith/splatt

中国知网 https://www.cnki.net

运算，然而缺乏对 CPU-GPU 平台以及 GPU 上最新的张量核心单元的支持。设计先进的稀疏张量运算库满足大数据分析和深度学习应用领域需求，是当前一项重要的研究任务。因此，本文旨在填补该研究领域的此项空缺，设计一个 CPU-GPU 平台上先进的稀疏张量运算库。

1.3 研究内容与主要贡献

本文的核心目标是设计一个 CPU-GPU 平台上稀疏张量运算库，用于优化稀疏张量运算的并行计算性能，以应对各种稀疏张量应用中高维多模态数据处理的挑战。针对这一目标，本文设计一个异构自适应稀疏张量库（Heterogeneous Adaptive Sparse Tensor Computing Library，HASTensor），并提出了一系列创新性的并行算法和关键技术。本节将总结本文的主要创新点和贡献，包括 HASTensor 的整体设计，以及以四类典型的稀疏张量运算为例详细的并行算法设计和性能优化方法介绍。

1.3.1 基于 CPU-GPU 平台的稀疏张量运算库 HASTensor

如图1.3所示，HASTensor 是一个基于 CPU-GPU 平台设计的稀疏张量运算库，适用于多个领域，包括 Tucker 分解、CP 分解、Tensor Train 分解、Tensor Ring 分解、张量网络分解、深度学习等众多应用。这些广泛的应用场景导致了数据稀疏分布的复杂性以及运算形式的多样性，其中稀疏张量运算的并行算法设计和性能优化的挑战主要体现在访存受限、批量计算、高维计算和链乘计算四个方面。此外，CPU-GPU 平台上异构的体系结构对高效并行算法设计提出了更高要求，主要包括高速计算单元引发的内存墙问题和异构硬件之间性能差异问题两个方面。

为应对复杂多样的张量稀疏分布和计算硬件，HASTensor 采用了两种不同粒度的稀疏格式，即切片（Slice）粒度和张量块（Tile）粒度。切片粒度的稀疏格式使用成熟的稀疏矩阵格式存储每个切片，借鉴现有的稀疏矩阵性能优化方案，以提高稀疏张量运算性能。而张量块粒度的稀疏格式将张量分割成多个小张量，并对每个小张量进行存储。这些小张量被称为张量块，其计算任务更容易受益于张量核心的加速。

为适应不同种类的张量运算过程，HASTensor 将这些运算过程分为基础运算和高级运算两类。稀疏张量基础运算的计算过程相对较为简单直接，是稀疏张量运算的基本单元，例如 SpTV 和 SpTC。相对而言，稀疏张量高级运算的计算过程更为复杂，可视为对稀疏张量基础运算的进一步扩展，包括 SpTTM、SpMTTKRP 和 SpTTMChain。

与现有稀疏张量运算库相比，HASTensor 的主要特点有五个：

1、支持异构并行计算：能够较好地利用 CPU 和 GPU 上多种处理器计算核

中国知网 https://www.cnki.net

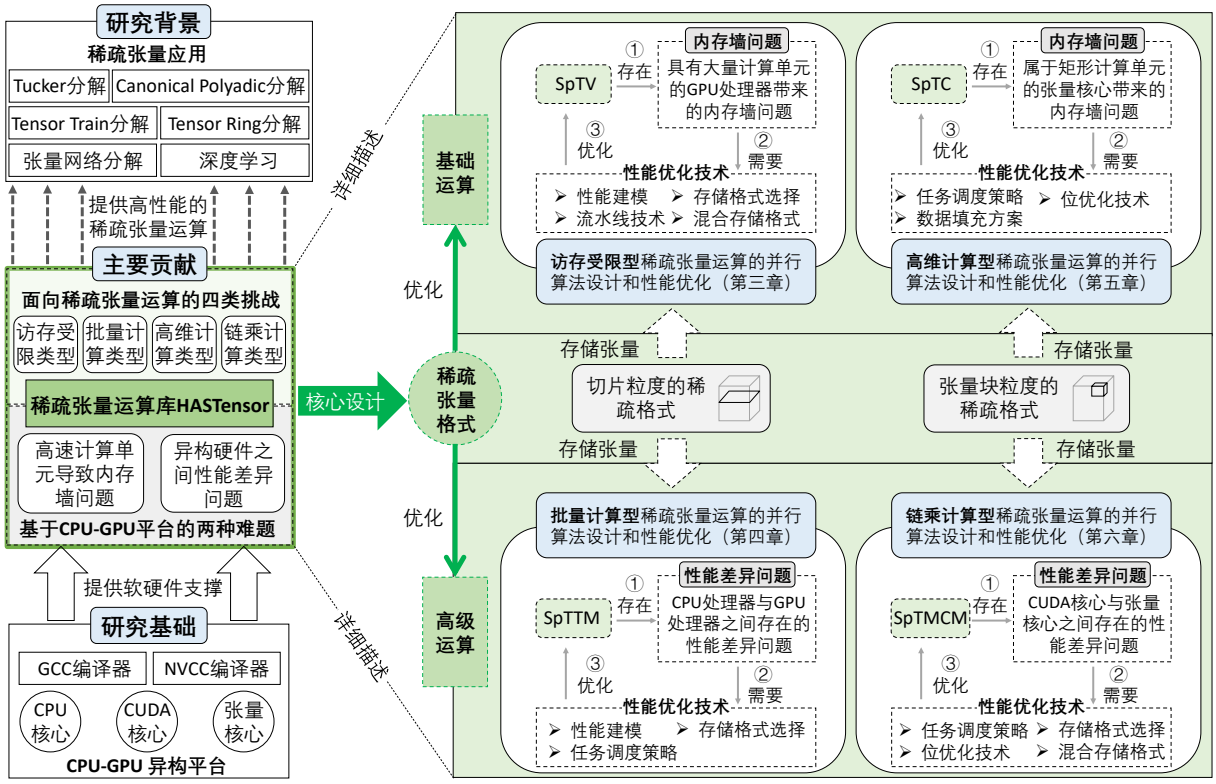


图 1.3　本文研究内容和稀疏张量运算库 HASTensor 的概述图

心，以提升计算速度。

2、支持输入感知格式：通过应用人工智能方法，可以捕捉和辨识输入张量的稀疏分布类型，并选用适当的稀疏格式存储张量数据。

3、支持混合存储格式：针对输入张量中复杂的稀疏分布情况，能够采用多个稀疏格式来分别存储部分数据，以提高各部分的计算性能，从而优化整体的张量运算效率。

4、支持自动性能调优：可应用性能建模方法为并行算法选择最佳的参数设置，以确保并行算法高效运行。

5、支持张量核心单元加速：能够利用张量核心单元加速计算，充分享受 GPU 上最新的矩形计算单元所带来的高速计算。

1.3.2　面向四类稀疏张量运算的并行算法设计和性能优化

本文将以四种不同类型的稀疏张量运算作为例子，详细介绍 HASTensor 所包含的方法和技术。这包括对访存受限型稀疏张量运算的并行算法设计和性能优化、批量计算型稀疏张量运算的并行算法设计和性能优化、高维计算型稀疏张量运算的并行算法设计和性能优化，以及链乘计算型稀疏张量运算的并行算法设计和性能优化的详细介绍。

中国知网　https://www.cnki.net

1、 访存受限型稀疏张量运算的并行算法设计和性能优化

SpTV 是 HASTensor 中的基础运算，也是典型的访存受限型稀疏张量运算，其计算效率通常受到数据移动和数据访问的制约。因此，本研究以 SpTV 为例进行详细介绍。为了充分发挥 GPU 的高速计算潜力，同时降低因数据传输引起的时间开销，本研究引入了一种输入感知和自适应流水线技术相结合的创新性方法，以缓解 GPU 的高速计算带来的内存墙问题。该方法融合了混合张量格式和流水线技术，通过动态选择最佳的存储格式和流水线数量，显著提升 SpTV 的执行效率。该方法的独特之处在于其适应性，不仅适用于各类数据类型和硬件环境，还能够较好地掩盖数据传输时间，从而增强计算性能和效率。

2、 批量计算型稀疏张量运算的并行算法设计和性能优化

SpTTM 是 HASTensor 中的高级运算，也是一种典型的具有批量计算特征的张量运算，其计算方式类似于批量的 SpTV 运算。因此，本研究以 SpTTM 为例进行详细介绍。随着处理器架构的不断演进，异构计算架构提供了多种解决方案，以优化具有批量计算特征的稀疏张量运算性能。本研究专注于 CPU-GPU 平台，为 SpTTM 设计了一系列专门的异构并行算法，以充分发挥异构平台上各硬件架构的独特特性。通过合理规划的任务映射策略、稀疏格式选择模型和内存访问模式的优化，本方法实现了稀疏张量乘以矩阵运算的高效并行计算。这一创新性的设计不仅显著提升了计算速度，同时也优化了计算资源的高效利用。

3、 高维计算型稀疏张量运算的并行算法设计和性能优化

张量的高维特性为设计高效的张量运算并行算法带来了全新的挑战，而这一问题在属于 HASTensor 中基础运算的 SpTC 中显得尤为突出。因此，本研究以 SpTC 为例进行详细介绍。为了解决高维计算特性引发的问题，本研究采用了分块技术（Tiling）和位操作技术，以减少高维张量的存储开销，从而显著提高内存访问效率，以应对张量核心的矩形计算所导致的内存墙问题。此外，本研究引入了一种创新的冲突解决方法，通过构建任务列表和消除任务之间的依赖关系，有效提高了线程块的利用率。通过合理设计高效的张量核心填充策略，以确保在处理各类不同的张量收缩任务时，张量核心得到较好的利用。这些策略不仅在充分发挥了新一代 GPU 性能优势的同时，也显著加速了高维稀疏数据的并行计算。

4、 链乘计算型稀疏张量运算的并行算法设计和性能优化

SpTMCM 作为链乘计算型张量运算的代表，包括多次连续的乘法运算，属于 HASTensor 中的高级运算。为了解决这一挑战，本研究以 SpTMCM 为例进行详细介绍。为了克服链乘计算带来的困难，本研究引入了一项创新性的统一计算方式，将计算粒度从非零值粗化到切片粒度，以适应张量核心的矩形计算方式。通过采用分块技术的混合张量格式和高效的内存访问模式，进一步实现了 SpTMCM 的

中国知网 https://www.cnki.net

高效并行算法。这一方法能够较好地利用 GPU 上不同计算核心的优势，从而实现对连续张量乘法运算的高速加速。

1.4 本文组织结构

本文共分为六个章节围绕稀疏张量运算库的设计和性能优化展开研究，针对 CPU-GPU 平台的体系结构特点，设计了稀疏张量运算库 HASTensor，包括先进的算法设计和性能优化的关键技术，并抽取了属于访存受限类型的 SpTV，属于批量计算类型的 SpTTM，属于高维计算类型的 SpTC，和属于链乘计算类型的 SpMTTKRP 和 SpTTMChain 作为具体的研究案例。

如图1.4所示，随着张量运算的乘数维度逐渐增加，张量运算的计算形式变得更加复杂。本文以此为线索，依次对访存受限型稀疏张量运算、批量计算型稀疏张量运算和高维计算型稀疏张量运算的并行算法设计和性能优化进行了深入研究。此外，随着乘数维度的增加，张量运算也呈现出更多的多样性，例如，乘数从一个矩阵扩展到多个矩阵的情况。因此，以计算强度的增加为线索，本文进一步深入研究了链乘计算型稀疏张量运算的并行算法设计和性能优化。

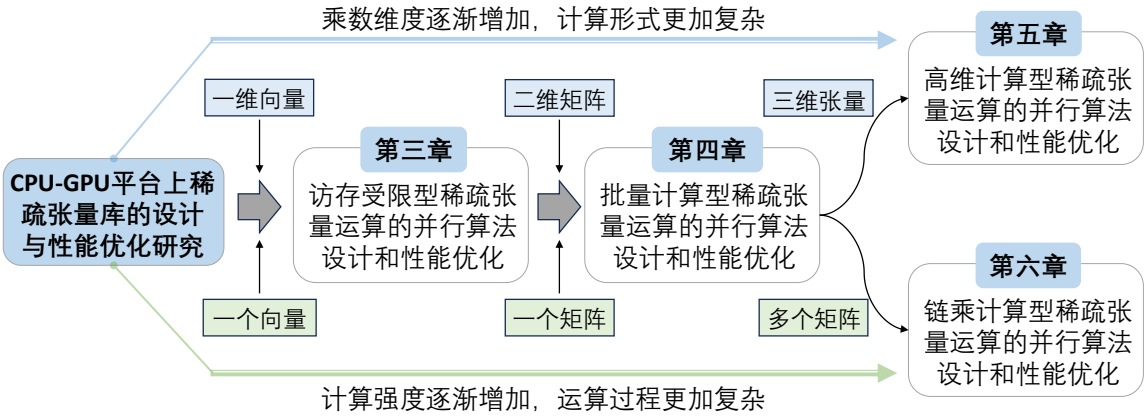


图 1.4　本文主要研究内容之间的关系结构图

全文共六章节，各章节的内容具体如下：

第一章，绪论。本章从稀疏张量应用的广泛性角度概括了稀疏张量运算，强调了稀疏张量运算的类型以及并行算法设计和性能优化的重要性。此外，分析了处理器架构从单一到异构的发展趋势，讨论了对稀疏张量运算库和性能优化的研究意义和挑战。还回顾了国内外相关研究，并介绍了本文贡献的稀疏张量运算库 HASTensor 的主要特点，最后给出了本文的结构安排。

第二章，张量运算背景。本章首先介绍了张量的定义和符号表示，然后阐述了稀疏张量运算的概念和计算示例，接着对不同的稀疏张量格式进行了分析和总结，并介绍了 CPU-GPU 平台上的编程基础知识。

第三章，访存受限型稀疏张量运算的并行算法设计和性能优化。以 SpTV 运

中国知网　https://www.cnki.net

算为例，提出了一种输入感知的自适应流水线 SpTV（Input-aware Adaptive Pipeline SpTV，IAP-SpTV）新方法，该方法基于混合存储格式和流水线技术，并采用图神经网络模型进行切片存储格式选择，同时利用性能建模来分析最佳流水线数量，从而全面提高访存受限型稀疏张量运算的计算性能。

第四章，批量计算型稀疏张量运算的并行算法设计和性能优化。以 SpTTM 运算为例，提出了一种异构逐切片的 SpTTM（Heterogeneous Slice-wise SpTTM，HS-SpTTM）新方法，该方法基于任务调度策略，并使用图神经网络模型选择张量存储格式，以及性能建模分析均衡的任务负载策略，从而全面提高批量计算型稀疏张量运算的计算性能。

第五章，高维计算型稀疏张量运算的并行算法设计和性能优化。以 SpTC 运算为例，提出了一种基于位坐标位图格式的 SpTC（Bit Coordinate Bitmap Format Based SpTC，BCB-SpTC）新方法，该方法基于任务调度策略，并采用位优化技术提高访存效率，同时设计张量核心的数据填充方案，从而全面提高高维计算型稀疏张量运算的计算性能。

第六章，链乘计算型稀疏张量运算的并行算法设计和性能优化。以 SpTMCM 运算为例，提出了一种基于混合线性格式的张量矩阵链乘算子（Hybrid Linear Storage Format Based Tensor Matrix Chain Multiplication Operation，HLSTO）方法，该方法基于混合存储格式和任务调度策略，并采用稀疏度阈值作为张量块存储格式选择方法，同时使用位优化技术提高访存效率，从而全面提高链乘计算型稀疏张量运算的计算性能。

总结与展望。本章对本文的工作进行了全面总结，同时对未来工作进行了展望，在稀疏张量库的设计和性能优化的技术问题和应用场景上列举了一些有前景的研究方向。

中国知网 https://www.cnki.net

第 2 章　张量基础概念和并行编程基础

张量作为高维数据分析和处理关键的数据组织形式，在众多领域中扮演着重要角色。本章旨在详细介绍张量的基本概念和并行编程的基础知识，为后续章节的研究提供必要的理论基础。

2.1　张量基础和相关符号

在物理学领域，张量通常用于描述物质的性质和各种物理现象，例如应力张量、电磁场张量等。而在数学领域，张量通常被抽象表示为多线性映射。本文研究的张量是计算机软件处理中多维数组的推广，是一种在多维空间中表达数据的数学对象和存储数据的数据结构。一维张量称为向量，二维张量称为矩阵。而高维张量则是在更多维度上扩展的数据结构，适用于多模态数据的表示和处理。本文遵循 Kolda 和 Sun 使用的张量定义和符号[96]。张量的每一个维度被称为模态，而张量的阶指的是维度或模态的数量，即一个 N 阶张量有 N 个维度或模态。

给定一个张量，通过不同的划分方式可以得到许多子张量，其中可能包括矩阵和向量。具体而言，对于一个三阶张量，通过固定其中两个模态的索引，可以获得一个被称为纤维的向量。通过固定其中一个模态，可以获得一个被称为切片的矩阵。在符号表示中，一个一阶张量可以表示为一个向量 v，也可以表示为矩阵的一行 $U(i,:)$ 或一列 $U(:,j)$，还可以表示为张量的一个纤维 $\mathcal{X}(i,j,:)$。类似的，一个二阶张量可以表示为一个矩阵 U，也可以表示为张量的一个切片 $\mathcal{X}(i,:,:)$。一个三阶或更高阶的张量表示为一个张量 $\mathcal{X}$。张量 $\mathcal{X}$ 在索引 (i,j,k) 的值表示为 $\mathcal{X}(i,j,k)$，矩阵 U 在索引 (i,j) 的值表示为 $U(i,j)$，向量 v 在索引 i 的值用 v_i 表示。

2.1.1　矩阵化

矩阵化，也称为展开或压平，是一种通过重新排列张量的值并将张量转换为矩阵形式的操作。一个三阶张量 $\mathcal{X} \in \mathbb{R}^{I\times J\times K}$，第一个模态的大小为 I，第二个模态的大小为 J，第三个模态的大小为 K。通过矩阵化成矩阵 $X_{(1)} \in \mathbb{R}^{I\times JK}$ 的操作记为模一矩阵化，矩阵化成矩阵 $X_{(2)} \in \mathbb{R}^{J\times IK}$ 的操作记为模二矩阵化，矩阵化成矩阵 $X_{(3)} \in \mathbb{R}^{K\times IJ}$ 的操作记为模三矩阵化。以一个例子为例，假设张量 $\mathcal{X} \in \mathbb{R}^{3\times 4\times 2}$ 的切片为：

$$\mathcal{X}(:,:,1) = \begin{pmatrix} 1 & 7 & 13 & 19 \\ 3 & 9 & 15 & 21 \\ 5 & 11 & 17 & 23 \end{pmatrix}, \mathcal{X}(:,:,2) = \begin{pmatrix} 0 & 6 & 12 & 18 \\ 2 & 8 & 14 & 20 \\ 4 & 10 & 16 & 22 \end{pmatrix} \tag{2.1}$$

中国知网　https://www.cnki.net

则张量 $\mathcal{X}$ 经过模一矩阵化后，得到矩阵 $X_{(1)} \in \mathbb{R}^{3\times 8}$ 为：

$$X_{(1)} = \begin{pmatrix} 1 & 7 & 13 & 19 & 0 & 6 & 12 & 18 \\ 3 & 9 & 15 & 21 & 2 & 8 & 14 & 20 \\ 5 & 11 & 17 & 23 & 4 & 10 & 16 & 22 \end{pmatrix} \tag{2.2}$$

2.1.2 稀疏张量乘以向量

张量乘以向量的基本形式是将张量中的值沿着一个模态与向量相乘，得到一个新的张量。如果一个张量是稀疏的，它被称为 SpTV。以三阶张量 $\mathcal{X}$ 为例，它是将张量 $\mathcal{X} \in \mathbb{R}^{I\times J\times K}$ 与向量 $v \in \mathbb{R}^{J}$ 在第二个模态上相乘的结果，表示为 $Y = \mathcal{X} \times_2 v$。结果是一个二阶张量 $Y \in \mathbb{R}^{I\times K}$，其计算的过程如下：

$$Y(i,k) = \sum_{j=1}^{J} \mathcal{X}(i,j,k)v(j) \tag{2.3}$$

假设向量 $v = [1 \quad 2 \quad 3 \quad 4]^{\top}$，根据公式2.3，则张量 $\mathcal{X}$ 与向量 v 在第二个模态上的乘积的结果为 $Y \in \mathbb{R}^{3\times 2}$，$Y$ 的值表示为：

$$Y = \begin{pmatrix} 130 & 120 \\ 150 & 140 \\ 170 & 160 \end{pmatrix} \tag{2.4}$$

2.1.3 稀疏张量乘以矩阵

张量乘以矩阵的基本形式是将张量中的值沿着一个模态与矩阵相乘，得到一个新的张量。如果参与计算的张量是稀疏的，则该稀疏张量乘以矩阵被称为 SpTTM。以一个三阶张量 $\mathcal{X}$ 为例，将张量 $\mathcal{X} \in \mathbb{R}^{I\times J\times K}$ 与矩阵 $U \in \mathbb{R}^{J\times R}$ 在第二个模态上乘积的结果，记为 $\mathcal{Y} = \mathcal{X} \times_2 U$。结果张量 $\mathcal{Y} \in \mathbb{R}^{I\times R\times K}$，其计算的过程为：

$$\mathcal{Y}(i,r,k) = \sum_{j=1}^{J} \mathcal{X}(i,j,k) \times U(j,r) \tag{2.5}$$

中国知网 https://www.cnki.net

例如，假设矩阵 $U \in \mathbb{R}^{4\times 6}$ 的值为：

$$U = \begin{pmatrix} 1 & 5 & 9 & 13 & 17 \\ 2 & 6 & 12 & 14 & 18 \\ 3 & 7 & 11 & 15 & 19 \\ 4 & 8 & 12 & 16 & 20 \end{pmatrix} \tag{2.6}$$

根据公式2.5，张量 $\mathcal{X}$ 与矩阵 U 在第二个模态上的乘积的结果为 $\mathcal{Y} \in \mathbb{R}^{3\times 6\times 2}$ 。结果张量 $\mathcal{Y}$ 模二矩阵化的矩阵 $Y_{(2)} \in \mathbb{R}^{6\times 6}$ 的值表示为：

$$Y_{(2)} = \begin{pmatrix} 130 & 150 & 170 & 120 & 140 & 160 \\ 290 & 342 & 394 & 264 & 316 & 368 \\ 450 & 534 & 618 & 408 & 492 & 576 \\ 610 & 726 & 842 & 552 & 668 & 784 \\ 770 & 918 & 1066 & 696 & 844 & 992 \\ 930 & 1110 & 1290 & 840 & 1020 & 1200 \end{pmatrix} \tag{2.7}$$

2.1.4 稀疏张量乘以稀疏张量

张量乘以张量运算又被称为张量收缩（Tensor Contraction），其基本形式是将张量中的值沿着一个或者多个模态与另一个张量相乘，从而得到一个新的张量。如果两个张量都是稀疏的，则它被称为 SpTC。以一个三阶张量 $\mathcal{X}$ 为例，将张量 $\mathcal{X} \in \mathbb{R}^{I\times J\times K}$ 与张量 $\mathcal{M} \in \mathbb{R}^{R\times J\times K}$ 在第二个和第三个模态上进行乘积运算，简称为模 $(2,3)$ 收缩，结果记为 $\mathcal{N} = \mathcal{X} \times_{2,3} \mathcal{Y}$。得到的结果张量 $\mathcal{N} \in \mathbb{R}^{I\times R}$，其计算的过程如下：

$$\mathcal{N}(i,r) = \sum_{j=1}^{J}\sum_{k=1}^{K} \mathcal{X}(i,j,k) \times \mathcal{M}(r,j,k) \tag{2.8}$$

例如，假设张量 $\mathcal{M} \in \mathbb{R}^{3\times 4\times 2}$，其模一矩阵化 $M_{(1)} \in \mathbb{R}^{3\times 8}$ 的值表示为：

$$M_{(1)} = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 \\ 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 \end{pmatrix} \tag{2.9}$$

根据公式2.8，张量 $\mathcal{X}$ 与张量 $\mathcal{M}$ 在第二个和第三个模态上进行乘积运算的结

中国知网 https://www.cnki.net

果张量为 $\mathcal{N} \in \mathbb{R}^{3\times3}$，其值为：

$$\mathcal{N} = \begin{pmatrix} 394 & 1002 & 1610 \\ 466 & 1202 & 1938 \\ 538 & 1402 & 2266 \end{pmatrix} \tag{2.10}$$

2.1.5 稀疏张量矩阵链乘

张量连续与多个矩阵的执行乘法操作的运算又被称为张量矩阵链乘。一些向量和矩阵之间的乘法运算也是张量矩阵链乘基本操作之一[57,97]，包括 Hadamard 乘积、Kronecker 乘积和 Khatri-Rao 乘积[98]。

Hadamard 乘积是一种二元运算，也称为逐元素乘积或 Schur 乘积，它接受两个具有相同维度的矩阵（或向量），并生成另一个与操作数维度相同的矩阵（或向量）。两个维度为 $\mathbb{R}^I$ 的向量 a 和 b 的 Hadamard 乘积是一个维度为 $\mathbb{R}^I$ 的向量 $c = a * b$，其元素为 $c_i = a_i \cdot b_i$。

Kronecker 乘积是对两个任意大小的矩阵（向量）进行的运算，得到一个矩阵（向量）。它是外积（用相同符号表示）从向量推广到矩阵的操作。两个维度分别为 $\mathbb{R}^I$ 和 $\mathbb{R}^J$ 的向量 a 和 b 的 Kronecker 乘积得到一个维度为 $\mathbb{R}^{IJ}$ 的向量 $c \in \mathbb{R}^{IJ}$，其定义和计算过程如下：

$$c = a \otimes b = \begin{bmatrix} a_1 b \\ a_2 b \\ \vdots \\ a_I b \end{bmatrix} \tag{2.11}$$

Khatri-Rao 乘积是两个矩阵中相应列向量的 Kronecker 乘积，表示为 $C = A \odot B$，其中矩阵 $A \in \mathbb{R}^{I\times K}$ 和 $B \in \mathbb{R}^{J\times K}$，其 Khatri-Rao 乘积的定义和计算过程如下：

$$C = A \odot B = \begin{bmatrix} a_1 \otimes b_1, & a_2 \otimes b_2, & ..., & a_K \otimes b_K \end{bmatrix} \tag{2.12}$$

其中 $C \in \mathbb{R}^{IJ\times K}$。

常用的张量矩阵链乘有两种形式，分别是用于 CP 分解的矩阵化张量乘 Khatri-Rao 积和用于 Tucker 分解的张量乘矩阵链。如果张量是稀疏的，则它被称为 SpTMCM，其两种计算形式分别被称为 SpMTTKRP 和 SpTTMChain。

以一个三阶张量 $\mathcal{X}$ 为例，如果张量 $\mathcal{X} \in \mathbb{R}^{I\times J\times K}$ 与矩阵 $B \in \mathbb{R}^{J\times R}$ 和 $C \in \mathbb{R}^{K\times R}$ 在第二个模态和第三个模态上执行矩阵化张量乘 Khatri-Rao 积的结果，

中国知网 https://www.cnki.net

记为 $A = \mathcal{X}\hat{\times}_2 B\hat{\times}_3 C$。结果矩阵 $A \in \mathbb{R}^{I\times R}$，其计算的过程为：

$$A(i,r) = \sum_{j=1}^{J}\sum_{k=1}^{K}\mathcal{X}(i,j,k) \times B(j,r) \times C(k,r) \tag{2.13}$$

如果张量 $\mathcal{X}$ 与矩阵 B 和 C 在第二个模态和第三个模态上执行张量乘矩阵链的结果，记为 $\mathcal{Y} = \mathcal{X} \times_2 B \times_3 C$。结果张量 $\mathcal{Y} \in \mathbb{R}^{I\times R\times R}$，其计算的过程为：

$$\mathcal{Y}(i,r,r) = \sum_{j=1}^{J}\sum_{k=1}^{K}\mathcal{X}(i,j,k) \times B(j,r) \times C(k,r) \tag{2.14}$$

例如，假设矩阵 $B \in \mathbb{R}^{4\times 2}$ 和 $C \in \mathbb{R}^{2\times 2}$ 的值分别为：

$$B = \begin{pmatrix} 1 & 5 \\ 2 & 6 \\ 3 & 7 \\ 4 & 8 \end{pmatrix}, C = \begin{pmatrix} 0 & 2 \\ 1 & 3 \end{pmatrix} \tag{2.15}$$

因此，根据公式2.13，张量 $\mathcal{X}$ 与与矩阵 B 和 C 在第二个模态和第三个模态上执行矩阵化张量乘 Khatri-Rao 积的结果矩阵为 $A \in \mathbb{R}^{3\times 2}$，其值表示为：

$$A = \begin{pmatrix} 135 & 1363 \\ 155 & 1623 \\ 175 & 1883 \end{pmatrix} \tag{2.16}$$

而根据公式2.14，张量 $\mathcal{X}$ 与矩阵 B 和 C 在第二个模态和第三个模态上执行矩阵化张量乘矩阵链的结果为 $\mathcal{Y} \in \mathbb{R}^{3\times 2\times 2}$。结果张量 $\mathcal{Y}$ 模一矩阵化的矩阵 $Y_{(1)} \in \mathbb{R}^{3\times 4}$ 的值表示为：

$$Y_{(1)} = \begin{pmatrix} 135 & 555 & 335 & 1363 \\ 155 & 655 & 387 & 1623 \\ 175 & 755 & 439 & 1883 \end{pmatrix} \tag{2.17}$$

2.2 稀疏张量存储格式

稀疏张量的存储格式主要分为两大类。第一类来自于稀疏矩阵存储格式的扩展，其主要思路是通过以矩阵格式保存张量的每个切片。第二类是针对高维张量设计的存储格式，对高维索引和数据的存储更友好。

中国知网 https://www.cnki.net

2.2.1 稀疏矩阵存储格式的扩展

张量的每个切片都是一个矩阵结构，因此，张量通过使用矩阵的存储格式保存切片并记录切片的索引，则稀疏矩阵的存储格式扩展为稀疏张量的格式。这类方法的优势在于可以有效的利用丰富的稀疏矩阵格式来压缩稀疏张量的存储空间。矩阵的稀疏格式探索有着悠久的研究历史，针对不同的稀疏结构，当前已经有许多稀疏格式被提出。除了将矩阵的值以稠密格式（Dense Format，Dense）直接存储外，坐标格式（Coordinate Format，COO）、压缩稀疏行格式（Compressed Sparse Row Format，CSR）、ELLPACK 格式（ELL-PACK Format，ELL）和混合 ELL/COO（Hybrid ELL/COO Format，HYB）都是经典的稀疏存储格式。如图 2.1 所示，对于张量中的一个切片，使用一个切片索引记录该切片在张量中的位置。每个切片中的值，则可以使用不同的矩阵格式进行存储。

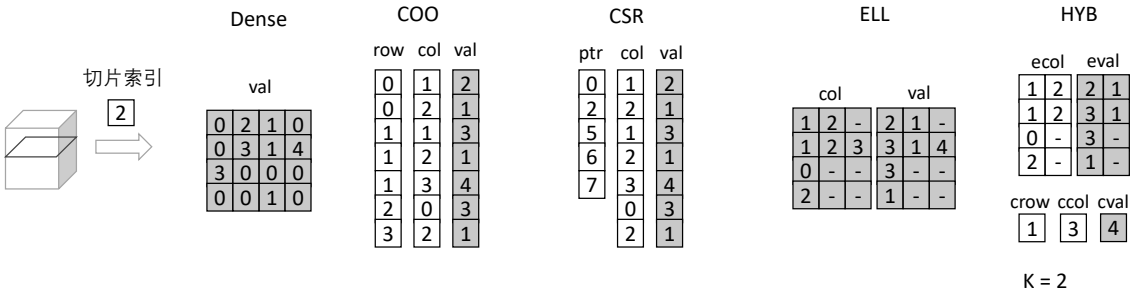


图 2.1 矩阵格式存储张量切片的示意图

1. Dense 格式中使用数组 val 以行优先或列优先的次序存储了矩阵中的所有元素值。这种方式是最简单且直观的存储方式，适用于稠密矩阵。然而，对于稀疏矩阵而言，它会占用大量的存储空间，在处理大规模数据时可能不太实际。
2. COO 格式直接存储非零值的索引和值。它使用了三个数组 row，col 和 val 来存储矩阵，每个元组表示一个非零值的位置和对应的值。这种格式在稀疏矩阵中非常适用，因为它只存储了实际存在的元素，从而降低了存储开销。
3. CSR 格式采用了行压缩技术，广泛应用于稀疏矩阵的存储中。它将矩阵数据按行进行划分，使用数组 ptr 来记录每行的偏移量，每一行存储了非零值的列索引 col 和对应的值 val。这种格式在并行计算中表现出色，因为每个处理单元可以独立地处理矩阵的不同行，实现了并行计算。
4. ELL 格式使用数组 col 和 val 来存储矩阵。数据数组存储了每行的非零值，而整数数组索引则存储了每个非零值的列索引。这种格式适用于那些非常稀疏但行长度相对均匀的矩阵。在某些并行计算环境中，它也能够提供出

中国知网 https://www.cnki.net

色的性能。

5. HYB 格式采用了混合的 ELL 和 COO，根据设定的阈值 K 来划分稠密部分和稀疏部分。对于小于阈值 K 的部分，使用 ELL 格式的数组 $ecol$ 和 $eval$ 来存储，而对于超出阈值 K 的部分，使用 COO 格式的数组 row，col 和 val 来存储。这种混合方式较好地利用了两种格式的优势，既能有效地存储稠密数据，又能高效地处理稀疏数据，从而提高了矩阵处理的灵活性和性能。

2.2.2 高维稀疏张量存储格式

高维稀疏张量存储格式的设计旨在解决高维特性导致的大量索引的挑战。其中最常用的格式是 COO，它通过记录每个非零值在不同模态的位置作为索引来存储稀疏张量。COO 格式被广泛用于稀疏张量数据的存储和处理，因其直观性而著称。在 COO 中，数据点以元组形式表示，每个元组包含非零值的坐标索引和相应数值。这种格式在处理高维稀疏数据时表现出显著的优势。

使用 COO 张量格式执行张量运算是一种高效处理稀疏张量数据的方法。它以单个元素为基本单位进行计算，能够有效地处理张量中的非零值。COO 的一个显著优点是其对张量稀疏模式的表示既简单又不受模态变化的影响。这意味着，不论稀疏张量的模态如何变化，均可采用 COO 格式进行数据的存储和处理，而无需修改存储结构。

然而，COO 在存储张量数据时，其多维索引通常需要占用较大的内存空间，这对于大规模稀疏张量可能构成挑战。此外，在 GPU 等并行计算设备上执行 COO 格式的计算通常需要依赖原子操作，这可能会影响计算效率。为了克服 COO 的一些缺点，学术界提出了一系列 COO 的变体，包括标记坐标格式（Flagged-Coordinate Format，F-COO）[59]，层次坐标格式（Hierarchical Coordinate Format，HiCOO）[58]，分块线性化坐标格式（Blocked Linearized Coordinate Format，BLCO）[49]，压缩稀疏纤维格式（Compressed Sparse Fiber Format，CSF）[41] 等。这些变体旨在改善稀疏张量在内存占用和计算效率方面的性能，以满足不同应用场景对稀疏数据处理的需求。图 2.2展示了 COO 以及其变体的形式的张量示例。

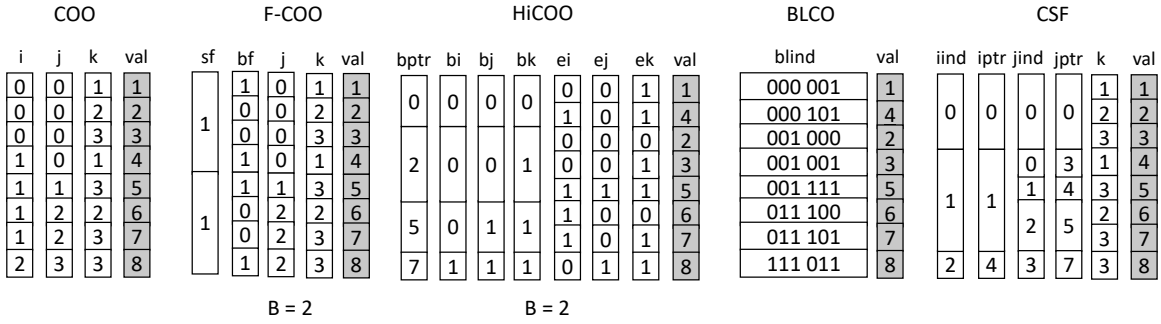


图 2.2 张量存储格式的示意图

中国知网 https://www.cnki.net

F-COO 引入了两个关键的标志数组，起始标志为 sf 和位标志为 bf。这些标志用于跟踪切片的索引模态是否在块的起始或元素级别发生变化。在 SpTMCM 中，当索引模式 i 发生变化时，bf 从 1 变为 0。sf 表示当前线程是否在前一个线程的索引模式上启动了新的索引。线程 0 的 Sf 总是 1，因为它始终启动新的索引模式。这两个标志数组的引入用于引导段扫描，以避免在 GPU 上执行 MTTKRP 中的原子操作，从而提高了计算效率。

HiCOO 采用了一种层次化的方法，将每个维度划分为大小为 B 的块，并利用更为紧凑的比特压缩非零元组。在 HiCOO 的设计中，$bptr$ 数组被用于记录块的起始位置以及每个非零值的索引。此外，HiCOO 将这些块组织成一个大型块，以有效地规避写冲突的问题。在新构建的大型块总是以集中的方式进行调度和分配给单个线程，进一步提高了并行计算的效率，并消除了可能出现的竞争条件。

BLCO 格式的特点在于以一维数据结构的方式来组织和存储张量中的非零值，同时保留了紧凑的索引数据。这种设计使得内存中相邻的非零值能够彼此更为接近，从而提高了数据访问的效率。BLCO 格式采用了自适应位编码方案，该方案基于目标张量的形状和尺寸将索引数组转换成线性数组 $blind$ 和值数组 val。这种自适应性意味着 BLCO 格式能够根据具体的张量特征来灵活地生成索引信息，而不需要固定的索引结构。这种自适应性是 BLCO 格式与众不同之处，使得该格式在索引计算方面表现出色, 例如去线性化。其设计旨在在降低内存使用率的同时更有效地利用内存带宽。通过将相邻的非零值存储得更加接近，并采用自适应的位编码方案，BLCO 格式在稀疏数据的存储和处理方面取得了平衡，以满足高性能计算的需求。

作为一种稀疏矩阵存储格式，CSF 格式采用了基于树的结构，包含 6 个关键数组。其中，$iptr$ 数组用于记录切片中第一个纤维的位置，而 $jptr$ 数组则追踪每个纤维中第一个元素的位置。数组 $iind$，$jind$，k 和 val 则分别存储了元素的相应索引和值。相较于传统的 COO 格式，CSF 格式在内存占用和缓存性能方面具有显著优势。这得益于 CSF 采用了高效的切片和纤维压缩技术，使得在存储稀疏数据时能够更加高效地利用内存，并且在数据访问时具备更好的缓存命中率。此外，CSF 格式还支持根据第二种模式使用分块技术来对矩阵因子进行划分。这种分块技术使得矩阵因子可以分布在不同的线程之间，从而消除了对锁（Lock）的需求，提高了并行计算的效率。然而，值得注意的是，尽管 CSF 在内存和缓存性能方面表现优越，但由于其基于树的结构，执行张量操作时可能需要借助递归算法的支持，在 GPU 上运行时可能效率不高。

为了更好地适应 GPU 等并行计算硬件，CSF 格式进行了扩展，产生了多种变种格式，如 B-CSF[28]、MM-CSF[48] 等。B-CSF 采用了纤维分裂和切片分裂等细粒度张量划分技术，有效提高了 GPU 的内存效率，并获得了良好的负载平衡。而 MM-CSF 则是一种混合模态张量表示方法，当切片仅包含一个元素时，采用 COO

中国知网 https://www.cnki.net

格式存储，而当切片包含多个纤维，并且每个纤维只包含一个元素时，则应用 CSL 格式。对于其他情况，MM-CSF 依然采用 CSF 格式。此外，MM-CSF 还将张量的非零单元划分为不相交的部分，并对每个部分进行压缩，以生成不同模态的纤维，从而进一步提高了 GPU 上的计算性能。

2.3 编程基础

2.3.1 NVIDIA 架构介绍

最初 GPU 的应用仅限于计算机图像显示，然而在 2003 年，随着研究人员开始尝试使用 GPU 加速并行计算，NVIDIA 等主要的 GPU 供应商意识到了其潜在的替代用途。NVIDIA 通过 CUDA 平台提供了一种解决方案，使开发人员能够较好地利用 GPU 的强大计算能力来加速各种应用程序。

如图2.3所示，CUDA 线程在执行期间可以访问多个内存空间中的数据。每个 CUDA 线程都有私有本地内存。每个线程块都有对该块的所有线程可见的共享内存，并且与该线程块具有相同的生命周期。线程块簇中的线程块可以对彼此的共享内存执行读、写和原子操作。所有 CUDA 线程均可访问相同的全局内存。

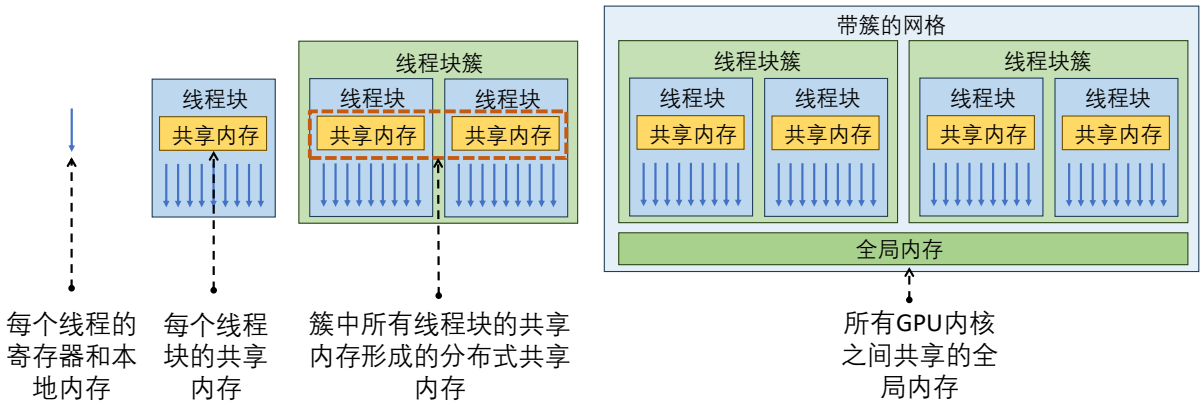


图 2.3 GPU 内存层次的示意图

GPU 的存储器类型多样，包括寄存器、本地内存、共享内存、常量内存、纹理内存和全局内存，每种类型都有其独特的优缺点和适用场景。

1、寄存器是访问最快的内存类型，在每个线程中存储自动变量。然而，每个线程块的寄存器数量有限，可能限制活跃线程数。本地内存的访问延迟与全局内存类似，因此较少使用。

2、共享内存是线程块级别的片上内存，具有高带宽和低延迟的特点。它可用于线程块内部的协作通信和片上数据共享，减少对全局内存的访问。但如果线程块需要大量共享内存，会影响可调度的并发线程块数，从而降低性能。

3、常量内存和纹理内存为只读，常用于特定场景且速度比全局内存快。常量内存提供只读虚拟地址形式的全局内存，而纹理内存针对二维数据进行优化读

中国知网 https://www.cnki.net

取，并具有只读缓存。

4、全局内存是网格中所有线程可访问的读写内存，虽然访问延迟较高，但容量最大。全局内存通过二级缓存进行访问，一级缓存的性能取决于 GPU 架构和程序设置。

图2.4中呈现了 NVIDIA A100 GPU（以下简称 A100）的流多处理器架构，它由四个子核心组成。每个子核心包括线程束调度器和数学调度单元，用于高效分配和管理并行任务，从而显著提高数据处理效率。此外，子核心内部设有流处理器阵列，也称为 CUDA 核心，负责多种数据类型的高速处理。这些阵列布局紧密、并发性强，可快速执行各种计算。



图 2.4　NVIDIA A100 中流多处理器架构的示意图

每个子核心集成了特殊功能单元、张量核心单元、数据加载/存储单元以及寄存器文件等高级组件。这些组件在特定应用场景下具有至关重要的功能。例如，张量核心能够高效执行矩阵积累操作，大大缩短了深度神经网络训练所需的时

中国知网　https://www.cnki.net

间。

另外，共享内存和L1数据缓存位于四个子核心之间的流多处理器内，实现了资源的高效共享。这一设计使得数据能够在不同核心之间流畅传递，极大地提升了系统的吞吐量。

具体而言，共享内存被划分为多个存储体，其目的在于实现片上内存的并行访问。通过将数据分散到不同的存储体中，可以同时读取和写入不同的存储体，从而提高共享内存的访问效率。然而，存储体之间的并行访问可能导致冲突。若多个线程同时访问同一个存储体，将会引发冲突问题，从而导致性能下降。为了解决这一问题，GPU架构通常采用一种称为存储体冲突解决方案的机制。具体而言，该机制将共享内存的存储体划分为更小的单位，以更好地支持并行访问。这样，即使多个线程同时访问同一个存储体，也可以通过分别访问不同的子存储体来避免冲突。

自伏打（Volta）架构以来，NVIDIA已引入了张量核心以处理稠密矩阵计算。在安倍（Ampere）架构中的A100型号GPU上，流多处理器相较于Volta和Turing架构的流多处理器，进一步提升了计算性能并引入了多项新功能。

每个A100流多处理器包含四个张量核心，而Volta和Turing的每个流多处理器则包含八个张量核心。在每个时钟周期内，每个A100张量核心能够执行256个16位浮点数（FP16）与32位浮点数（FP32）的融合乘加（Fused Multiply Accumulate，FMA）操作，而相较之下，Volta和Turing的每个张量核心仅执行64个FMA操作。因此，A100流多处理器提供更高的计算吞吐量，每个流多处理器在每个时钟周期内能够执行1024个密集的FP16/FP32 FMA操作，相对于Volta和Turing，性能提升了2倍。

借助这些改进和增强，A100流多处理器实现了更高效的计算和数据处理。引入张量核心使得每个时钟周期内能够处理更多的矩阵运算，进一步提高了GPU的性能。这些架构设计的改进为科学研究和计算提供了更为强大的计算能力和吞吐量。

2.3.2 CUDA编程模型

CUDA是由NVIDIA开发的一种并行计算平台和编程模型。它允许开发人员将通用计算任务映射到GPU上，并实现高性能的并行计算。CUDA编程模型基于主机-设备模型，其中主机函数表示运行在CPU上的程序，设备函数表示运行在GPU上的程序。线程（Thread），线程束（Thread Warp），线程块（Thread Block），线程网格（Thread Grid）是CUDA编程模型中调度计算任务的抽象概念，其具体介绍如下：

1. 线程是最小的执行单元。线程可以在GPU上独立执行，并且能够访问设备上的全局内存。线程可以以多维的方式进行索引，以适应各种计算任务。

中国知网 https://www.cnki.net

2. 线程束是 GPU 中的基本执行单元，线程束调度器依据线程束的状态进行调度，同一个线程束中所有的线程按照 SIMT 方式执行。
3. 线程块是一组相关联的线程。块中的线程可以协同工作，在共享内存中进行通信和数据共享。块是 GPU 调度计算任务的最小单位。开发人员可以根据任务的特点，合理配置块的大小。
4. 线程网格是一组相关联的块。网格中的块可以独立执行，并且能够通过全局内存进行通信和数据共享。开发人员可以根据任务的规模，合理配置网格的大小。

在处理控制流时，CPU 和 GPU 表现出明显的差异。CPU 拥有复杂的分支预测机制，可以预测控制流指令的结果，并提前执行最有可能的操作路径，从而有助于提高程序的运行速度。然而，分支预测错误会引入执行延迟。相比之下，GPU 采用线程束作为执行单元，要求线程束内的所有线程在同一周期内执行相同的指令。当线程束内的线程需要执行不同的指令时，会导致线程束的分化，迫使 GPU 停用不符合当前分支条件的线程，从而影响指令吞吐量和性能。为了实现最佳性能，GPU 编程通常需要避免线程束分化，并采用组织线程束以及优化控制流的策略，以确保线程束内的线程具有相同的执行路径，以提高性能。

在进行并行计算时，通常遵循以下五个步骤。

第一步，需要在主机端分配内存来存储输入和输出数据，并将数据传输至设备的全局内存中，以确保设备可以访问所需数据。

第二步，需要指定适当的网格和块大小的配置，以配置即将在设备上执行的核函数。通过合理的配置，可以充分发挥设备的并行处理能力，以提高计算效率。

第三步，一旦配置完成，可以将核函数发送至设备并执行。每个线程将根据其在网格和块内的索引执行相应的计算。开发人员可以通过核函数的参数向线程传递所需的输入数据，以满足各种计算需求。

第四步，在核函数执行完毕后，需要将计算结果从设备的全局内存传输回主机内存，以进行后续处理和分析。

第五步，在计算结束后，必须释放之前分配的内存空间，以确保资源得到正确释放。

2.3.3 张量核心编程模型

张量核心编程模型在 GPU 上的矩阵计算中扮演着重要的角色，极大地推动了机器学习、深度学习和高性能计算的进步。相较于每个时钟周期执行一个浮点运算的 CUDA 核心，张量核心在每个时钟周期内可执行一个矩阵与矩阵的乘加操作（$D = C + A \times B$），其中 A, B, C 和 D 均为大小为 4×4 的矩阵。

为了简化张量核心的编程和使用，NVIDIA 提供了线程束级矩阵乘法和累加（Warp-level Matrix Multiply and Accumulate，WMMA）接口。WMMA 接口为开发

中国知网 https://www.cnki.net

人员提供了专门的矩阵加载和存储原语，旨在优化数据在寄存器文件和内存层次结构之间的传输，并可执行张量核心的矩阵乘法和累加运算。

通过 WMMA 接口的应用，开发人员能够轻松高效地利用张量核心的强大计算能力。首先，WMMA 接口提供了矩阵加载原语，使得从内存到寄存器文件的数据传输更加高效。该加载原语能够自动将矩阵数据分块加载到张量核心的寄存器文件中，以减少访问内存的次数和延迟。

此外，WMMA 接口还提供了专门的矩阵存储原语，以便将张量核心计算结果快速准确地写回到内存中。这些存储原语能够将结果数据以块的形式存储到内存中，从而最大限度地减少存储操作的开销，提高整体性能。

除了矩阵加载和存储原语，WMMA 接口还允许开发人员直接调用张量核心执行矩阵乘法和累加运算。这种紧密集成的设计使得矩阵运算的执行更加高效，同时减少了数据传输和处理的复杂性。通过使用 WMMA 接口，开发人员可以充分发挥张量核心的并行计算能力，实现更快速的矩阵运算。

如图2.5所示，使用张量核心进行矩阵乘法可以分为三个主要阶段。

在第一阶段，初始化 WMMA 接口使用的专用寄存器（Segment）$Afrag$、$Bfrag$ 和 $Cfrag$。

在第二阶段，通过使用 $wmma::load_matrix_sync()$ 命令，将输入矩阵 A 和 B 加载到 $Afrag$ 和 $Bfrag$ 中，如果矩阵 D 中有初始值，则使用 $wmma::load_matrix_sync()$ 命令将 D 中的数据存入 $Cfrag$ 中。

在第三个阶段，通过调用 $wmma::mma_sync()$ 命令，执行矩阵与矩阵的乘法操作。随后，将计算结果从 $Cfrag$ 传输到位于 GPU 全局内存中的矩阵 D 中。

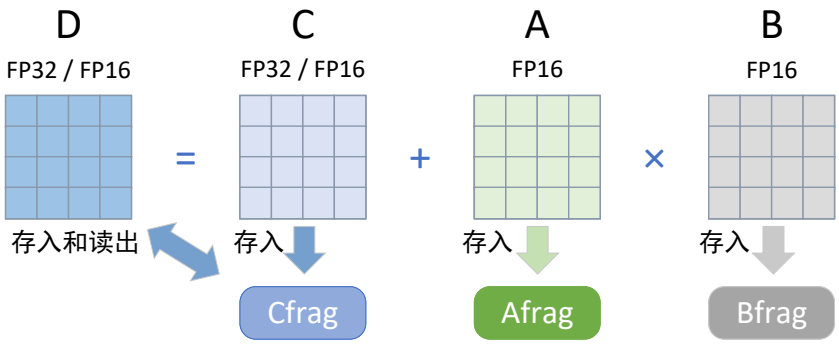


图 2.5　张量核心上的矩阵乘加示意图

2.3.4　CPU-GPU 异构并行编程

在现代计算环境中，随着多核技术的不断发展，CPU 核心数量的增加已达到 8 核、32 核甚至更多的配置，极大地提升了 CPU 在并行计算方面的潜力[99]。尽管 CPU 在通用计算领域表现出色，然而 GPU 在浮点运算速度方面仍占据优势。尽管 GPU 的每秒浮点运算次数远高于 CPU，但在 CUDA 编程模型中，GPU 缺乏进

中国知网　https://www.cnki.net

程控制的能力，通常是由 CPU 来控制[100]。在这一背景下，构建异构编程环境，较好地利用 CPU 和 GPU 的各自优势，弥补彼此不足之处，已经被广泛认可。

在这一异构编程模型中，数据通过高速串行计算机扩展总线标准（Peripheral Component Interconnect express，PCIe）从 CPU 传输至 GPU，然后由 CPU 调度 GPU 上的核函数进行计算过程。同时，采用共享存储并行编程框架（Open Multi-Processing，OpenMP）在 CPU 上实现多线程并发，其中一个线程负责协调 GPU 的运算，而其他线程在剩余的 CPU 核心之间均衡分担工作负载。为了更精确地协调任务分派，数据需求被分割成两个部分，并分别分配给 CPU 和 GPU。然后，通过 OpenMP 创建两组线程，一组用于 GPU 的管理与调度，另一组则在 CPU 上负责任务的并行执行。

CUDA 流（CUDA streams）作为一个关键概念，在 CPU-GPU 混合编程中发挥着关键作用。CUDA 流允许不同的计算任务被异步提交到 GPU，提高了并行性，使 GPU 可以同时执行多个任务。这种并行性对于较好地利用 GPU 的计算资源至关重要，特别是在需要处理大规模数据集或进行复杂计算的情况下。

在这一混合编程模型中，开发人员可以创建多个 CUDA 流，每个流代表一个独立的任务队列。通过将不同的计算任务异步提交到这些流中，可以实现任务的并行执行。这些任务可以是不同的核函数调用，也可以是数据传输操作，如从 CPU 到 GPU 的数据传输或从 GPU 到 CPU 的数据传输。通过合理使用 CUDA 流，可以实现任务的最大并行性，从而提高程序的整体性能。此外，CUDA 流还提供了同步机制，允许开发人员在流之间进行同步，以确保任务按照特定的顺序执行。这对于一些需要特定顺序执行的任务非常重要，同时也提供了更精细的控制能力。

总而言之，CUDA 流作为 CPU-GPU 混合并行编程的关键工具之一，使得开发人员能够更好地管理和调度 GPU 上的计算任务，从而高效地利用 GPU 的性能。通过合理使用 CUDA 流，结合 CPU-GPU 协同工作，可以实现高效的任务并行处理，这在处理复杂计算和大规模数据集时尤为重要。尽管这种编程模型仍然面临一些挑战，如数据传输和同步，但它已被广泛应用于科学研究、深度学习、高性能计算等领域，为加速计算任务的执行提供了强大的工具。

中国知网 https://www.cnki.net

第 3 章　访存受限型稀疏张量运算的并行算法设计和性能优化

本章以 SpTV 为例，重点介绍了 HASTensor 内关于访存受限型稀疏张量运算的并行算法设计和性能优化方法。SpTV 作为一种典型的访存受限型张量运算，同时也是张量分解算法的主要计算方式之一。由于稀疏张量的非零值呈现出复杂的分布特性，因此在 CPU-GPU 异构架构上优化 SpTV 计算性能面临着重大挑战。

为了克服这一挑战，采用了一种输入感知的自适应流水线 SpTV 方法，即 IAP-SpTV，以在 CPU-GPU 异构平台上加速 SpTV 运算。首先，设计了一种名为混合张量格式（Hybrid Tensor Format，HTF）的数据表示形式，并且探讨了基于该格式的流水线 SpTV 算法所面临的挑战。HTF 属于 HASTensor 中切片粒度的存储格式。通过以混合格式的形式存储张量数据，可以更充分地利用不同稀疏格式的优势，从而提高整体的计算性能。然后，构建了一个用于切片格式选择的图卷积神经网络模型（Graph Convolutional Network for Slice Format Selection，Slice-GCN），用于选择适合于切片的稀疏张量表示格式。该模型的目标是克服在使用 HTF 格式时需要为每个切片选择合适格式的问题，从而更有效地利用混合格式的优势。随后，建立了流水线性能模型，旨在实现在 IAP-SpTV 计算过程中传输时间和计算时间的最大重叠。通过合理的流水线设计，可以进一步减少整体计算时间，提高计算效率。最后，在两种不同的 CPU-GPU 异构平台上进行了实验，以验证 IAP-SpTV 方法的正确性、有效性以及在不同平台之间的可移植性。实验结果表明，所提出的方法在不同硬件环境下均取得了显著的性能提升，充分证明了其在优化稀疏张量乘法计算方面的潜力。

3.1　引言

张量，作为广泛应用于大数据领域的高维数据表示形式[101]，其分析与应用一直是数据分析的基本问题之一。在数据分析领域，张量分解是一种常用技术，而 SpTV 运算则是其中的关键环节，但一直受制于性能瓶颈。为解决 SpTV 性能问题，早期方法主要采用以下三种策略之一：

1、将张量转换为等价的矩阵，然后利用张量工具箱[102] 中的 SpMV 的方法。这种方法借鉴了现有的矩阵优化技术，但在处理过程中但需要昂贵的数据转换成本。

2、采用了以非零值为计算粒度的 ParTI! 方法[103]。然而，该方法中必要的原

中国知网　https://www.cnki.net

子更新操作导致了额外的等待时间[24]。

3、基于流水线技术和混合格式的 HOCFS 方法[22]。该方法根据经验设置流的数量，并使用两种格式的混合格式。然而，该传统方法需要大量实验来确定最佳流的数量，并且混合格式的适用性有限。

相对于上述方法，本研究考虑引入一种新的基于切片的混合格式，该混合格式可以为切片提供更多的格式选择，以增强对稀疏结构的适应性。然后通过流水线技术将数据传输和计算的执行时间重叠，以优化整体计算效率。此外，各种稀疏格式适用于不同类型的稀疏分布数据，选择不适当的稀疏格式会导致数据局部性不足和性能下降。尽管如此，为各种类型的稀疏数据选择合适的格式仍然是可行的，这为本研究进一步优化 SpTV 计算提供了可能性。

深度学习的迅速发展为张量格式选择带来了新的机遇[104]。在大规模线性系统和张量分解的迭代过程中，机器学习和深度学习选择张量格式的额外成本通常会由多次 SpTV 迭代中分摊。近期的研究中，采用深度学习方法来选择张量的稀疏格式，例如利用卷积神经网络（Convolutional Neural Network，CNN）为张量选择合适的格式。然而，这种方法需要预先将张量缩放为固定大小的矩阵，对于大尺度的张量可能会导致丢失张量非零值稀疏分布的特征，进而使得所选的存储格式不是最优的。图卷积神经网络（Graph Convolutional Network，GCN）为切片选择最佳稀疏格式提供了一种潜在方法，它已在结构化数据上已经展现出强大的分类性能[105]。

流水线技术是一种用于在 CPU-GPU 上加速计算的技巧，其核心思想是通过重叠 CPU-GPU 上的计算时间和数据传输时间来覆盖通信开销[106]。流水线计算模式的实现依赖于 CUDA 流，而确定最佳的流数量是一个值得研究的问题。较少的流数可能无法充分覆盖数据传输和计算时间，而使用过多的流则会导致不必要的启动流开销。

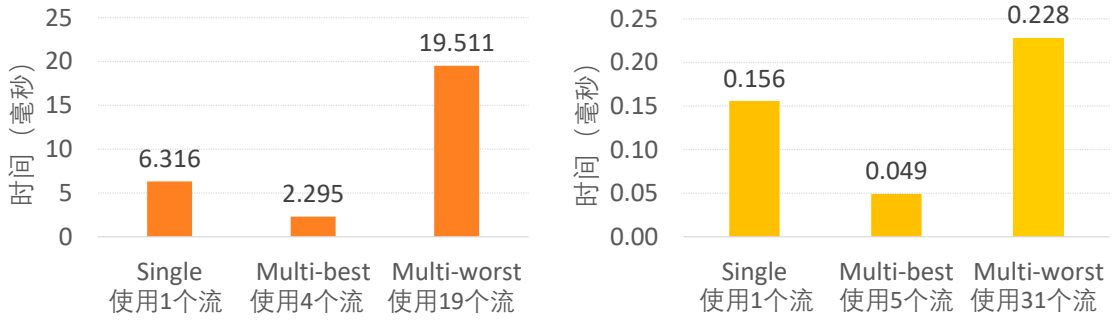
如图3.1所示，在最大流数限制为 32 的情况下，不同流设置下的 SpTV 性能存在显著差异。可以清晰地观察到，相对于仅使用一个流（Single），使用合理的流数量（Multi-best）能够显著提高性能。然而，使用过多的流数量（Multi-worse），性能反而会下降。在通常的流水线计算模式中，经验性地确定流的数量是常见的做法。然而，由于张量具有高维度和复杂的稀疏结构，给经验性地设定流数量的方法带来了新的挑战。

为了解决上述问题，本研究提出了一种基于 CPU-GPU 的自适应流水线 SpTV 算法，简称 IAP-SpTV。具体而言，本章的主要贡献如下：

1、 本章提出了一种稀疏张量存储格式 HTF，并基于 HTF 设计了一种自适应流水线 SpTV 算法。该算法通过流水线技术，以掩盖数据在 CPU 和 GPU 之间的传输开销。

2、 本章构建了一个用于稀疏格式选择的模型 Slice-GCN。该模型能够有效地

中国知网 https://www.cnki.net



(a) CSR 格式　　(b) ELL 格式

图 3.1　不同 CUDA 流数量对应 SpTV 性能的结果图

识别切片的稀疏类型，为切片选择适当的存储格式。

3、 本章开发了一个流水线性能分析模型，用于确定每种稀疏格式上 CUDA 流的数量，以最大化流水线期间数据传输与计算的并行性。

4、 本章使用真实数据集进行广泛的实验验证，证实了 IAP-SpTV 方法在提高计算效率以及可移植性方面的优势。

3.2　HTF 格式及其流水线 SpTV 算法

在存储和处理张量时，通常面临有效利用内存空间和高效处理数据的挑战。传统的单一格式存储非零值未充分考虑张量的空间结构，导致内存访问延迟问题[107]。尽管存在多种稀疏存储格式，但都未较好地利用张量的局部稀疏性。为解决此问题，本研究提出了一种名为 HTF 的压缩存储格式，并设计了基于 HTF 的流水线 SpTV 算法。

3.2.1　HTF 存储结构

本研究着重于较好地利用张量切片的局部稀疏性，而非整个张量的全局稀疏性。HTF 格式具备以下主要特征：1、优化了局部切片的稀疏性，而不是整个张量的全局稀疏性；2、以最适宜的格式存储稀疏张量的各个切片；3、为每一类稀疏格式选取相应的核函数进行计算，以高效地利用数据局部性。

HTF 格式将整个张量划分为多个等尺寸的切片，每个切片都以 CSR、COO、ELL、HYB 以及 Dense 这五种中的一种格式来存储，如图3.2所示。由于 SpTV 中张量与向量在不同模态下的乘积计算过程相似，为简化符号表示与减少冗余，本研究以张量与向量在第一模态下的乘积 SpTV 为例来描述 HTF 格式。此时一个切片的大小为 $J \times K$。HTF 采用三个数组记录各个切片的格式，$sliceIdx$ 数组表示每个切片的索引，$sliceNnz$ 数组表示每个切片中非零值的起始位置，$format$ 数组则存储了相应类型的格式信息。

中国知网　https://www.cnki.net

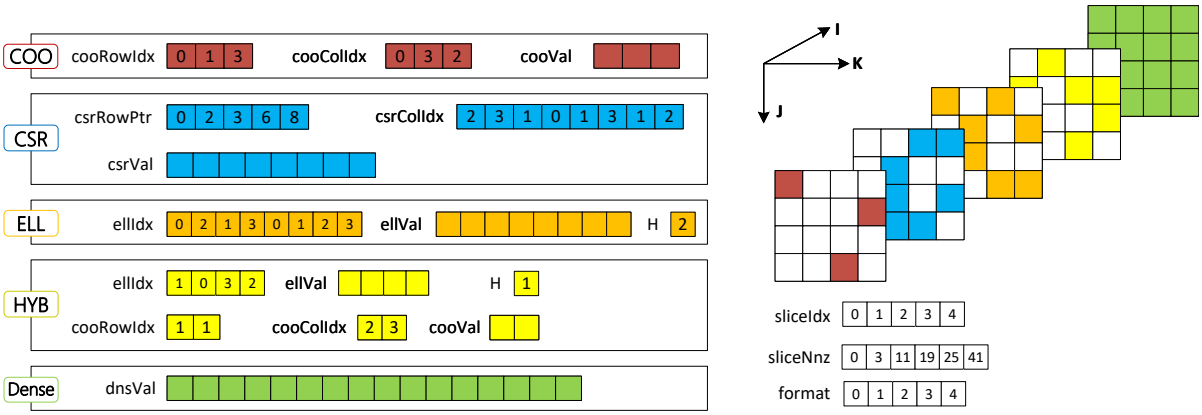


图 3.2　HTF 格式储存张量的示意图

3.2.2　基于 HTF 格式的流水线 SpTV 算法

为了提高 SpTV 计算过程中 CPU 和 GPU 之间的通信带宽，本章设计了一种基于 HTF 格式的流水线 SpTV 算法。由于张量数据的稀疏和高维特性，导致了较长的数据传输时间，而 GPU 上的计算则涉及大量的不规则计算和不连续的内存访问。为了应对这些问题，该算法采用了多流水线的方式来配置 CUDA 流，以实现数据传输和计算过程的并行执行，从而降低整体处理时间。具体而言，基于 HTF 的流水线 SpTV 算法将整个张量分割为多个切片，并以批处理方式将数据从 CPU 传输到 GPU，以确保在计算上一批数据时可以传输当前批次的数据。

在详细介绍具体算法之前，需明确切片模态的选择。为确保切片模态对性能产生积极影响，最小尺寸的维度被选为切片模态。这种选择可生成包含更多非零值的大型切片，同时减少了切片的数量。更多的非零值意味着更高的数据重用概率，而较少的切片数量则降低了格式选择的成本[108]。因此，在执行 SpTV 操作之前，除计算模态外，将尺寸最小的维度选为切片模态。

基于 HTF 的流水线 SpTV 算法的详细步骤见算法3.1。该算法接受四个输入参数，分别是 HTF 中五种格式的切片集合、乘法向量、乘法模式以及每种格式所需的 CUDA 流数量。算法首先在第 1 行初始化用于存储结果的矩阵空间。接下来，在第 2 行至第 4 行，将五个格式化的切片集合、结果集合和流编号集合各自存入单独的数组中，以便后续的遍历操作。在第 5 行，算法将向量 v 从 CPU 传输到 GPU。接着，在第 6 行和第 7 行，根据每种格式的最大流数初始化 CUDA 流数 N_{max}，以便后续的流重用，从而降低了计算的复杂性。从第 8 行至第 23 行，算法逐一计算不同格式的切片集和向量的乘积。这一过程以顺序执行的方式执行，换而言之，在处理当前格式的所有切片之前，会先处理上一个格式的所有切片。从第 10 行至第 21 行，各 $N[t]$ 条流水线被并行执行。第 11 行和第 12 行将切片集按流的数量分割成多个子集，这相当于将多个切片融合成子集，以提高内核函数的执行效率。第 13 行调用了 *cudaMemcpyAsync()* 函数，将切片从 CPU 的锁定页面内存异步传输到 GPU。在乘法计算之前，第 14 行获取了这种格式下切片集的数量。

中国知网　https://www.cnki.net

算法 3.1 基于 HTF 格式的流水线 SpTV 算法

输入:

HTF 格式中的五个切片集合 $\mathbb{S}_{CSR}, \mathbb{S}_{COO}, \mathbb{S}_{ELL}, \mathbb{S}_{HYB}, \mathbb{S}_{Dense}$ of the tensor $\mathcal{X}$;

向量 v;

相乘的模态 J;

每种格式的 CUDA 流数量 $N_{CSR}, N_{COO}, N_{ELL}, N_{HYB}, N_{Dense}$;

输出:

矩阵 Y;

1: 设 $Y_{CSR}, Y_{COO}, Y_{ELL}, Y_{HYB}, Y_{Dense}$ 是五个切片集合对应的结果;
2: 设 Y = [$Y_{CSR}, Y_{COO}, Y_{ELL}, Y_{HYB}, Y_{Dense}$];
3: 设 S = [$\mathbb{S}_{CSR}, \mathbb{S}_{COO}, \mathbb{S}_{ELL}, \mathbb{S}_{HYB}, \mathbb{S}_{Dense}$];
4: 设 N = [$N_{CSR}, N_{COO}, N_{ELL}, N_{HYB}, N_{Dense}$];
5: 将 v 从 CPU 传输到 GPU 上;
6: 设 N_{max} 是数组 N 中的最大值;
7: 创建 N_{max} 个 CUDA 流;
8: **for** $t = 0$ **to** 5 **do**
9: # 流水线并行执行 for 循环
10: **for** $n = 0$ **to** $N[t]$ **do**
11: 设 S_e 是切片集合 $S[t]$ 中分配给每个流的切片的子集;
12: 设 Y_e 是分配给结果矩阵中每个流的子矩阵 $Y[t]$;
13: 通过流 n 从 CPU 到 GPU 异步传输 S_e;
14: 设 L 为子集 S_e 中的片数;
15: **for** $l = 0$ **to** L **do**
16: 设 $s[l]$ 是 S_e 集合的 l 片;
17: 设 $y[l]$ 是 Y_e 的 l 切片的结果;
18: 在 GPU 上执行 Multiply($s[l]$, v, J, $y[l]$, n);
19: **end for**
20: 通过流 n 从 GPU 异步传输 Y_e 到 CPU;
21: **end for**
22: cudaDeviceSynchronize();
23: **end for**
24: 将 $Y_{CSR}, Y_{COO}, Y_{ELL}, Y_{HYB}, Y_{Dense}$ 按照切片序号拼接成结果 Y;
25: **return** Y;

从第 15 行至第 19 行，算法遍历了分配给流 n 的切片子集中的每个切片，并为每个切片执行乘法计算。第 20 行将结果从 GPU 异步传输回 CPU。接着，第 22 行同

中国知网 https://www.cnki.net

步等待所有流的计算完成。最后，在第 24 行将五种格式的结果拼接到结果张量中。在第 25 行，算法将结果张量返回并输出。

为了优化常见的数据并行处理方式，切片被分成多个段，并对每个段执行 SpMV。这种分段方法具有多个优点之一，即每个段在很大程度上独立于其他段，有助于更好地重用已获取的矩阵值并降低内存占用。CUDA 线程作为 NVIDIA GPU 的最小执行单位[109]，在 GPU 上执行 SpMV 时，每个 CUDA 线程负责计算一个段。与矩阵类似，张量也按切片方式分成多个段，以更好地利用数据并行性。然而，由于张量的稀疏结构，可能会导致不连续的内存访问和并行加速中的负载不均衡等问题[110]。因此，为了加速 SpTV 核心计算，采用已有的矩阵格式来捕捉切片的稀疏分布，并通过多种格式的组合来提升整个张量的计算性能是一种有效的解决方案。

1、对于 COO 切片，使用三个数组，即 *cooVal*、*cooRowIdx* 和 *cooColIdx* 以存储非零坐标和值的列表。每个非零值被分配给一个 CUDA 线程用于乘法计算。假设一个整数的存储大小为 β_i，单精度浮点值的存储大小为 β_f。则 $O(nnz)$ 个非零值的 COO 格式占用的总内存大小为 $(2 \times \beta_i + \beta_f) \times O(nnz)$。

2、对于 CSR 格式切片，使用三个数组，即 *csrRowPtr*、*csrColIdx* 和 *csrVal* 来存储非零值。数组 *csrColIdx* 存储非零值的列索引，数组 *csrRowPtr* 存储每行的偏移量。最后一个数组 *csrVal* 储存非零值本身[111]。每个 CUDA 线程负责计算结果矩阵中的一个值，即组织张量一个纤维与向量的相乘。该格式占用的内存大小总内存占用为 $\beta_i \times (F_n + 1 + O(nnz)) + \beta_f \times O(nnz)$，其中 F_n 是纤维数量。

3、对于 ELL 格式切片，使用数组 *ellVal* 来存储每行非零值的值，数组 *ellIdx* 来存储每个非零值的列索引。这两个数组逐行顺序存储每个非零值的值和列索引。每行中非零值的最大数量记为 H_e，在图3.2中为 2。因此，该格式占用的内存大小为 $(\beta_f + \beta_i) \times H_e$。

4、对于 HYB 格式切片，将 ELL 和 COO 组合在一起用来存储非零值[112]。使用经验确定的阈值 H 来确定存储行中的非零值的格式。具体而言，如果一行中的非零值数量大于 H，则超过 H 的部分使用 COO 存储，其余部分使用 ELL 存储。在图3.2中，H 设置为 1。该格式占用的总内存大小可以表示为 $G_{hybcoo} + G_{hybell}$。

5、对于 Dense 格式切片，仅使用一个数组 *dnsVal* 来存储切片中每个值，每个 CUDA 线程计算结果矩阵中的一小部分。该格式占用的内存大小为 $\beta_f \times J \times K$。

总而言之，基于 HTF 格式的流水线 SpTV 算法面临着两个挑战：第一是为 HTF 中的每个切片选择合适的格式。第二是自适应地设置每种格式的最佳流数。因此，除了基于 HTF 的流水线 SpTV 算法外，IAP-SpTV 还包括了应对这些挑战的解决方案。

中国知网 https://www.cnki.net

3.3 Slice-GCN 格式选择模型

为了解决 HTF 中每个切片选择适当格式的问题，通过一种两阶段的格式选择方法，在预处理阶段为切片找到最佳的格式。

第一阶段旨在评估切片的稀疏性，并将一部分切片设置为密集格式，当切片的非零值稀疏度大于或等于阈值 sp 时，密集格式相对于所有稀疏格式具有明显的优势。

第二阶段专注于使用 Slice-GCN 模型为切片选择适当的格式，当切片的非零值稀疏度低于阈值 sp 时，切片的格式选择变得较为复杂。因此，本章构建了一个用于稀疏切片格式选择的图神经网络模型，即 Slice-GCN 模型，以帮助选择最适合的格式。

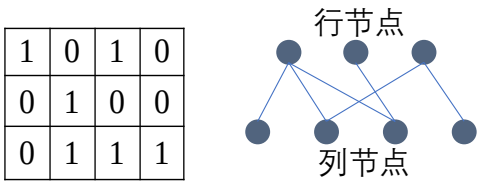
3.3.1 稀疏结构的图表示形式

为切片选择适当格式是一项复杂任务，通常需要对大量数据进行分析。因此，设计一种有效的稀疏特征提取方法对于切片的格式选择任务具有重要价值。

一种常见的方法是采用 CNN 作为特征提取工具，将任意大小的切片通过统计学方法转换成固定大小的矩阵，以作为 CNN 的输入。然而，这种方法可能导致形状较大的切片的稀疏信息丧失。因此，本研究提出了一种基于二分图的方法，用于表征切片的稀疏特征，并将这些特征分为两类，即细粒度特征和全局特征。

细粒度特征以二分图的形式表示切片内部不同行和列之间的相似性关系。在研究矩阵行列划分时，二分图模型得到广泛应用[108,113]。这种由张量转换而来的二分图是一个无向图，包括两个顶点集，分别代表矩阵的行和列，而连接这两个顶点集的边则代表矩阵中的非零值。用 $\mathbb{G} = (\mathbb{H}, \mathbb{C}, \mathbb{E})$ 来表示该二分图，其中 $\mathbb{H}$ 表示行的顶点集，$\mathbb{C}$ 表示列的顶点集，而 $\mathbb{E}$ 表示边的集合，用以表示矩阵 X。

例如，如果矩阵 X 中在第二行和第三列存在一个非零值，则在图 $\mathbb{G}$ 中，将从顶点集 $\mathbb{H}$ 的节点 h_2 连接到顶点集 $\mathbb{C}$ 的节点 c_3，形成一条边 $e_{2,3} = (h_2, c_3)$。图3.3（a）和（b）分别展示了矩阵和由矩阵转换得到的二分图。



(a) 矩阵　　(b) 二部图

图 3.3　矩阵到二部图的转换示意图

全局特征表示与张量整体稀疏度相关的 9 个统计特征。这包括总行数、总列数、总非零值数，以及每行非零值的均值、最大值、最小值，以及每列非零值的

中国知网 https://www.cnki.net

均值、最大值和最小值。这 9 个特征可通过遍历一次张量后统计得出。

3.3.2 Slice-GCN 设计

图神经网络模型，作为广泛用于关系抽取、推荐系统等领域的流行图数据处理方法之一，通过采用图卷积和池化层，从图结构中提取特征，并借助多层感知器来执行特征分类[114]。图卷积层的核心原理在于进行节点之间的消息传递，以便捕获图中的依赖关系。消息传递的核心思想涉及反复汇总来自相邻节点的特征消息，并将这些消息整合到当前中心节点的特征中[105]。多层感知器包括至少一个输入层、一个输出层和一个或多个隐藏层，其中每个非输入节点均配备有非线性激活函数的神经元。多层感知器通过反向传播算法来学习输入层和输出层之间的映射关系。

Slice-GCN 是一种专用的 GCN 模型。由于训练 Slice-GCN 需要大量已标记的矩阵数据，因此使用了 SuiteSparse ① 矩阵集合中的 2041 个矩阵作为训练数据集。标记矩阵的过程包括两个步骤：首先，在目标硬件平台上测量每种不同格式矩阵的执行时间；然后，选择执行时间最短的格式作为矩阵的标签。

借助图卷积层的有效图结构特征提取和多层感知器强大的数据分类能力，设计了 Slice-GCN 为切片选择最佳存储格式。如图3.4所示，其中 GCN 用于提取有关图结构的信息，多层感知器用于对 GCN 提取的特征信息分类。

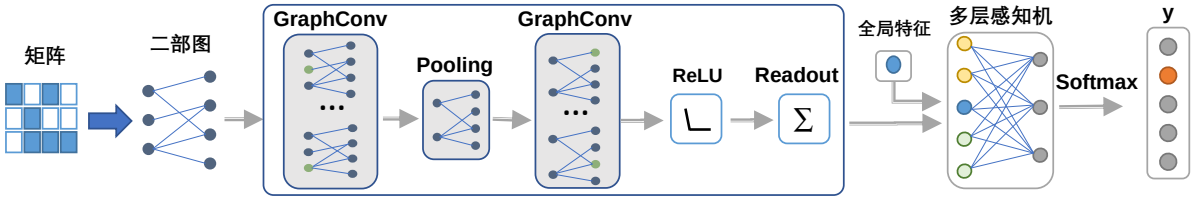


图 3.4　Slice-GCN 模型结构示意图

为了在预测精度和计算开销之间取得平衡，研究测试了多个常用的参数选项，并最终选择了最适合的参数设置。观察结果表明，GCN 输出特征向量的长度设置为 512，隐藏层神经元的数量设置为 128。Slice-GCN 模型中的 GCN 包括两个 GraphConv 层，输出特征向量的长度设置为 512，即 256 乘以 2。

多层感知机的输入由长度为 521 的特征向量组成，其中包括 GCN 的输出向量和长度为 9 的全局特征向量的拼接。此外，多层感知机的隐藏层神经元数设置为 128，鉴于研究考虑了五种稀疏格式作为备选方案，输出层的神经元数设置为 5。

3.3.3 超参数设置

切片稀疏度阈值 *sp* 是两阶段格式选择的关键超参数。相对于在第一阶段也使用 Slice−GCN，使用稀疏度阈值 *sp* 可以快速过滤出适合使用 Dense 的矩阵。原

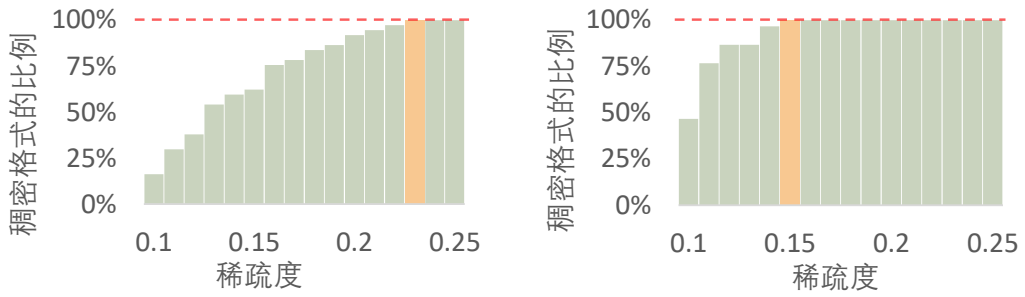
① http://sparse.tamu.edu/

中国知网　https://www.cnki.net

因在于从稠密切片中得到的二部图中存在大量边，这将使 Slice−GCN 的计算开销增大，并给旨在识别细粒度稀疏结构的 Slice−GCN 带来显著的误差。因此，阈值 *sp* 被用来筛选 Dense 的切片，以平衡计算开销和准确性。

为了找到合适的稀疏度阈值 *sp*，随机生成了一组 500 行 500 列、稀疏度从 0.1 到 0.25、间隔为 0.01 的矩阵集合。然后，将所有矩阵根据稀疏度分为 16 组，每组有 100 个矩阵，并记录最优格式为 Dense 的矩阵数量与该组中所有矩阵数量的百分比值。

每个稀疏度下的矩阵集合中最优格式为 Dense 的数量占该组矩阵总数量的百分比结果显示在图 3.5 中。对于稀疏度大于 0.23 的组，所有矩阵的最优格式是稠密的。因此，切片的稀疏度阈值被设置为 0.23。



(a) V100　　(b) TITAN

图 3.5　矩阵稀疏度与稠密格式是最佳格式的关系图

此外，为了提高训练期间的数据稳定性，Slice-GCN 使用 *LogSoftmax* 作为输出层，*NLLLoss* 作为损失函数，其中 *LogSoftmax* 和 *NLLLoss* 是由 PyTorch Geometric 库[115] 提供的开源类和函数。

3.4　基于 HTF 的流水线性能分析模型

为了解决在 HTF 中确定每种格式的 CUDA 流数量的最佳配置问题，本节通过对不同格式的传输和计算执行时间定量分析，然后对最佳流数问题建模，以实现自适应地设置 IAP-SpTV 中的流数。

3.4.1　HTF 格式的性能分析

启动多个内核以处理不同格式是一项耗时的操作，会增加上下文切换的额外开销，从而降低性能。通过使用 CUDA 流，GPU 程序可以高效地并行执行内存访问和计算操作，从而提高数据吞吐量。具体而言，本研究利用 CUDA 流在设备上并行执行内核操作，并在批处理切片执行完成后才同步流。因此，采用一种基于纤维的静态分析方法来评估 CPU-GPU 上述格式的时间开销。

内核计算时间与内存访问时间 *MA*、CUDA 核心计算时间 *CC* 和切片格式相

中国知网　https://www.cnki.net

关。由于不同格式的分析过程相似，因此，本节以 COO 为例分析。对于 COO 而言，内核计算时间主要受内存访问时间影响，使得 $CT = \text{Max}(MA, CC)$。而对于可以有效利用内存带宽的 ELL、CSR、HYB 以及 Dense 而言，计算时间表示为 $CT = MA + CC$。具体如下：

$$CT = \begin{cases} MA + CC, & \text{For CSR, ELL, HYB, Dense} \\ \text{Max}(MA, CC), & \text{For COO} \end{cases} \tag{3.1}$$

因此，假设 SpTV 是串行执行的，其在 CPU-GPU 上的总执行时间 T 表示为：

$$T = TT + CT \tag{3.2}$$

其中传输时间 TT 和计算时间 CT 与非零值的数量密切相关。对于异步并行的情况，将在3.4.2节中进一步讨论。

GPU 上的底层编译优化通常会对 MA 和 CC 之间的重叠产生影响。这种重叠的定量分析不容易通过直接分析硬件来完成。因此，为了简化模型求解，本文简化了这种重叠的影响，并引入了一个分布函数来分析张量纤维中非零值的数量。该分布函数表示了一个张量的样本空间中，具有给定长度的纤维所占的比例。

给定一个随机变量 ξ，表示纤维的非零值的数量，样本空间 $\Omega = \{1, 2, ..., E\}$，其中 E 表示纤维的长度。因此，表示纤维中非零值个数的分布函数为：

$$P(\xi = z) = \Upsilon_z / F_n, z \in \Omega \tag{3.3}$$

其中 Υ_z 表示非零值个数为 z 的纤维个数，F_n 表示纤维总数。纤维中非零值个数的期望值为：

$$\mathbb{E}(\xi) = \sum_{z=1}^{z \in \Omega} (z \times P(\xi = z)) \tag{3.4}$$

因此，F_n 纤维的非零值总数记为：

$$O = \mathbb{E}(\xi) \times F_n \tag{3.5}$$

由于结果是一个稠密的张量 $Y \in \mathbb{R}^{I \times K}$，假设 PCIe 的带宽大小为 B，PCIe 传

中国知网 https://www.cnki.net

输时间表示为：

$$TT = TT_{h2d} + TT_{d2h} = \frac{\beta_i \times A_i + \beta_f \times A_v}{B} + \frac{\beta_f \times I \times K}{B} \tag{3.6}$$

其中 A_i 为索引的数量，A_v 为值的数量。五种格式的传输时间分别表示为：

$$TT_{COO} = \frac{(2 \times \beta_i + \beta_f) \times O}{B} \tag{3.7}$$

$$TT_{CSR} = \frac{\beta_i \times (F_n + 1 + O) + \beta_f \times O}{B} \tag{3.8}$$

$$TT_{ELL} = \frac{(\beta_f + \beta_i) \times F_n \times H_e}{B} \tag{3.9}$$

$$TT_{Dense} = \frac{\beta_f \times J \times K}{B} \tag{3.10}$$

$$TT_{HYB} = \frac{G_{hybcoo} + G_{hybell}}{B} \tag{3.11}$$

其中 H_e 为所有纤维中非零值的最大值。对于切片的每个纤维，HYB 中高于阈值 H 的部分使用 COO 存储，其余部分使用 ELL 存储。假设 H 小于 $\mathbb{E}(\xi)$，HYB 的 COO 部分中非零值的总数记为 $F_n \times (\mathbb{E}(\xi) - H)$，ELL 部分中非零值的总数记为 $F_n \times H$。因此，可以得到：

$$G_{hybcoo} = (\beta_f + 2 \times \beta_i) \times F_n \times (\mathbb{E}(\xi) - H) \tag{3.12}$$

$$G_{hybell} = (\beta_f + \beta_i) \times F_n \times H \tag{3.13}$$

内存访问时间 MA 由读取和写入两个部分组成，主要关注对全局内存的访问所需的时间。缓存和共享内存的性能优化受多种复杂因素的影响，如中间辅助数组等，因此难以精确评估。针对 CUDA 流数量的选择问题，本研究对计算性能进行了简化分析。在全局内存中，同时读取具有相同内存事务长度的连续数据。如果一个线程束的数据访问是连续的，则可以隐藏对全局内存的访问延迟。为了选择适当的稀疏格式以增加内存访问的合并性，这项研究假设所有访问操作都能够

中国知网 https://www.cnki.net

合并。*MA* 表示为：

$$MA = \left\lceil \frac{G_{v+s}}{M} \right\rceil \times \frac{1}{R} \tag{3.14}$$

其中 M 是内存事务的大小，R 是全局内存的时钟频率。G_{v+s} 包含两部分，向量内存大小 G_v 和切片内存大小 G_s。N_{sp} 是流处理器的数量，N_w 是每个线程束中 CUDA 线程数，N_b 是每个线程块中 CUDA 线程数。

对于 COO，每个 CUDA 线程访问的内存包括三个部分，*cooRowIdx* 和 *cooColIdx* 数组中的索引，在 *cooVal* 中的值和输入向量中的对应的值。切片占用的内存大小是 $(2 \times \beta_i + \beta_f) \times O$，输入向量占用的内存大小是 $\beta_f \times J$。由公式（3.14）可知，*MA* 表示为：

$$MA = \left\lceil \frac{(2 \times \beta_i + \beta_f) \times O + \beta_f \times J}{M} \right\rceil \times \frac{1}{R} \tag{3.15}$$

GPU 中每个线程束通过两个向量之间的内积操作来处理一行。因此，计算切片的一行所需的时间表示为 $\left\lceil \frac{\mathbb{E}(\xi)}{N_w} \right\rceil \times \frac{1}{R_{fma}}$，其中 $\frac{1}{R_{fma}}$ 是每个 CUDA 线程浮点数的 FMA 操作的计算时间，期望值 $\mathbb{E}(\xi)$ 用于表示每行非零值的数量。切片由 J 行纤维组成，如果仅使用一个线程束来计算这 J 行，则所需的计算时间为 $\left\lceil \frac{\mathbb{E}(\xi)}{N_w} \right\rceil \times \frac{1}{R_{fma}} \times J$。由于 GPU 可以同时执行 $\frac{N_{sp}}{N_w}$ 个线程束，因此内核的计算时间 *CC* 表示为：

$$CC = \left\lceil \frac{\mathbb{E}(\xi)}{N_w} \right\rceil \times \frac{1}{R_{fma}} \times J \times \frac{N_w}{N_{sp}} \tag{18}$$

3.4.2 流水线性能模型

针对流的重叠程度问题，本研究仅考虑具有两个复制引擎且不存在隐式同步的情形。拥有两个复制引擎的 GPU 可通过 PCIe 总线实现计算和通信两个方向的完全重叠。在数据传输的内核计算中，不涉及隐式同步。如图 3.6 所示，当使用 4 个流时，不存在隐式同步，且两个复制引擎的重叠被实现。然而，如果内核计算时间较短，启动流所带来的额外开销将相对较大，无法实现传输时间和计算时间的有效重叠。

为了平衡启动开销和执行时间，本研究采取了一种同时传输多个流中相同格式的切片集的策略，以增加执行时间。对于整个张量，也就是多种格式的切片集族，采用分批传输的方式，每个批次传输一种格式的切片集，以覆盖整个张量。鉴于内核计算时间较长，除了第一个流之外的其他流需要等待主机传输数据。在以传输为主导的模式中，每个流都能够快速获取来自主机的数据，从而最小化所有流的等待时间。

中国知网 https://www.cnki.net

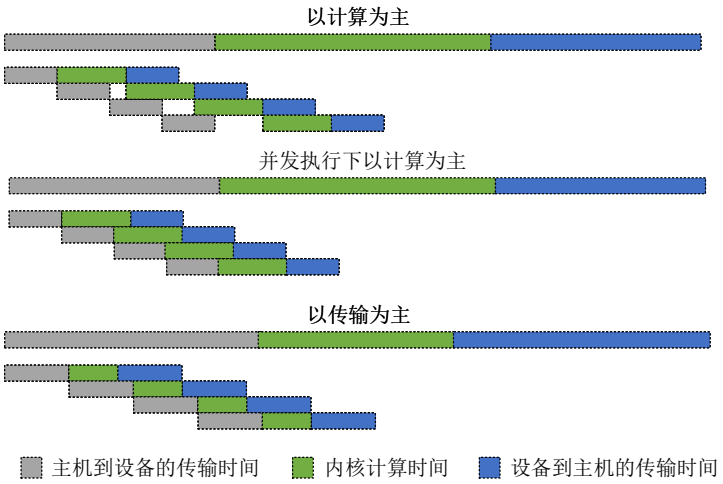


图 3.6　流水线计算类型的示意图

因此，所有流的总执行时间 $T(n)$ 被建模为：

$$T(n)=\begin{cases}\frac{TT_{h2d}}{n}+CT+\frac{TT_{d2h}}{n}+\lambda n, & CT \geqslant TT_{h2d}\\ \\ TT_{h2d}+\frac{CT}{n}+\frac{TT_{d2h}}{n}+\lambda n, & CT < TT_{h2d}\end{cases} \tag{3.16}$$

其中 n 是流的数量。λ 是惩罚系数，用于量化表示创建流的开销。

另外，在以计算为主导的模式中，需要考虑每个流的内核函数是否充分占用了流多处理器的计算资源。如图3.6中顶部情况所示，每个流需要有足够的计算任务来较好地利用流多处理器。

当一条流水线上的任务无法占满流多处理器时，会导致流的并发执行状态，如图3.6中间情况所示，该情况下总的处理时间表示为：

$$T(n)=TT_{h2d}+\frac{CT}{n}+\frac{TT_{d2h}}{n}+\lambda n \tag{3.17}$$

这与以传输为主导的模式的情况相同，因此可以参照以传输为主导的模式来获得并发执行的最佳结果。

流水线优化的问题表示如下：

$$\arg\min_{n\in[1,N]} T(n) \tag{3.18}$$

其中 N 是 CUDA 中的最大并发流数。在实际应用上还受限于寄存器的总数量等，因此，可以并发执行的流的数量是有限的，而不是可以创建无数个 CUDA 流。

定理 3.1　从 (3.18) 的函数形式来看，n 的优化目标 $T(n)$ 是一个凸函数。

中国知网　https://www.cnki.net

证明 给定两个变量 n_1, n_2 和任意有理数 $\theta \in (0,1)$，然后定义函数 $f(n_1, n_2)$：

$$\begin{aligned} f(n_1,n_2) &= T(\theta n_1 + (1-\theta)n_2) - \theta T(n_1) - (1-\theta)T(n_2) \\ &= \begin{cases} \frac{(TT_{h2d}+TT_{d2h})\times\theta(\theta-1)(n_1-n_2)^2}{[\theta n_1+(1-\theta)n_2]n_1n_2}, & CT \geqslant TT_{h2d} \\ \\ \frac{(CT+TT_{d2h})\times\theta(\theta-1)(n_1-n_2)^2}{[\theta n_1+(1-\theta)n_2]n_1n_2}, & CT < TT_{h2d} \end{cases} \end{aligned} \tag{3.19}$$

显而易见，$\theta - 1 < 0, f(n_1, n_2) \leqslant 0$，因此 $T(n)$ 在这两种情况下满足凸函数[116] 的定义。

□

因此，根据定理3.1，必须有一个最优的流数量来最小化总体 GPU 执行时间。

定理 3.2 根据定理3.1，由 n_{opt} 定义的公式3.18存在满足计算时间和传输时间重叠最大化的解。最优流数 n_{opt} 表示为：

$$n_{opt} = \begin{cases} \sqrt{\frac{TT_{h2d}+TT_{d2h}}{\lambda}}, & CT \geqslant TT_{h2d} \\ \\ \sqrt{\frac{CT+TT_{d2h}}{\lambda}}, & CT < TT_{h2d} \end{cases} \tag{3.20}$$

证明 由定理3.1可知，n 的优化目标是一个凸函数，$T(n)$ 的导数为：

$$\frac{\mathrm{d}T(n)}{\mathrm{d}n} = \begin{cases} \lambda - \frac{TT_{h2d}+TT_{d2h}}{n^2}, & CT \geqslant TT_{h2d} \\ \\ \lambda - \frac{CT+TT_{d2h}}{n^2}, & CT < TT_{h2d} \end{cases} \tag{3.21}$$

令 $\frac{\mathrm{d}T(n)}{\mathrm{d}n} = 0$，便可证明定理3.2。

□

因此，通过定理3.2，可以确定算法3.1中每个切片需要的流水线数量。即以最小化 T 为目标，通过建模分析，可以推导出算法3.1中第 4 行的最佳配置的 CUDA 流数量组合。每个切片都有其最佳格式，对于具有相同格式的切片集合，执行过程分为 n_{opt} 个组，每个组以切片为最小粒度，以实现完全重叠的流水线技术。

3.5 实验评估

本实验在 CPU-GPU 上评估了多个数据集上的 IAP-SpTV 性能，所有的比较实验都使用单精度浮点数，且结果是 10 次运行的平均值。

中国知网 https://www.cnki.net

3.5.1 平台和数据集

本实验使用了两个不同架构的CPU-GPU平台，其完整的硬件配置详见表3.1。Slice-GCN的实现所需的软件库包括Python 3.7、PyTorch 1.4.0和PyTorch Geometric 1.6.0。

表 3.1 SpTV 实验平台配置总结表

平台缩写	V100		TITAN	
参数	Intel(R) Xeon(R) Gold 6248 CPU	NVIDIA V100 GPU	Intel(R) Xeon(R) Silver 4110 CPU	NVIDIA TITAN RTX
微体系结构	Cascade Lake	Volta	Skylake	Turing
频率	2.50 GHz	1.455 GHz	2.10 GHz	1.77 GHz
# 物理核心	20	5120	8	4608
FP32 峰值性能	-	15 TFLOP/s	-	16.3 TFLOP/s
最后一级缓存大小	27.5 M	16 M	11 M	6 M
内存大小	132 G	32 G	132 G	24 G
最大带宽	131 GB/s	900 GB/s	107 GB/s	672 GB/s
编译器	GCC 5.5.0	NVCC 10.1	GCC 5.4.0	NVCC 10.0

本实验使用的张量数据集均来自公开可获取的资源，如表3.2所示，其中包括Hetrec2011-lastfm-2k[117]、MovieLens Ratings[118]、Uber Pickups[119]、Chicago Crime[119]和NeurIPS Publications[120]等数据集。

此外，Slice-GCN的训练数据还包括SuiteSparse矩阵集合中的2041个矩阵。在这些矩阵中，M-Format表示了当前切片最适合的稀疏格式，即执行SpMV时间最短的稀疏格式。其中M-Format的0、1、2、3、4分别代表存储格式CSR、COO、ELL、HYB和Dense。

表 3.2 SpTV 实验中张量数据集的总结表

缩写	张量数据集	I	J	K	非零元数量
Mov	MovieLens Ratings	71568	65134	6	10000054
Het	Hetrec2011-lastfm-2k	2100	18744	12647	186479
Ube	Uber Pickups	4392	1140	1717	3309490
Chi	Chicago Crime	148464	77	32	5330673
Nip	NeurIPS Publications	42194	2862	14036	3101609

中国知网 https://www.cnki.net

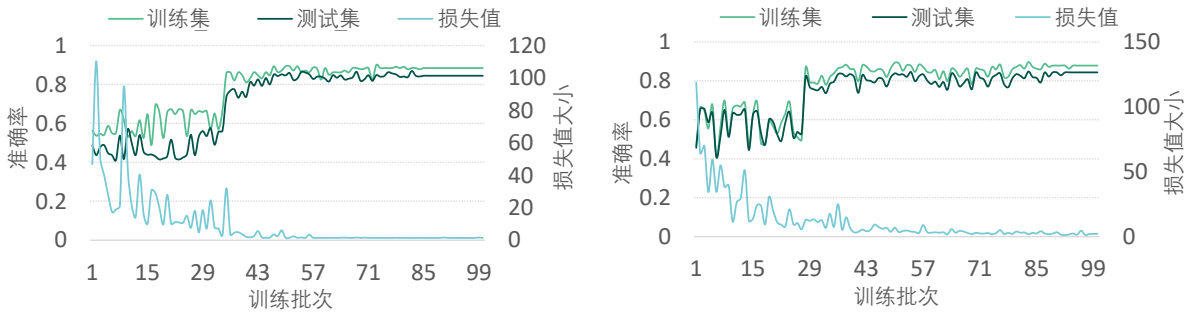
3.5.2 模型训练结果

为了全面评估 Slice-GCN 的性能，本实验给出了 Slice-GCN 在训练过程中的损失值、测试精度以及训练精度的变化过程。在训练阶段，Slice-GCN 会根据损失值来更新神经网络的权重，损失值表示了预测值与目标值之间的差异程度。训练精度和测试精度表示了在训练集和测试集中正确预测案例所占的比例，预测结果使用二进制值表示是否正确预测。

本实验将 SuiteSparse 矩阵集合中的所有矩阵按照 80% 和 20% 的比例划分为训练集和测试集，并标记了在 GPU 上 SpMV 性能最佳的格式为 M-Format。鉴于不同 GPU 上切片的计算性能可能存在差异，本实验比较了在 Tesla V100 GPU 和 TITAN RTX GPU 上所有矩阵的最佳格式。结果表明，在实验使用的 2041 个矩阵中，有 701 个矩阵在这两种 GPU 上具有不同的最佳格式。因此，M-Format 是特定于 GPU 的，不适用于所有不同架构的 GPU。并且使用特定于相应 GPU 的 M-Format 对模型重新训练是解决这一问题的可行方法。

为了展示 Slice-GCN 的训练过程，本实验给出了每一步训练中的损失值，并在每个训练步骤之后记录了模型在训练集和测试集上的准确率，结果如图3.7（a）和（b）所示。从图中可见，随着训练步数的增加，损失值逐渐减小并最终趋于稳定，同时训练集和测试集上的准确率逐渐上升并保持稳定。损失值逐渐减小的趋势表明网络对训练数据中的矩阵格式的预测逐渐接近最佳矩阵格式，这也是训练集准确率提高的原因。

具体而言，在 V100 GPU 上，模型在约 85 个训练批次后达到稳定状态，总训练时间为 1.01 小时。而在 TITAN GPU 上，模型在大约 92 个训练批次后达到稳定状态，总训练时间为 2.29 小时。一旦模型达到稳定状态，V100 GPU 上的训练集准确率为 87.81%，而 TITAN GPU 上的为 88.35%。在 V100 GPU 和 TITAN GPU 上的测试集准确率分别为 84.31% 和 85.78%。



(a) V100 GPU　　(b) TITAN RTX GPU

图 3.7　模型在训练阶段的损失值、训练集精度和测试集精度的结果图

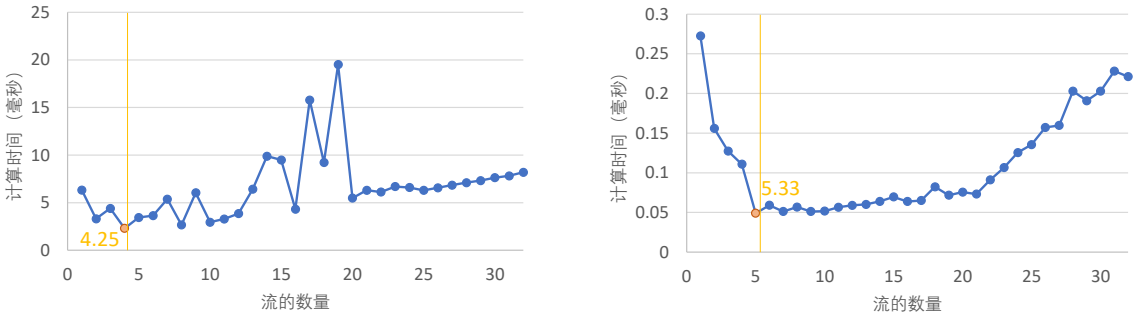
中国知网　https://www.cnki.net

3.5.3 最佳流数

为验证流水线性能模型确定的最佳流数的有效性，以 Nip 数据集为例，本实验对比了在不同流数量下使用 CSR 和 ELL 格式执行 SpTV 操作的执行时间。TITAN RTX GPU 上的实验结果如图3.8所示，并记录了每种格式使用 1 到 32 个流所对应的执行时间。理论计算得出的最佳流数以黄色垂直线表示，而实际执行时间最低的流数以图中的红点表示。可观察到，理论计算得出的流数（黄线）与实际最佳执行时间（红点）非常接近，进一步验证了流水线性能模型的有效性。

此外，这两种格式的执行时间都表现出随着流数量增加而先下降后上升的趋势。这一趋势的原因在于只有少数流能够成功地隐藏数据传输时间。然而，初始化更多流所带来的额外开销抵消了隐藏数据传输的好处，导致每个流的所需处理数据量、传输时间和计算时间均减少。

值得注意的是，对比 CSR 和 ELL 的实验结果，CSR 的折线图出现了明显的振荡，这表明随着流数量的增加，CSR 中每个流的计算粒度相对更小，因此更容易负载不平衡。



(a) CSR　　(b) ELL

图 3.8　理论分析流数与实际测量流数的结果图

3.5.4 性能比较

为了评估本技术的有效性，本研究将基于自适应流水线技术的方法与当前最先进的 GPU 上实现的 SpTV 方法进行比较。因此，本研究的基线包括了 ParTI![103] 和 HOCFS[22]，其中 ParTI! 是一种在 GPU 上广泛使用的稀疏张量运算库，而 HOCFS 则代表了 SpTV 流水线的最新研究成果。

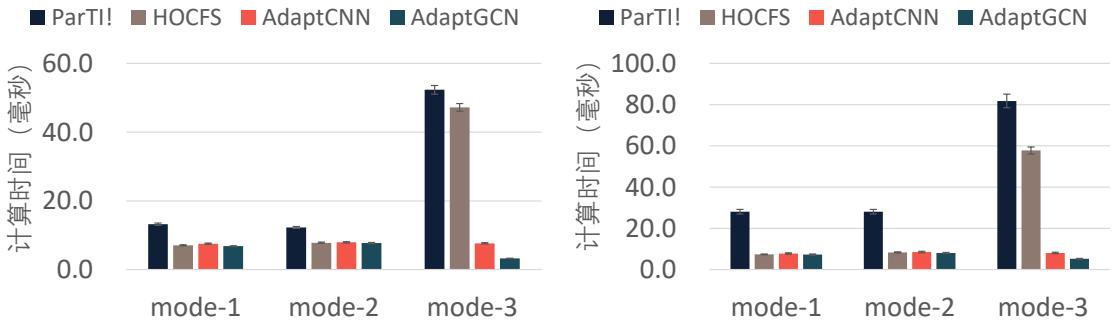
概而言之，ParTI! 库采用同步的单一 COO 格式，而 HOCFS 方法采用两种格式混合、流水线异步执行，具有经验依赖性和静态特性。相较之下，本文提出的 IAP-SpTV 具备多格式混合、流水线异步执行、自适应和动态调整的特点。

此外，为了展示 Slice-GCN 在切片格式选择方面的进展，本研究将其与目前广泛使用的 CNN[65] 进行了对比。为了便于区分，基于 Slice-GCN 实现的 IAP-SpTV 被称为 AdaptGCN，将基于 CNN 和提出的自适应流水线技术实现的

中国知网　https://www.cnki.net

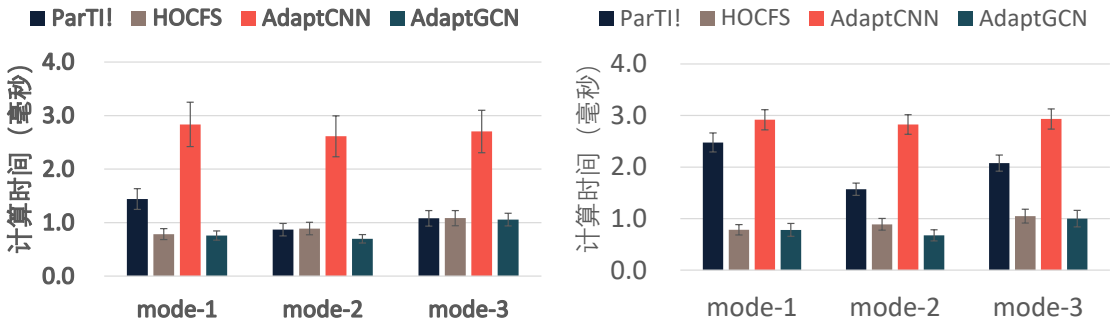
SpTV 称为 AdaptCNN。为了更公平地进行比较，实验中的 CNN 是使用与 Slice-GCN 相同的训练集重新训练得到的。然后通过统计方法将任意大小的矩阵缩放为固定大小的矩阵，然后将其输入到 CNN 中进行处理。

图3.9、3.10、3.11、3.12和3.13展示了在 Tesla V100 GPU 和 TITAN RTX GPU 上的性能数据，以验证所提出方法的可移植性，其中 mode-1 表示在第一个模态上执行 SpTV 的实验，mode-2 表示在第二个模态上执行 SpTV 的实验，mode-3 表示在第三个模态上执行 SpTV 的实验。具体而言：



(a) V100　　(b) TITAN

图 3.9　Mov 数据集上 SpTV 的平均执行时间的结果图



(a) V100　　(b) TITAN

图 3.10　Het 数据集上 SpTV 的平均执行时间的结果图

1、与 ParTI! 相比，AdaptGCN 在大多数测试中取得了显著的性能提升。这是因为 ParTI! 以非零值方式实现了细粒度 SpTV，但仅在 GPU 默认流上执行，这是同步操作，因此相对于 AdaptGCN，它的执行时间更长。

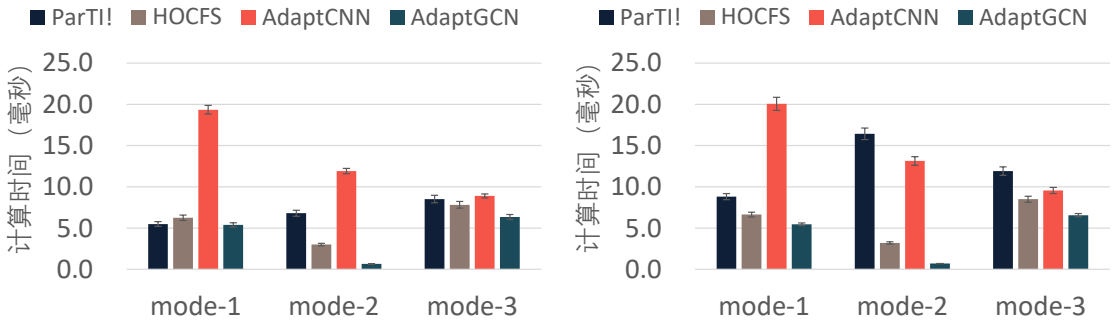
2、与 HOCFS 相比，AdaptGCN 提高了近一半的性能。这是因为 HOCFS 仅区分了切片的 CSR 和 ELL 格式，无法提高以 COO、HYB 和 Dense 存储的切片计算性能。

3、与 AdaptCNN 相比，AdaptGCN 在部分测试中表现出明显的性能优势。这是因为使用 CNN 选择的切片格式通常不是最佳的，不合适的切片格式会导致

中国知网　https://www.cnki.net

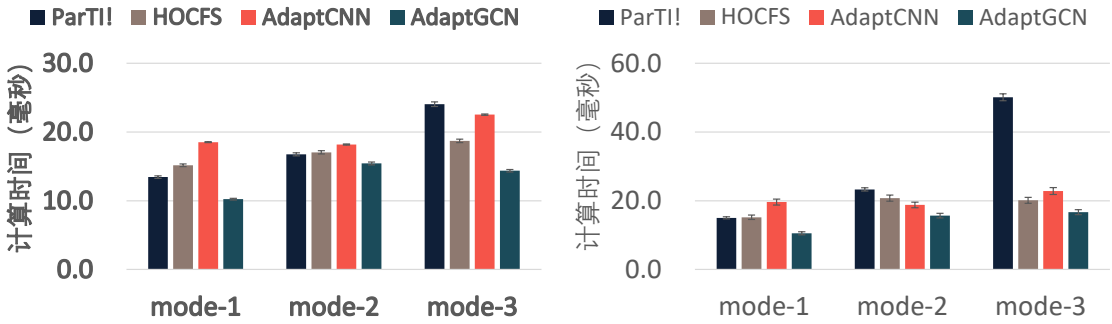
SpTV 的总体执行时间显著增加。此外，由于 CNN 输入的矩阵大小是固定的，因此在将切片输入到 CNN 之前，对它们进行了缩放操作。然而，这种缩放操作可能导致丢失切片的局部稀疏特征[65]。相比之下，稀疏矩阵可以转化为二部图输入到图神经网络中而无需缩放。因此，图神经网络比 CNN 更适合用于切片的稀疏格式选择。

4、总体而言，不同的 SpTV 执行方法在 V100 和 TITAN 平台上的计算性能因 CPU 和 GPU 处理器的不同而有所不同。而且，所有 SpTV 测试在 V100 平台上的计算性能均优于 TITAN 平台，这也与两种硬件配置的差异一致。



(a) V100　　(b) TITAN

图 3.11　Ube 数据集上 SpTV 的平均执行时间的结果图

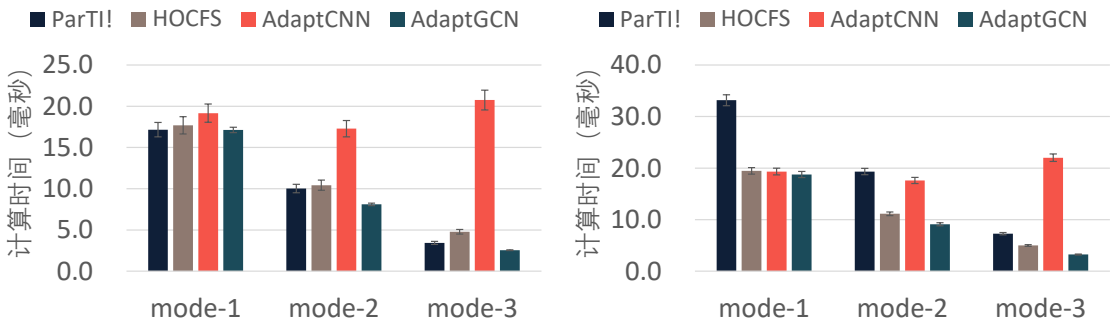


(a) V100　　(b) TITAN

图 3.12　Chi 数据集上 SpTV 的平均执行时间的结果图

此外，ParTI! 仅使用一种格式，因此不需要进行格式选择。与使用纤维密度阈值选择切片格式的 HOCFS[121] 相比，本研究所设计的切片格式选择方法具有更好的适应性。HOCFS 和本研究提出的 IAP-SpTV 方法都采用混合存储格式，因此都存在预处理开销，即选择切片并将其转换为适当格式所需的时间。在实际应用中，如张量分解，存在大量 SpTV 迭代执行过程，这些迭代过程中所节省的时间可以抵消预处理执行时间。

中国知网　https://www.cnki.net



(a) V100　　(b) TITAN

图 3.13　Nip 数据集上 SpTV 的平均执行时间的结果图

本研究将模型的推理时间与一个实际 SpTV 执行时间的结果进行了比较，如图3.14所示。可以看出，如果 SpTV 只执行一次，推理时间将多于实际的 SpTV 执行时间。

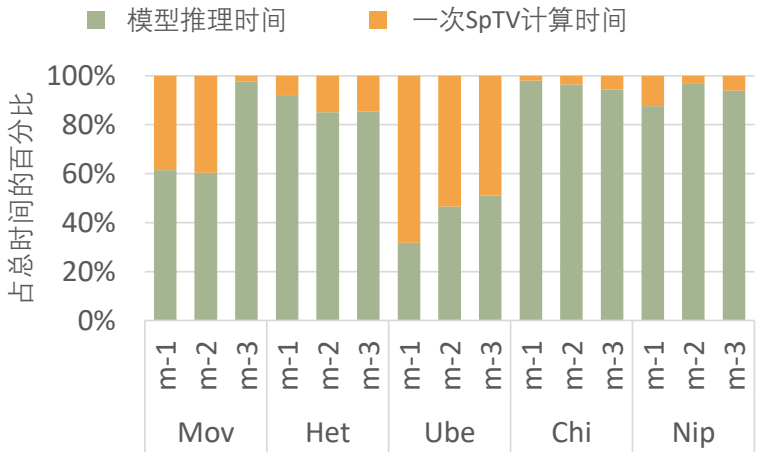


图 3.14　一次 SpTV 计算的时间开销与模型推断的时间开销的比较结果图

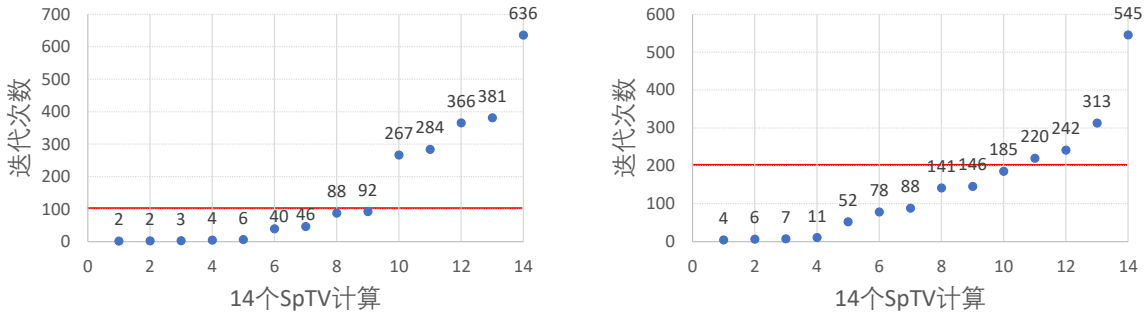
此外，为了确定本研究的方法相对于其他方法表现更佳所需的迭代次数，SpTV 计算在本研究中被重复执行。对于 5 个张量（Mov、Het、Ube、Chi 和 Nip）在 3 种模态下的 SpTV 计算，共得出 15 种 SpTV 计算类型。在图3.15（a）中，Nip 张量在一个模态下的 SpTV 计算所需的迭代次数为 3872 次，已超出了坐标轴范围。因此，图3.15（a）仅呈现了其他 14 种类型的 SpTV 计算。同样，在图3.15（b）中，Het 张量在一个模态下的 SpTV 计算所需的迭代次数为 1244 次，同样已超出了坐标轴范围，因此图3.15（b）也仅呈现了其他 14 种类型的 SpTV 计算。

根据实验结果可以观察到，相较于 ParTI! 和 HOCFS，IAP-SpTV 只需要少量的迭代次数即可获得更好的性能。

综上所述，AdaptGCN 方法在大多数情况下优于其他方法。在 TITAN 平台上，AdaptGCN 方法比 ParTI! 库平均提升了 58.42%，比 HOCFS 方法平均提高了

中国知网　https://www.cnki.net

24.85%。在 V100 平台上，AdaptGCN 方法比 ParTI! 库平均提升了 32.23%，比 HOCFS 方法平均提高了 24.87%。

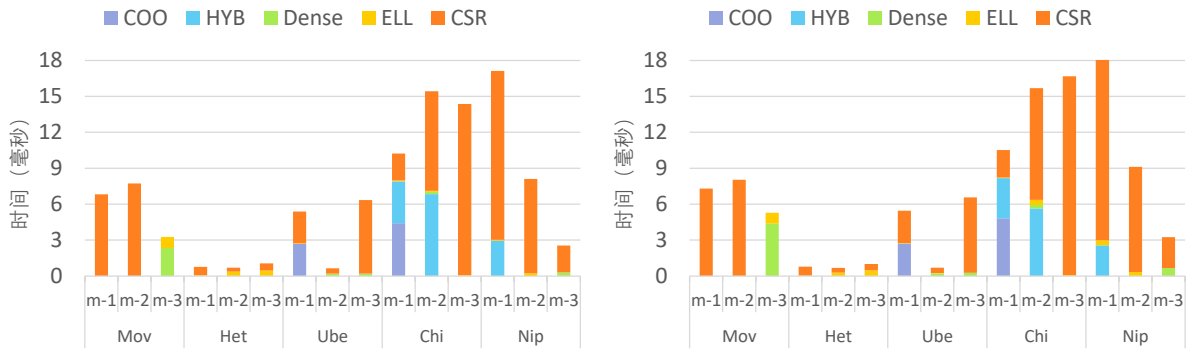


(a) 与 ParTI! 库的比较　　(b) 与 HOCFS 库的比较

图 3.15　分摊不同类型 SpTV 计算的模型推理时间所需的迭代次数的结果图

3.5.5　结果分析

为了更全面地评估 IAP-SpTV，首先比较了每种格式的 HTF 执行时间，如图3.16（a）和（b）所示。其中，m-1、m-2 和 m-3 分别代表 mode-1、mode-2 和 mode-3。当不存在某一种格式的切片时，该格式对应的流数量记为 0。从图中可以观察到，CSR 是切片的主要格式，其次是 ELL，这表明选择 CSR 与 ELL 格式作为 HOCFS 主要计算格式是合理的。并且，COO、HYB 和 Dense 格式在计算中也占有一定比例，这是 IAP-SpTV 相对于 HOCFS 方法性能提升的重要因素。此外，大部分 Mov 张量的 mode-3 使用 Dense 和 ELL 格式进行计算，在图3.9（a）和（b）中可以观察到，对于 Mov 张量的 mode-3，AdaptGCN 相较于 HOCFS 方法性能提升显著。



(a) V100　　(b) TITAN

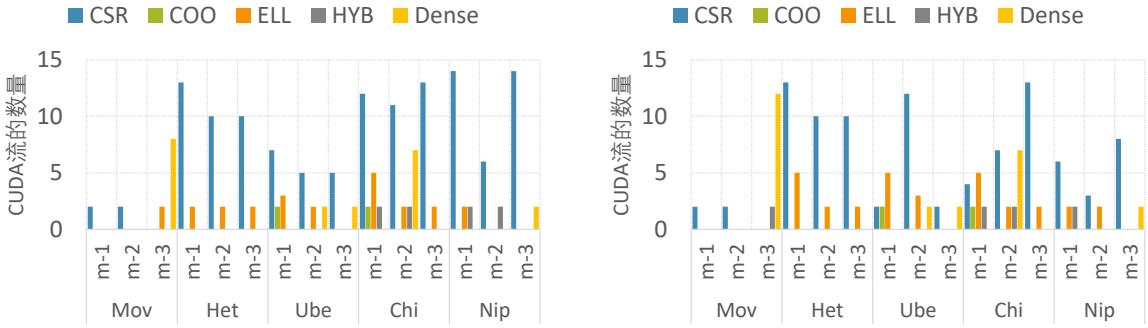
图 3.16　HTF 中每种格式的切片集的平均执行时间的结果图

然后，对比格式选择阶段中 CNN 和 Slice-GCN 在 M-Format 上的差异。如果 Slice-GCN 预测的格式为 1，而 CNN 预测的格式为 0，则 M-Format 差值为 1；如果

中国知网　https://www.cnki.net

它们的预测格式相同，则 M-Format 差值为 0。比较在不同平台上 CNN 和 Slice-GCN 的预测结果差异可知，在 V100 平台上，切片预测结果之间存在 15.86% 的差异，在 TITAN 平台上，该差异值是 17.64%。这些 CNN 和 Slice-GCN 预测结果的差异会影响后续 SpTV 执行的性能，如图3.9、3.10、3.11、3.12和3.13中 AdaptCNN 和 AdaptGCN 的比较所示。

最后，自适应流水线方法在计算过程中引发了 CUDA 流数量的变化，如图3.17（a）和（b）所呈现。在各个模态下，每一种格式的切片集均产生了不同数量的 CUDA 流。因此，自适应流水线技术对于 HTF 混合格式而言显得不可或缺。当比较相同数据集和相同模态在 V100 和 TITAN 两个平台上的 CUDA 流数量时，可以发现流的数量因硬件平台而异。这一结果明显展示了流水线性能模型成功地为不同的硬件平台自动适配了合适的 CUDA 流数量。



(a) V100　　(b) TITAN

图 3.17　自适应流水线方法中用于不同格式的 CUDA 流数量的结果图

3.6　本章小结

本章为 SpTV 为例，基于 HASTensor 中的优化技术提出了一种输入感知的自适应流水线 SpTV 方法，即 IAP-SpTV，表明了 HASTensor 库对访存受限型稀疏张量运算的性能优化方面的有效支持。首先给出了 HTF 混合格式的设计以及基于 HTF 的流水线 SpTV 算法的详细阐述。然后设计了 Slice-GCN 以用于选择最为适用的格式。随后建立 IAP-SpTV 的性能分析模型用于自适应设置 CUDA 流的数量。最后通过一系列的实验验证，确认了该方法在正确性、有效性以及可移植性等方面的优势。

中国知网　https://www.cnki.net

第 4 章　批量计算型稀疏张量运算的并行算法设计和性能优化

本章以 SpTTM 为例，重点介绍了 HASTensor 中用于批量计算型稀疏张量运算的并行算法设计和性能优化方法。SpTTM 作为一种典型的张量运算，广泛应用于张量分析领域，属于批量计算类型，不同张量的稀疏分布特性以及批量计算的特点给高效执行 SpTTM 带来了新的挑战。

为了克服这些挑战，提出了一种异构逐切片计算的 SpTTM 方法，即 HS-SpTTM。首先，描述了在 CPU-GPU 平台上实现 SpTTM 的方法，并将稀疏张量以切片粒度进行存储。针对不同的稀疏格式，提供了并行执行策略，以最大化计算效率和吞吐量，充分发挥了 CPU 和 GPU 的各自优势，从而实现高效的并行计算。然后，通过对理论计算能力和任务量的分析，实现了在异构平台中的负载平衡，引入了一种动态调整计算任务分配的方法，以确保在计算过程中 CPU 和 GPU 的负载保持平衡，提高整体计算性能。随后，讨论了一种描述张量稀疏结构的图结构方法，并设计了一种用于张量格式选择的图卷积神经网络模型（Graph Convolutional Network for Tensor Format Selection，SPT-GCN）。这种方法能够自动选择适用的张量稀疏格式，进一步提高了 SpTTM 的计算效率。最后，通过广泛的实验验证了提出的 HS-SpTTM 相对于其他方法在提高计算效率和准确度方面的优势，使用真实数据集进行了验证。

4.1　引言

稀疏张量运算在诸如医疗保健分析、信号处理、神经科学和机器学习等多个领域中都扮演着核心角色[122–125]。高效的 SpTTM 算法能够提升诸如 Tucker 分解等张量分解算法的计算速度[23,24,57,96,126]。因此，设计一种高效的 SpTTM 算法是迫切且关键的。

在实际应用中，不同的稀疏张量具有各自独特的非零值分布和稀疏结构，这些因素直接影响着 SpTTM 的性能。图4.1展示了两个三阶张量数据集使用不同的稀疏格式执行 SpTTM 时的计算性能。这图表明不同张量对应的最佳稀疏格式存在差异，同一张量在不同模态下执行 SpTTM 时，最优的稀疏格式也不尽相同。

因此，对于提高不同张量数据集的计算效率，关键在于确定不同数据集的非零值稀疏分布所对应的最佳稀疏格式。寻找适合不同张量的计算格式能够有效提高张量计算的效率。然而，目前的大部分工作仅仅是基于人工经验来选择张量的

中国知网　https://www.cnki.net

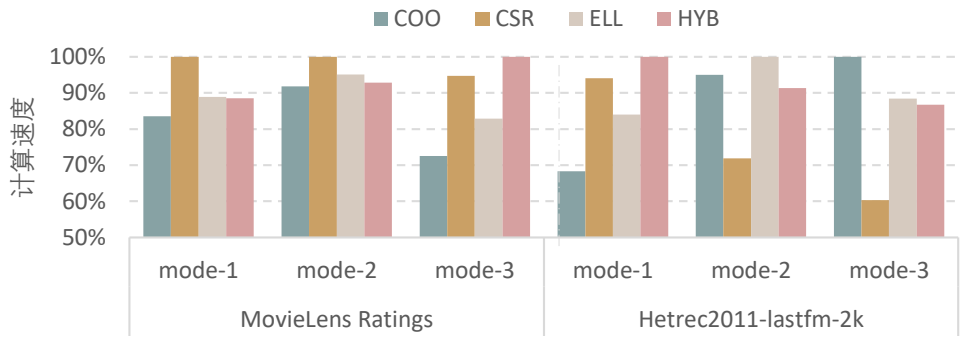


图 4.1　不同格式张量对应的 SpTTM 性能结果图

存储格式，这种方法可能会导致所选的存储格式并不适用于特定的张量，进而影响其计算性能。

随着人工智能的快速发展[127]，稀疏张量的格式选择迎来了新的思考。尽管采用机器学习和深度学习方法会增加额外的计算成本，但这些成本可以通过多次迭代 SpTTM 操作来分摊。传统的机器学习方法，如支持向量机（Support Vector Machine，SVM），会依据张量的统计特性来选择其稀疏格式[69,128]。而深度学习方法，如 CNN，用于选择张量的稀疏格式，则需要将张量首先压缩成一个固定大小的矩阵[65,66]。这两种方法都可能导致张量的某些稀疏分布特性损失，而且随着张量规模的增加，这种损失可能会更加显著。

近年来，利用 GPU 加速密集型计算任务的使用方式正受到广泛关注[129–131]，而 SpTTM 作为一种可利用 GPU 加速的计算任务也备受瞩目。在典型的 CPU-GPU 平台中，包含两个或多个相互连接的处理器，即 CPU 与 GPU，每个处理器的计算速度都存在差异。然而，目前的张量运算库通常仅使用 CPU 或 GPU 来执行计算任务，导致了资源闲置的情况，这限制了 CPU-GPU 平台计算性能的充分发挥。

为了解决以上问题，本章的设计一种基于 CPU-GPU 平台的 SpTTM 并行算法，主要贡献如下：

1、 本章设计了 SpTTM 在 CPU-GPU 平台上的并行计算流程，并给出了不同稀疏格式下 SpTTM 的并行执行策略。
2、 本章通过分析峰值计算能力，评估 CPU 和 GPU 各自的理论任务量，得到了 CPU-GPU 平台上任务映射策略以实现负载均衡。
3、 本章构建了一个名为 SPT-GCN 的张量稀疏格式选择模型，该模型使用图卷积神经网络提取张量的稀疏特征，并为稀疏张量选择合适的存储格式。
4、 本章使用真实数据集执行广泛的实验，结果表明与在 CPU-GPU 平台上的其他库相比，HS-SpTTM 方法可以获得 1.310× 的平均加速比。

中国知网　https://www.cnki.net

4.2 HS-SpTTM 的概述

为了有效利用 CPU-GPU 平台的计算能力来提升 SpTTM 算法的性能，本章设计并实现了一种输入感知的 SpTTM 逐切片计算方法，并命名为 HS-SpTTM。在本节中，首先介绍该方法的整体工作流程，然后介绍其在 CPU-GPU 平台上的异构并行计算模式，并给出该算法在实现过程中需要解决的两个关键问题。

4.2.1 整体工作流程

HS-SpTTM 的整个工作流包括七个步骤，如图4.2所示。

1. **输入张量：**输入数据被组织为输入张量。
2. **展开：**张量沿着模态 *I* 展开为一系列切片。
3. **任务映射：**将 SpTTM 计算任务负载均衡地映射到异构设备上。
4. **格式选择：**SPT-GCN 用于在 CPU 和 GPU 上为张量选择适当的稀疏格式。
5. **格式转换：**张量被转换为 SPT-GCN 选择的格式。
6. **相乘：**在 CPU-GPU 平台上执行逐切片计算过程。
7. **结果张量：**在 CPU 和 GPU 上的计算结果沿着模态 *I* 拼接为结果张量。

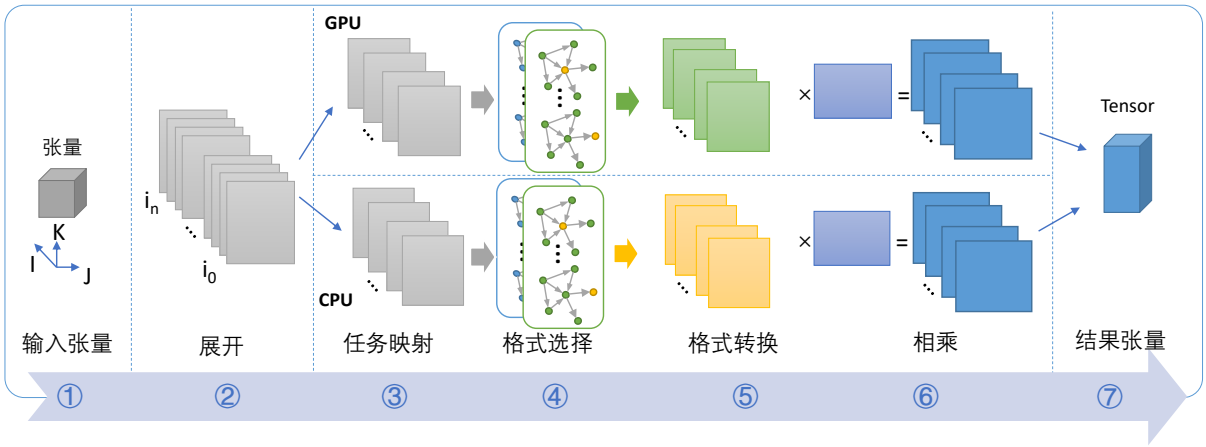


图 4.2 HS-SpTTM 的工作流程示意图

4.2.2 异构并行计算模式

CPU-GPU 平台上的异构并行计算模式主要包括两部分内容，其一是 HS-SpTTM 方法的计算流程，其二是不同稀疏格式的 SpTTM 并行实现方案。

1、 HS-SpTTM 方法的计算流程

在常见的 CPU-GPU 异构并行计算中，通常是由 CPU 上的一个线程来启动 GPU 上的核函数，然后等待 GPU 上的核函数运行完成后，CPU 再执行后续的操作。在这种模式下，CPU 在 GPU 上的计算任务完成之前会一直处于空闲状态，导致计算资源的闲置，降低了平台的计算效率。

中国知网 https://www.cnki.net

通过采用 OpenMP 编程模型，可以有效地将 SpTTM 运算的计算任务分发到 CPU-GPU 平台上。为了提高 CPU 的利用率，HS-SpTTM 采用了结合 OpenMP 与 CUDA 编程的异构并行计算方式。在该方式下，单个 CPU 线程被指定负责控制 GPU，而其他 CPU 线程则可以并行执行计算任务。如图4.3所示，计算任务首先被分成大小不同的两部分。计算量较大的部分被分配给 GPU 进行处理，而另一部分则在 CPU 上并行执行。随后，CPU 上的线程被分成两组，同时开始执行。其中一组专门负责控制 GPU CUDA 线程网格中的核函数，而另一组则负责在 CPU 上进行并行计算任务。为了提高 CPU 和 GPU 上的计算性能和效率，基于 SPT-GCN 的方法被用于在这两种处理器上选择最适合各自硬件结构的稀疏格式来存储张量。最后，假如后续操作是在 CPU 上处理，则所有的计算结果都被收集到 CPU 上，以备后续计算使用。

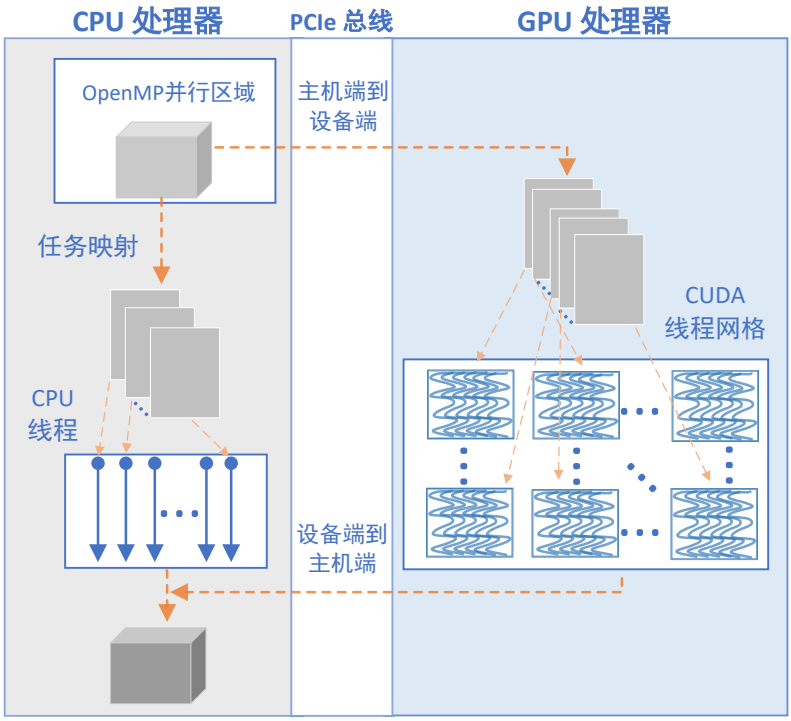


图 4.3　CPU-GPU 上 HS-pTTM 异构并行计算模式的示意图

2、 CPU-GPU 上 SpTTM 逐切片计算的并行算法

如算法4.1的描述所示，SpTTM 逐切片计算方法在 CPU-GPU 平台上的执行流程可以细分为以下四个步骤：

1. 在给定的 CPU-GPU 平台上，将稀疏张量划分为两个切片集合：$S_{CPU_\mathcal{X}}$ 和 $S_{GPU_\mathcal{X}}$。
2. 通过 SPT-GCN 模型，为 $S_{CPU_{\mathcal{X}}}$ 和 $S_{GPU_{\mathcal{X}}}$ 分别确定最佳的存储格式，分别标记为 F_{CPU} 和 F_{GPU}。并且最佳存储格式的选择必须满足一定的条件，即 F_{CPU} 和 F_{GPU} 必须属于 COO、CSR、ELL 和 HYB 这四种稀疏格式之一。一旦选择了适当的稀疏格式，便可将两个切片集转换为相应的选定格式。

中国知网 https://www.cnki.net

算法 4.1 CPU-GPU 平台上 SpTTM 逐切片计算的并行算法

输入:

张量 $\mathcal{X}$;

矩阵 U;

相乘的模态 J;

输出:

张量 $\mathcal{Y}$;

1: 通过任务映射策略设置 $S_{CPU_\mathcal{X}}$ 和 $S_{GPU_\mathcal{X}}$;
2: 通过 SPT-GCN 为 $S_{CPU_\mathcal{X}}$ 和 $S_{GPU_\mathcal{X}}$ 选择一种最佳格式 F_{CPU} 和 F_{GPU}，并转换这些切片的格式;
3: # pragma omp parallel
4: {
5: # pragma omp sections nowait
6: {
7: // GPU 并行部分
8: # pragma omp section
9: {
10: $S_{GPU_\mathcal{Y}} \leftarrow$ 调用 F_{GPU} 核函数 $SpTTM_{GPU}$ ($S_{GPU_\mathcal{X}}$);
11: }
12: // CPU 并行部分
13: # pragma omp section
14: {
15: $S_{CPU_\mathcal{Y}} \leftarrow$ 调用 F_{CPU} 核函数 $SpTTM_{CPU}$ ($S_{CPU_\mathcal{X}}$);
16: }
17: }
18: }
19: $\mathcal{Y} \leftarrow$ 拼接 $S_{GPU_\mathcal{Y}}$ 和 $S_{CPU_\mathcal{Y}}$;
20: **return** $\mathcal{Y}$;

3. 将 $S_{GPU_\mathcal{X}}$ 以 F_{GPU} 对应的格式存储在 GPU 上，通过 CUDA 执行 $SpTTM_{GPU}$ 获取 $S_{GPU_\mathcal{Y}}$，$S_{CPU_\mathcal{X}}$ 以 F_{CPU} 的对应格式存储在 CPU 上，并利用 OpenMP 并行执行 $SpTTM_{CPU}$ 操作，得到 $S_{CPU_\mathcal{Y}}$。
4. 在 CPU 上收集结果 $S_{GPU_\mathcal{Y}}$ 和 $S_{CPU_\mathcal{Y}}$，最终拼接得到结果张量 $\mathcal{Y}$。

不同格式在 CPU 和 GPU 上的执行方式也存在差异，为了支持异构调用，这四种稀疏格式需要进行并行化设计。对于每种格式，本节都提供了相应的线程并行执行的计算方式。

中国知网 https://www.cnki.net

对于以 COO 格式存储的切片，由于 COO 中的值是相互独立的，因此乘法运算可以在值之间直接并行执行。然后进行分段归约，最终实现中间结果的累加。在 CPU 上的并行方法中，非零值被分配给执行值乘法的线程，每个线程执行分段归约并对中间结果求和。而在 GPU 上，一个线程束中的连续线程获取相邻的值，最终通过交错归约得到最终结果。

对于以 CSR 格式存储的切片，其中包含的所有非零值均经过紧凑的连续压缩。在 CPU 上，每一行的计算任务都被独立地分配给了 CPU 中的一个线程，这样每个线程可以独立地执行矩阵向量乘法，从而获得最终结果切片的一行。在 GPU 上，可以为每一行分配了一个 CUDA 线程，并按列优先的顺序存储数据，以实现更佳的内存访问合并效果。这种列优先的内存布局确保了切片中的每一行都能够独立进行计算，而不会混淆中间结果。在计算过程中，需要对切片中的各行进行遍历，遍历次数等于输入矩阵的列数。通过利用低延迟的共享内存来保存这些行的数据，可以有效地减少对全局内存的访问次数。

对于以 ELL 格式存储的切片，其中的非零值被紧密连续地存储，切片经过压缩而变得更加密集。所有的非零值被打包到两个相同大小的矩阵数组中，以便存储每个非零值及其对应的列索引。ELL 格式的切片通过零填充的方式，确保每一行都包含相同数量的值。因此，ELL 格式中非零值和列索引矩阵数组的行数与切片的行数相同，但列数则受制于切片中所有行中最大的非零值数量。在 CPU 上，每个 CPU 线程负责计算结果切片中的一行，以便允许该线程同时访问这行对应的非零值和列索引数组。这种方法将一行的数据作为执行单元，有效避免了线程间的写-读冲突。在 GPU 上，本章通过 CUDA 线程束来访问输入矩阵的每一行，而不是访问扩展和合并的输入切片。这种方式确保了可重复利用性，使一个线程束及同一个线程束内的线程能够紧凑地访问切片中的每个非零值。

对于以 HYB 格式存储的切片而言，整个切片会被基于特定阈值分为稠密和稀疏两个部分，并采用两种不同的稀疏格式保存，即 ELL 和 COO。为了最小化 ELL 中零值的填充，每一行都会被分割以适应 ELL 和 COO 格式。具体而言，每行中小于阈值的部分存储为 ELL 格式，超出阈值的其余值则保存为 COO 格式。阈值通常取 4096 和 $row_number/3$ 的最大值。HYB 计算的线程分配与 ELL 相似，只是多分配了一些线程来执行 COO 操作。

3、 存在的关键问题

在 CPU-GPU 平台上实现 HS-SpTTM 方法，需要解决以下两个关键问题。首先，针对 CPU 和 GPU 两种计算资源在计算强度上的差异，需要设计合理的负载均衡策略。其次，考虑到张量稀疏分布的多样性，必须选择合适的稀疏张量格式存储张量。

中国知网 https://www.cnki.net

4.3 负载均衡的任务映射策略

在配备了 CPU 和 GPU 的异构平台上，为了实现应用程序的高效执行，需要利用这两种设备的计算资源。如果应用程序只在 CPU 或 GPU 上执行，将导致计算资源的浪费。因此，为了更高效地进行 SpTTM 计算，本章需要设计一种负载均衡策略，以便 CPU 和 GPU 可以同时进行计算。在 HS-SpTTM 方法中，一个关键步骤是将 SpTTM 分解为多个矩阵乘法计算任务。如何设计高效的任务映射方案，以便这些任务可以均衡地分配到不同的设备上，是首要需要解决的问题。

任务映射方案的设计需要考虑四个因素：

1. 不同的计算任务具有不同的计算量，需要进行合理的分配。
2. CPU 和 GPU 的计算能力存在差异，需要考虑如何根据它们的能力进行任务分配。
3. 不同的计算任务具有不同的内存访问量，需要避免将内存访问量大的任务都映射到内存访问量较小的设备上。
4. CPU 和 GPU 之间的数据传输开销也有所不同，需要尽量减少数据传输的开销。

为了解决上述问题，HS-SpTTM 包含了一种性能分析模型，用于确定负载均衡任务映射方案。

给定一个异构平台，CPU 的峰值计算能力记为 P_c，其计算方式如下：

$$P_c = Num_c \times F_c \times FMA_c \tag{4.1}$$

其中 Num_c 是 CPU 上的核心数，F_c 是 CPU 的主频，FMA_c 是 CPU 在单个周期中执行的浮点运算的数量，即乘法和加法的次数。GPU 的峰值计算能力为 P_g，其计算方式如下：

$$P_g = Num_g \times F_g \times FMA_g \tag{4.2}$$

其中 Num_g 是 GPU 上 CUDA 核心的数量，F_g 是 GPU 的主要频率，FMA_g 是 GPU 在单个周期内执行的浮点运算的数量。用待计算的数据量表示内存访问次数，总数据量表示为记为 M，CPU 上的数据量和内存访问次数记为 M_c，GPU 上的数据量和内存访问次数记为 M_g，则

$$M = M_c + M_g \tag{4.3}$$

分配给 CPU 的计算任务量为 Q_c，分配给 GPU 的计算任务量为 Q_g。因此假设计算任务量和数据量之间存在线性关系，给定相关系数为 μ，计算任务量可以表

中国知网 https://www.cnki.net

示为：

$$Q_c = \mu M_c, \quad Q_g = \mu M_g \tag{4.4}$$

CPU 和 GPU 上的计算时间分别记为 t_{cp} 和 t_{gp}，假设实际计算能达到峰值计算性能的百分比记为 λ，则 CPU 和 GPU 上的计算时间表示为：

$$t_{cp} = \frac{Q_c}{\lambda P_c}, \quad t_{gp} = \frac{Q_g}{\lambda P_g} \tag{4.5}$$

则 CPU 和 GPU 上的内存访问时间分别记为 t_{cm} 和 t_{gm}，计算过程表示为：

$$t_{cm} = \frac{M_c}{B_c}, \quad t_{gm} = \frac{M_g}{B_g} \tag{4.6}$$

其中 B_c 是 CPU 中主存的带宽，B_g 是 GPU 中全局内存的带宽。假设数据在 CPU 和 GPU 之间传输的带宽为 B，将数据从 CPU 传输到 GPU 的时间记为 t_r，其计算过程表示为：

$$t_r = \frac{M_g}{B} \tag{4.7}$$

鉴于结果张量可能在后续计算过程中继续在 GPU 或 CPU 上执行，因此，目前不涉及结果数据的传输时间。

为了便于后续计算，本章将执行时间分为计算时间和内存访问时间[132]。负载均衡策略需要使 CPU 上的执行时间等同于 GPU 上的执行时间，表示为：

$$t_{cp} + t_{cm} = t_{gp} + t_{gm} + t_r \tag{4.8}$$

代入公式（4.5）、（4.6）、（4.7）、（4.8），可以得到：

$$\frac{Q_c}{\lambda P_c} + \frac{M_c}{B_c} = \frac{Q_g}{\lambda P_g} + \frac{M_g}{B_g} + \frac{M_g}{B} \tag{4.9}$$

然后合并公式（4.3），（4.4）和 (4.9) 来获取分配给 CPU 的任务数量和分配给 GPU 的任务数量。公式（4.4）表明 Q 和 M 是线性关系，所以本章以 Mg 为例来解释 CPU 和 GPU 之间的分配关系。公式（4.9）可以进一步表示为

$$\frac{\mu(M - M_g)}{\lambda P_c} + \frac{M - M_g}{B_c} = \frac{\mu M_g}{\lambda P_g} + \frac{M_g}{B_g} + \frac{M_g}{B} \tag{4.10}$$

中国知网 https://www.cnki.net

因此，M_g 可以表示为：

$$M_g = \frac{P_g B_g B(\mu B_c + \lambda P_c)}{P_g B_g B(\mu B_c + \lambda P_c) + P_c B_c(\mu B_g B + \lambda P_g B + \lambda P_g B_g)} M \tag{4.11}$$

最终结合公式（4.3），（4.4）和（4.11），计算任务量和数据量之间的关系表示为：

$$Q_g = \mu M_g = \frac{\mu P_g B_g B(\mu B_c + \lambda P_c)}{P_g B_g B(\mu B_c + \lambda P_c) + P_c B_c(\mu B_g B + \lambda P_g B + \lambda P_g B_g)} M \tag{4.12}$$

$$Q_c = \mu(M - M_g) = \frac{\mu P_c B_c(\mu B_g B + \lambda P_g B + \lambda P_g B_g)}{P_g B_g B(\mu B_c + \lambda P_c) + P_c B_c(\mu B_g B + \lambda P_g B + \lambda P_g B_g)} M \tag{4.13}$$

在 HS-SpTTM 方法中，CPU 和 GPU 上计算任务的分配具有密切的关联。首先，通过计算每个切片中的非零值数量，并将非零值的总数确定为待计算的数据量 M。然后，根据每个切片的数据量从大到小对切片进行排序。随后，切片按照从大到小的顺序依次分配给 GPU 和 CPU，以确保它们的任务量能够尽可能接近上述 Q_c 和 Q_g 之间的比例。最终，通过综合考虑上述所有的分析过程，可以得出一个负载均衡的任务映射方案。

4.4 SPT-GCN 格式选择模型

本节将详细介绍格式选择方法的具体内容。首先通过使用二分图来表示张量的稀疏结构，给出了一个简洁清晰的表述方式。然后对图神经网络的基本概念介绍，以便为后续的模型解释打下基础。最后详细描述了稀疏张量格式选择模型 SPT-GCN 的网络结构以及超参数设置。

4.4.1 稀疏张量的图表示

张量降低技术被广泛应用于处理高维空间中的张量，其主要目的将张量中的稀疏信息有效地压缩到一个特定的矩阵中，该矩阵被称为映射矩阵。映射矩阵是由张量的所有切片所投影而成的，因此其形状与张量的切片形状相对应。在映射矩阵中，每个数值代表了在张量的所有切片中相应位置的非零值数量，如图4.4中所示。

二分图被广泛用于研究矩阵的行和列分割问题[108,113,133]，因此，映射矩阵的结构也可以被描述为一个二分图。通过对张量进行变换所得到的二分图，属于一种带权无向图，其中包含两个顶点集，分别代表映射矩阵的行和列。映射矩阵中的每个非零值都以连接这两个顶点集的边的形式进行表示，而每条边上都携带着

中国知网 https://www.cnki.net

相应的权值，该权值的大小等于对应非零值的数值。

矩阵 X 由带权无向图 $G=(R,C,E,W)$ 表示。如果矩阵 X 在第 2 行和第 3 列有非零值 3，则在图 G 中会有一条边 $e_{2,3}=(r_2,c_3,w_{2,3})$ 从属于顶点集 R 节点 r_2 连接到属于顶点集 C 的节点 c_3，其中 $w_{2,3}$ 的值为 3，是矩阵 X 中的一个非零值。

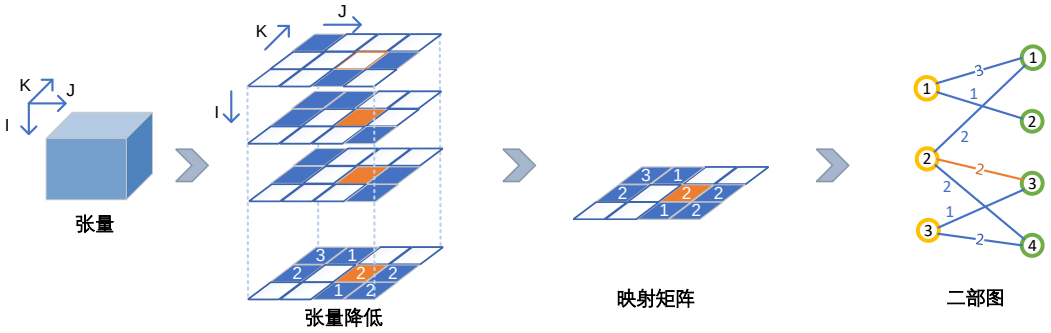


图 4.4　将张量转换成二部图的过程示意图

4.4.2　图卷积神经网络

符号定义： $G=(V,E)$ 被用来表示一张图，其中 $V=\{v_1,\cdots,v_n\}$ 表示节点的集合，E 表示边的集合。节点 $v_i\in V$ 代表图中的节点，边 $e_{i,j}=(v_i,v_j)\in E$ 表示从节点 i 到 j 的边。节点 v 的邻域被定义为 $N(v)=\{u\in V|(v,u)\in E\}$。图的邻接矩阵用 $A\in\mathbb{R}^{n\times n}$ 表示，若 $e_{ij}\in E$，则 $A_{ij}=1$，否则 $A_{ij}=0$。若每个图节点都具有 d 个特征，则节点 v_i 的属性向量记为 f_{v_i}，而图的属性矩阵则表示为 $F\in\mathbb{R}^{n\times d}$。

一个基本的 GCN 模型由一组神经网络层组成。每一层都收集每个节点周围的局部邻域信息，然后将这些信息传递到下一层[134]。假设中心节点为 v，特征向量记为 $f(v)$，邻居节点集合记为 $N(v)$，则第 t 层中心节点 v 的新特征表示为：

$$f^{(t)}(v)=\sigma\left(f^{(t-1)}(v)\cdot\mathbf{W}_1^{(t)}+\sum_{u\in N(v)}f^{(t-1)}(u)\cdot\mathbf{W}_2^{(t)}\right)\tag{4.14}$$

其中 $\mathbf{W}_1$ 和 $\mathbf{W}_2$ 是参数矩阵，σ 是激活函数。然后通过替代上述公式使用排列不变的可微函数，并计算新的公式 $f^{(t)}(v)$，

$$f^{(t)}(v)=f_{merge}^{(\mathbf{W}_1)}\left(f^{(t-1)}(v),f_{aggr}^{(\mathbf{W}_2)}\left(f^{(t-1)}(u)\right)\right)\tag{4.15}$$

其中 $u\in N(v)$ 是 v 的邻居节点，$f_{aggr}^{(\mathbf{W}_2)}$ 是对节点 v 的邻域特征进行合并的操作，$f_{merge}^{(\mathbf{W}_1)}$ 表示对节点 v 的当前特征与其邻域特征[135] 进行聚合的操作。

因此，整个图的向量表示 $f_{GCN}(G)$ 可以通过对所有节点的向量表示求和得

中国知网　https://www.cnki.net

到，结果表示如下：

$$f_{GCN}(G) = \sum_{v \in V(G)} f^{(T)}(v) \tag{4.16}$$

其中 $T > 0$ 表示图神经网络的最后一层[136]。

4.4.3 SPT-GCN 的结构设计

如图4.5所示，SPT-GCN 结合了 GCN 和多层感知机来预测张量的最佳存储格式[114]。SPT-GCN 的输入是由张量转换而来的二分图。具体而言，GraphConv 的一个输入节点表示映射矩阵的一行或一列。为了反映张量稀疏非零值的空间分布，考虑额外的统计属性作为节点的特征，包括该行/列非零值的**总数**，该行/列所有非零值的列/行索引的**最大值**、**最小值**、**平均值**和**方差**。

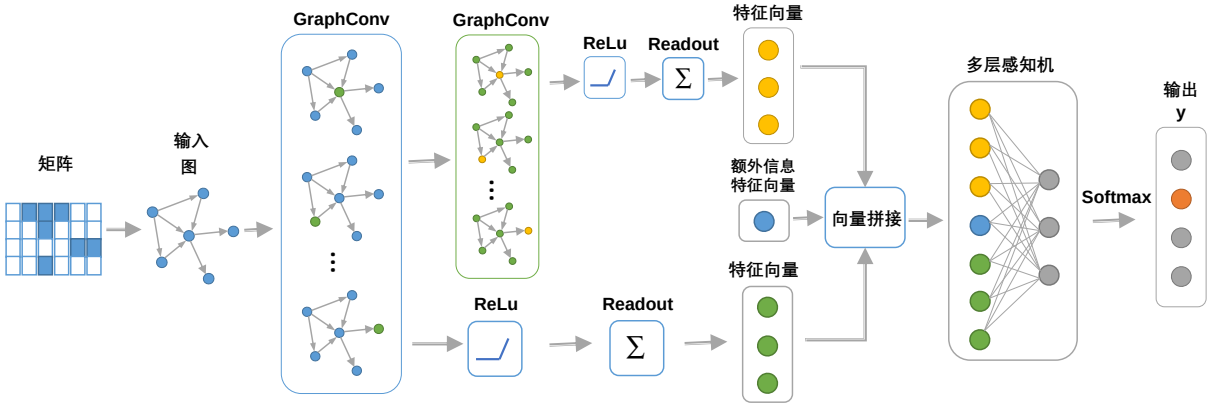


图 4.5　SPT-GCN 模型结构示意图

考虑到高阶结构在矩阵图表示中的重要作用，k-维图神经网络[105] 被用来聚合高维邻域信息。对于每一层 $t > 0$，新特征的计算过程可以表示为：

$$f^{(t)}(v) = \sigma\left(f^{(t-1)}(v)\mathbf{W}_1^{(t)} + \sum_{u \in N_L \cup N_G} f^{(t-1)}(u)\mathbf{W}_2^{(t)}\right) \tag{4.17}$$

其中 N_L 为节点 v 的局部邻域节点集合，N_G 为节点 v 的全局邻域节点集合。在 k 维图神经网络中，局部邻域是指 k 维中的所有邻域节点，全局邻域是指除局部邻域[105] 外的所有邻域节点。

SPT-GCN 模型的工作过程主要分为两个阶段。

第一阶段是图卷积阶段，旨在从图数据中获取知识。首先，对输入图进行图卷积运算，这里使用了 GraphConv 操作符，其目的是聚合相邻节点的信息。该操作会得到一阶图卷积结果。之后，对一阶图卷积结果再次执行 GraphConv 操作以获得二阶图卷积结果。每个图卷积操作都包括 GraphConv、ReLU 和 Readout 三个操作符。ReLU 是一种池化操作，用于通过对节点进行下采样减小参数的大小并

中国知网 https://www.cnki.net

产生更小的表示，从而避免过拟合和计算复杂等问题。最后，Readout 操作用于在节点表示的基础上生成图级表示。

第二阶段是使用多层感知机为矩阵对选择合适的稀疏格式。这一阶段的目的是为输入的张量选择适合的稀疏表示。第一阶段的输出被作为多层感知机的输入，并最终输出每种张量格式实现最佳性能的概率，预期具有最高概率的张量格式将是最适合给定张量的。

模型在被应用到张量格式选择后，需要被进一步优化。具体而言，当 k 维图神经网络的 k 值增大时，节点的感知域就会扩大，但同时相邻节点间可能会多次相互传播特征，从而产生冗余。尤其是当 k 值增大时，冗余条目数量也会增加，这对模型的最终预测结果是不利的。因此，经过多次试验，二维图神经网络被用来聚合节点的两跳邻居信息。这种二维聚合方式既可以利用更多节点的信息，同时也可以避免因 k 值过大而产生的冗余信息问题。

在具体操作上，二维图神经网络有两个图卷积过程。在第一个图卷积之后，输出结果表示长度为 128 的向量。在第二个图卷积之后，输出结果表示为长度为 128 的向量。这两个向量将会被拼接在一起，得到二维图神经网络的最终输出，它是一个长度为 256 的向量。最终输出的向量将会被送入一个多层感知机进行处理。

除了拼接两个 GCNs 输出的向量外，多层感知机的输入还包含一个包含额外信息的特征向量。该向量的长度为 4，它记录了一些关于张量整体相关的统计信息：张量的模态数、张量的稀疏度、张量的最大维数以及张量的最小维数。因此，多层感知机的输入层有 516 = 512 + 4 个神经元。此外，两个隐藏层的神经元数量分别被设置为 128 和 32，多层感知机输出层的神经元数量被设置为稀疏格式的类别数。

在训练过程中，经多次尝试以寻找最佳的超参数组合，包括批量大小和学习率。最终，确定了批量大小为 4 和学习率为 0.0001 的参数组合。为了提高操作效率和数据稳定性，损失函数采用了 *LogSoftmax* 和 *NLLLoss* 的组合。

4.5 实验评估

本节通过实验详细评估了 HS-SpTTM 的计算性能、SPT-GCN 的预测精度以及负载均衡方案的有效性。所有性能对比实验均采用单精度浮点数进行计算，并记录 10 次运行结果的平均值。

4.5.1 平台和数据集

1、 硬件和软件系统

使用配备了 Intel Xeon Gold 6248 CPU 和 NVIDIA V100 GPU 的异构系统作为性能对比实验的平台，该平台的硬件配置如表4.1所示。SPT-GCN 是使用 Python 3、

中国知网 https://www.cnki.net

PyTorch 1.4 和 PyTorch Geometric 1.6[115] 实现的。

表 4.1　SpTTM 实验平台配置总结表

参数	Intel(R) Xeon(R) Gold 6248 CPU	NVIDIA V100 GPU
微架构	Cascade Lake	Volta
频率	2.50 GHz	1.455 GHz
物理核心数	20	5120
最后一级缓存大小	27.5 M	16 M
内存大小	132 G	32 G
内存带宽	17.6 GB/s	900 GB/s
编译器	GCC 5.5.0	NVCC 10.1

2、 数据集

为了优化 SPT-GCN 的训练效果，选取大量的张量作为训练集。这些训练集主要来源于 SuiteSparse 矩阵集合中挑选的 2016 个矩阵所生成的 4426 个三阶张量。这些张量的非零值的数量在 42 至 2652858 范围内。各种稀疏张量存储格式均用于评估 SpTTM 的性能，包括 COO、CSR、ELL 和 HYB 等格式。训练集中张量的标签是由时间开销最低的存储格式来确定的。

测试数据来自于四个不同应用领域的数据集，包括 Uber Pickups、Hetrec2011 Lastfm 2k、Chicago Crime 和 NeurIPS Publications。这些数据集均采集自实际应用 *FROSTT*[①] 的数据。表4.2中呈现了这些数据集的维度特征。

表 4.2　SpTTM 实验中张量数据集的总结表

张量数据集名称	字段描述	I	J	K	非零元数
Uber Pickups	（时间，纬度，经度）	4,392	1,140	1,717	3,309,490
Hetrec2011-lastfm-2k	（用户，艺术家，播放量）	2,100	18,744	12,647	186,479
Chicago Crime	（时间，区域，类型）	148,464	77	32	5,330,673
NeurIPS Publications	（文章编号，作者，关键词）	42,194	2,862	14,036	3,101,609

3、 基准方法

机器识别稀疏矩阵结构最常用的方法是 SVM[69] 和 CNN[65]。因此，这两种方法被选为与 SPT-GCN 比较的对比方法。SVM 方法以稀疏矩阵的非零值及其坐标信息作为特征，并根据这些特征对矩阵进行分类。CNN 方法则将稀疏矩阵转化为

① frostt.io/tensors/

中国知网 https://www.cnki.net

直方图或稠密图的形式，然后通过卷积神经网络进行特征提取和分类。该方法可以利用深度学习的强大特征提取能力，自动学习输入数据的内在特征。

实验选用两个先进的张量库 ParTI! 和 SPLATT 作为性能比较的基线方法，以评估 HS-SpTTM 方法的效果。ParTI! 库分别在 CPU 和 GPU 上的实现了 SpTTM 运算，因此，为了便于区分，将在 CPU 多核上的实现记为 ParTI!-OMP，在 GPU 上的实现记为 ParTI!-GPU。实验针对四个张量数据集分别测试了 ParTI!-OMP、ParTI!-GPU，以及 HS-SpTTM 方法。

对于 HS-SpTTM 方法，将使用 SVM 作为格式选择模型的对比实验记为 AdaptSVM，使用 CNN 作为格式选择模型的对比实验记为 AdaptCNN，和使用 SPT-GCN 作为格式选择模型的对比实验记为 AdaptSPT-GCN。

为了更具体地展示异构计算的性能优势，实验还比较了仅在 CPU 上使用 SPT-GCN 模型的对比实验，记为 CPUSPT-GCN 和仅在 GPU 上使用 SPT-GCN 模型的对比实验，记为 GPUSPT-GCN。

4.5.2 性能比较

1、 实验设置

为了评估 HS-SpTTM 的性能，本实验记录了在不同张量数据集上执行 SpTTM 的时间开销。在诸如张量分解等实际应用中，通常需要对张量的所有模态执行 SpTTM 操作。因此，实验记录了张量数据集在每个模态下执行一次 SpTTM 计算所需的时间开销，并使用其平均值作为结果展示。此外，除了计算时间之外，对于 GPU 上的实现库还记录了 CPU 与 GPU 之间的数据传输时间。

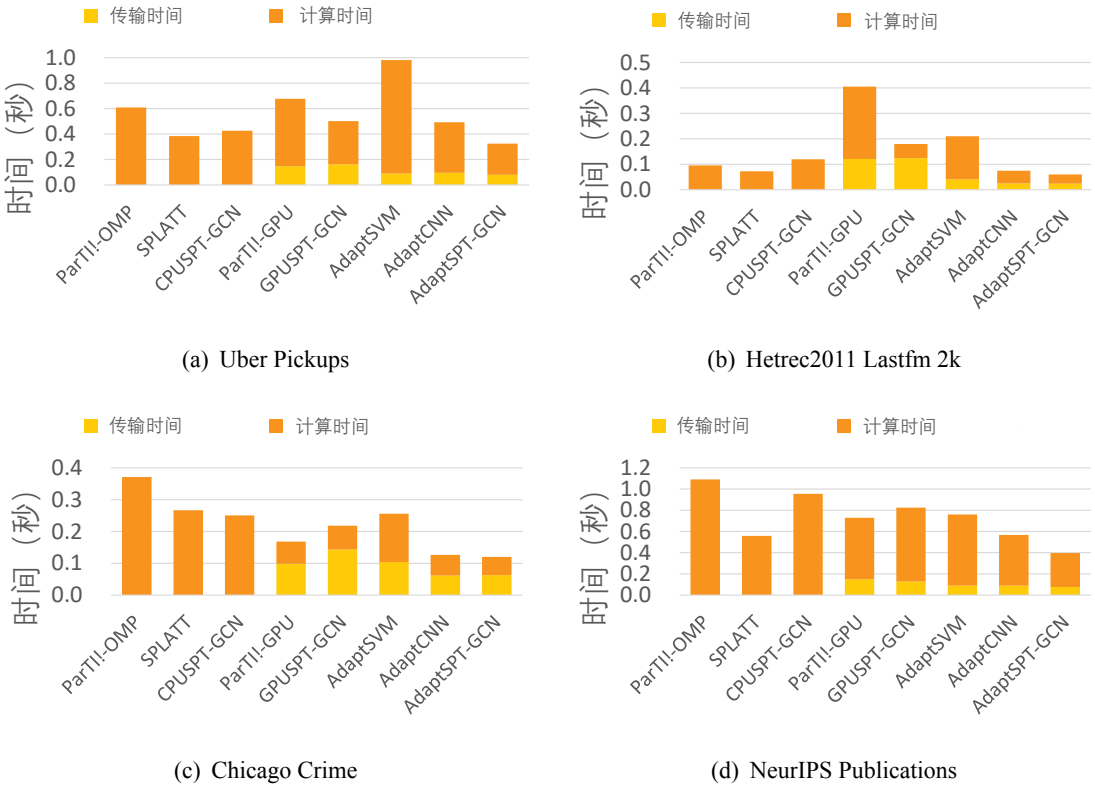
2、 时间开销

时间开销的对比实验结果如图4.6（a），（b），（c），（d）所示。AdaptSPT-GCN 与 AdaptCNN 以及 AdaptSVM 的结果进行比较表明，在优化 SpTTM 计算性能方面，相对于 CNN 和 SVM，SPT-GCN 具有更显著的加速效果。图4.7详细展示了各方法的预测精度和性能。在图中，SVM（GPU-62.77% 和 CPU-63.76%）表明 SVM 在 GPU 上的预测精度为 62.77%，在 CPU 上的预测精度为 63.76%，CNN 和 SPT-GCN 也具有类似的表现。实验结果表明，张量稀疏格式选择模型预测精度越高，其性能提升越显著。

图4.8展示了本章实现与 SPLATT 和 ParTI! 库的比较结果。可以看出，对于所有实现的库，格式转换的时间是一个较大的开销。然而，在张量分解应用中，SpTTM 已经迭代了数十次甚至数百次，因此格式转换的时间开销可以被这些迭代分摊。

与 CPUSPT-GCN 和 GPUSPT-GCN 进行比较的结果表明，AdaptSPT-GCN 异构计算方法的性能提升更显著。具体而言，与仅使用 CPU 或 GPU 相比，CPU-GPU

中国知网 https://www.cnki.net



(a) Uber Pickups　(b) Hetrec2011 Lastfm 2k

(c) Chicago Crime　(d) NeurIPS Publications

图 4.6　不同方法实现的 SpTTM 性能比较的结果图

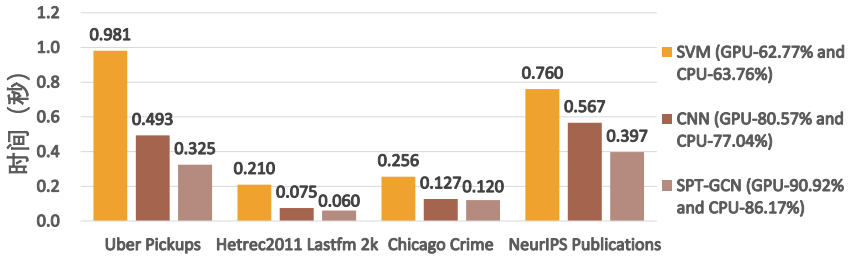


图 4.7　不同方法预测最佳存储格式的准确率及 SpTTM 性能比较的结果图

异构方法的性能提高了 45.98%，比 GPU 方法提高了 49.62%。此外，相对于 SPLATT 方法，AdaptSPT-GCN 方法获得了 1.50× 的加速效果，而相对于 ParTI!-OMP 和 ParTI!-GPU 方法，分别获得了 2.64× 和 1.31× 的加速效果。

4.5.3　负载平衡

1、　实验装置

为了验证所设计的负载均衡策略的效能，本章比较了无负载均衡和负载均衡后的时间开销。其中无负载均衡的任务配分方案指的是在 CPU 和 GPU 上随机分配切片。

中国知网　https://www.cnki.net

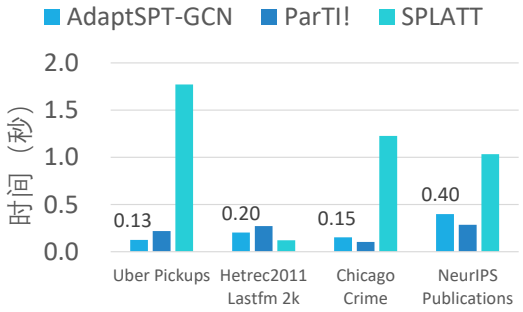
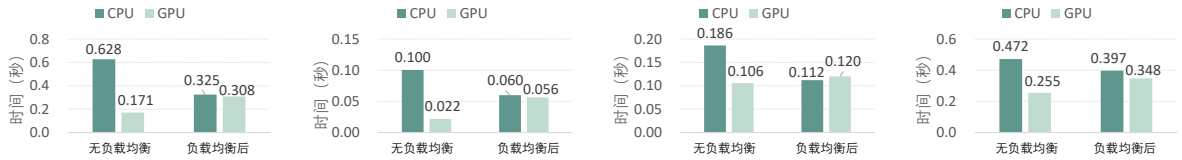


图 4.8　不同格式转换方法的时间开销结果图

2、 时间开销

图4.9中展示了四个张量数据集上的时间开销对比情况，包括是否采用负载均衡策略。可以观察到，在未应用负载均衡时，CPU 和 GPU 上的时间开销存在显著差异。这一差异主要受限于异构计算的时间开销，其受到 CPU 和 GPU 上的最大时间开销的制约，而二者之间的时间开销存在较大差距，从而导致了整体性能的下降。相比之下，应用负载均衡策略后，CPU 和 GPU 上的时间开销差距显著减小。具体而言，负载均衡策略的使用使平均性能相较于未采用负载均衡策略提高了 34.91%。这一结果表明了在 CPU-GPU 上采用负载均衡的任务分配方式是有效的。



(a) Uber Pickups　(b) Hetrec2011 Lastfm 2k　(c) Chicago Crime　(d) NeurIPS Publications

图 4.9　负载均衡策略对时间开销影响的结果图

4.5.4　格式选择的预测精度

1、 实验装置

交叉验证是统计学习中常见的评估方法，因此采用五折交叉验证来将验证数据从训练数据中分离出来。具体操作如下：首先，从数据集中随机选择并移除 20% 的数据以构建验证集，而剩余 80% 的数据则用于训练。然后，重复这一过程五次，每次选择不同的 20% 数据子集作为验证集。

为了评估 SPT-GCN 模型预测的准确性，本章采用了平均预测精度作为评价指标。具体而言，通过将 SPT-GCN 模型在验证集上成功预测出与标签相同类别的张量数量记为 m，将验证集中所有张量的总数记为 n，则准确率可以表示为 $\frac{m}{n}$。

中国知网　https://www.cnki.net

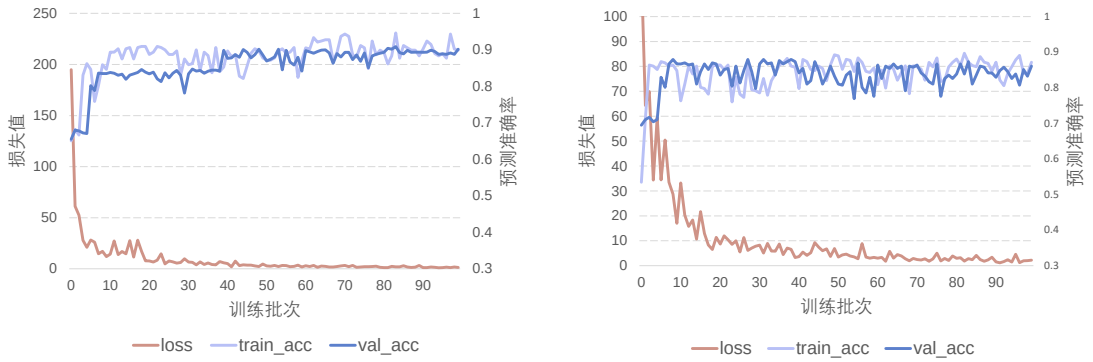
对于五折交叉验证，平均准确率可表示为：

$$acc = \frac{\sum_{f=1}^{F} \frac{m_f}{n_f}}{F} \tag{4.18}$$

其中 F 的值是 5，m_f 和 n_f 分别代表了与标签相同类别的张量的数量，以及在验证集中张量的总数。

2、 预测精度

SPT-GCN 训练过程的结果如图 4.10（a）和（b）所示。随着训练时间的增加，模型的损失值呈逐渐减小的趋势。最终，损失值趋于稳定，表明 SPT-GCN 模型已经达到了收敛状态。此外，train_acc 和 val_acc 的结果也显示，不论是在训练集还是验证集上，SPT-GCN 模型的预测准确性逐渐趋于稳定。具体而言，在 CPU 上，训练集的准确率稳定在 87.83%，验证集的准确率稳定在 85.12%。而在 GPU 上，训练集的准确率稳定在 92.41%，验证集的准确率稳定在 90.13%。



(a) CPU　　(b) GPU

图 4.10　SPT-GCN 的训练过程的结果图

为了深入研究本方法与其他方法的差异，整个数据集将按照非零值的个数划分为五个不同的子集，并分析每个子集上的结果。

不同子集的统计结果已被记录在表格4.3中。观察到在本章所使用的训练数据集中，高达 85.72% 的张量中非零值数量在 100 至 100000 个之间，而只有 5.69% 的张量中非零值数量少于 100，8.59% 的张量中非零值数量超过 100000。

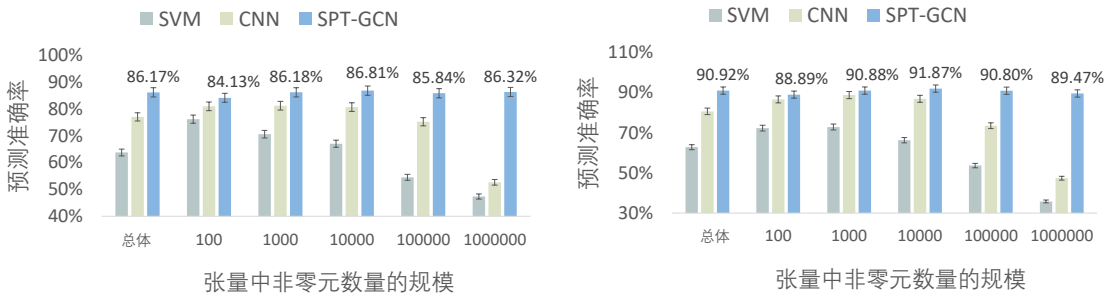
表 4.3　训练数据集中不同大小张量所占比例的总结表

张量的非零元数	0-100	100-1,000	1,000-10,000	10,000-100,000	>100,000
张量的个数	252	1360	1304	1130	380
百分比	5.69%	30.73%	29.46%	25.53%	8.59%

图4.11（a）和（b）呈现了 SPT-GCN 与其他机器学习方法的预测精度比较结

中国知网　https://www.cnki.net

果。可以观察到，在 CPU 和 GPU 环境下，SPT-GCN 均优于其他方法，并且随着张量非零值规模的增加，这一优势更为显著。这一现象的原因在于，对于大型矩阵，CNN 在将其缩放为固定大小矩阵的过程中可能会损失过多的结构信息。同样，当 SVM 仅使用矩阵的整体特征对大规模矩阵进行分类时也可能会产生较大的误差。相比之下，SPT-GCN 的预测效果不受矩阵规模的影响。



(a) CPU　　(b) GPU

图 4.11　最佳稀疏格式预测的准确性比较的结果图

3、 开销分析

通过比较格式选择方法与遍历搜索方法的时间开销，以证明格式选择方法的优势。具体而言，在格式选择方法方面，对比的是 SVM、CNN 以及 SPT-GCN 方法。而在遍历方法方面，对比的是穷举遍历法和贪婪遍历法。

穷举遍历法代表了最常见的遍历方式，它会彻底搜索每一种方法，并选择具有最短运行时间的方法作为最优选项。而贪婪遍历法则是建立在穷举遍历法的基础之上，它以已知速度最快的方法为上限标准，即当后续方法的运行时间超过该上限时，便停止该格式的计算。因此，贪婪遍历法的时间开销将会普遍低于穷举遍历法。

图4.12呈现了各种识别最佳矩阵格式方法的时间开销，时间开销越高的方法对应的背景色更红。由于不同方法的数值差异较大，因此，采用对数尺度来描述时间开销的大小，例如，-1 表示 10^{-1} 秒级的运行时间开销。

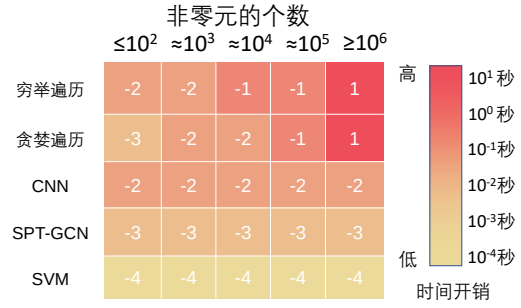


图 4.12　搜索最佳稀疏格式的时间开销的结果图

实验结果显示，在大多数情况下，格式选择方法的开销较低，相较之下，遍

中国知网　https://www.cnki.net

历方法的运行时间开销比格式选择方法至少高出一个数量级。CNN 的运行时间开销为 10^{-2} 秒级，而 SPT-GCN 为 10^{-3} 秒级。从数字上看，SPT-GCN 的运行时间开销略低于 CNN，表明 SPT-GCN 比 CNN 更快。而 SVM 的运行开销为 10^{-4} 秒级，尽管计算量较小，但其预测精确度较低。SPT-GCN 方法的运行时间开销略高于 SVM 方法，因为 SVM 仅利用了矩阵的统计特征，计算量相对较小。由于卷积运算的运行时间开销较大于图卷积运算，SPT-GCN 的开销比 CNN 低一个数量级。总而言之，与其他方法相比，SPT-GCN 在预测准确率和时间开销之间取得了最佳的权衡。

4.6 本章小结

本章以 SpTTM 为例，基于 HASTensor 中的优化技术提出了一种异构逐切片的 SpTTM 方法，即 HS-SpTTM，并展示了 HASTensor 库在批量计算型稀疏张量运算性能优化方面的有效支持。通过理论分析，该方法研究了不同处理器的峰值性能和稀疏张量运算的计算量之间的关系，并设计了一种任务映射机制，旨在提高 CPU-GPU 平台整体计算效率。在此基础上，引入图结构来描述张量非零值的分布，并构建了一种名为 SPT-GCN 的图神经网络模型，以智能地为张量选择合适的稀疏格式，并通过实验验证了其有效性。最后，本章比较了所提出的策略与 ParTI! 库和 SPLATT 库性能，实验结果显示，所提方法具有显著的性能优势。

中国知网 https://www.cnki.net

第 5 章　高维计算型稀疏张量运算的并行算法设计和性能优化

本章以 SpTC 为例，介绍了 HASTensor 中的高维计算型稀疏张量运算的并行算法设计和性能优化方法。SpTC 作为张量网络中的关键运算之一，通常操作稀疏的高维数据，代表着高维计算型稀疏张量运算的典型情况。该类计算要求处理器具备更强的计算性能和更高的内存带宽。为了提升 SpTC 的性能，充分发挥 GPU 的强大计算潜力被认为是一种可行的策略。然而，由于稀疏张量数据的高维和稀疏性，传统的 GPU 加速方法在应对这种计算任务时会遇到计算强度下降和内存占用过大的问题。尤其是最新引入的张量核心单元，虽然显著提升了 GPU 的计算能力，但也加剧了内存墙问题。

为了克服这些挑战，提出了一种基于位坐标位图的 SpTC 方法，即 BCB-SpTC。该方法首先设计了一种基于位操作的稀疏格式，通过线性化张量块的索引来减少张量块索引的访问次数，并使用位图来存储张量块中非零值的分布，从而降低存储开销。然后，将二进制线性索引划分为自由模态的索引和收缩模态的索引，以提高两个稀疏张量之间索引匹配的效率。随后，基于张量核心的特性构造了一种专用的数据填充方法，以改善在张量核心上处理稀疏数据时计算效率低下的问题。最后，在多个数据集上的实验结果表明，与现有的 SpTC 方法相比，BCB-SpTC 在 A100 GPU 上能够实现 1.10× 到 21.27× 的性能加速。

5.1　引言

SpTC 已经引起了来自不同领域的研究人员广泛的关注，包括量子物理[137,138]和机器学习[139–141] 等领域。在这些应用领域中，张量通常代表大型数据集或高维物理模型，SpTC 作为关键计算操作显得尤为重要。与线性代数中的矩阵乘法不同，张量缩并支持更高维度的计算，因此，设计高效的 SpTC 并行算法需要应对更多的挑战。

SpTC 面临的一个主要挑战是由其高维特性引发的多维索引匹配问题。在 SpTC 的计算过程中，需要匹配两个稀疏张量在多个模态下的相同索引，因此，多维索引匹配问题的计算复杂度会随着稀疏张量维度的增加而显著增加。这对稀疏张量索引的存储方式提出了新的挑战，即如何降低多维索引匹配问题所带来的计算复杂性。

另一个重要挑战是相对于其他稀疏张量运算，SpTC 的内存使用量更大，通

中国知网　https://www.cnki.net

常达到数千兆字节。具体而言，SpTC 计算过程中产生了过多的中间结果，而中间结果的存在加剧了对内存的需求，甚至超出了 GPU 的内存限制。这一问题尤为严重随着张量的维度和大小持续增长。

第三个挑战是稀疏性导致高维张量的访存离散化，使得计算强度降低的现象。计算强度是高性能计算领域一个常用的度量标准，用于量化算术操作与访存操作的比率。SpTC 的计算强度随着稀疏张量的维度增加会逐渐下降，如何有效地在计算过程中合并访存成为了设计高性能 SpTC 算法的一个关键所在。

硬件平台的发展促使了 SpTC 的性能优化成为重要的研究方向[25,142]。目前大部分工作都集中在稠密张量收缩[143,144]或者使用 CUDA 核心加速计算[43]方面。随着深度学习对计算能力的需求不断增加，最新引入的张量核心单元大幅提升了 GPU 的计算能力。然而，由于张量核心仅被设计用于支持小块稠密矩阵，大多数现有的稀疏张量存储格式与该计算硬件不兼容。并且稀疏张量通常具有高维稀疏特征和极其稀疏的非零值分布特征，使得设计张量核心加速 SpTC 的并行算法十分具有挑战性。

为了克服上述困难，本章的主要贡献如下：

1、 本章提出了一种称为位坐标位图（Bit Coordinate Bitmap，BCB）的稀疏张量存储格式，该格式直观地减少了内存占用和内存访问，并可用于加速数据索引匹配和内存访问，以减轻张量核心高速计算引起的内存墙问题。

2、 本章基于 BCB 格式设计了 SpTC 的并行算法，记为 BCB-SpTC，其引入了一种冲突解决方法，通过构建任务列表和消除任务之间的依赖关系，以增加线程块的占用率，从而提高 GPU 的利用率。

3、 本章实现了一个基于张量核心的内核函数，该函数可以根据不同的 SpTC 任务调整了张量核心的填充，以克服张量核心上稀疏数据的计算效率低下的问题。

4、 本章使用真实场景的数据集实验，研究结果表明，BCB 格式的内存占用比当前格式小，而 BCB-SpTC 在 A100 GPU 上相对于最先进的方法提高了 1.10× 到 21.27× 的加速比。

5.2 BCB 格式

本节将详细介绍用于存储稀疏张量的 BCB 格式，首先描述 COO 转换为 BCB 格式的过程来介绍 BCB 格式，然后阐述其中的存储优化技术，最后分析 BCB 格式的理论存储大小。

中国知网 https://www.cnki.net

5.2.1 格式转换

交集搜索是 SpTC 中的必要操作，用于搜索张量空间 $\mathcal{X}$ 和 $\mathcal{Y}$ 中具有相同收缩模态的索引 I_k 的交集。然而，该任务需要搜索张量整个空间，这是一项庞大的任务。为此，整个张量集合被分成多个子集，采用分块技术处理。BCB 格式将张量存储在张量块中，将稀疏张量视为稠密张量来处理。鉴于高维张量的稀疏性，导致了大量全零的空张量块的存在。这些空张量块对 SpTC 的结果没有影响，因此仅记录了包含非零值的非空张量块。

通过引入非零值的索引，COO 格式在稀疏张量中包含大量零元素时避免了内存传输开销，尽管计算强度相对较低。为了增加计算强度，需进一步考虑消除非零值索引中的冗余。因此，BCB 格式的核心思想在于将整数类型替换为位类型，以减少索引数据的冗余信息。位优化不仅有助于减少内存存储空间，还能执行集合的交集操作。BCB 格式以连续存储方式存储每个张量块的非零值，每个张量块中包含零值和非零值。一组与张量块中的每个元素对应的二进制数被用来标记每个非零值的位置，这组二进制数被称为位图索引。

如图5.1所示，COO 到 BCB 格式的转换过程主要分为三个步骤。首先，通过分块技术来增强数据的局部性。然后，将每个张量块内的非零值位置记录到位图索引中，以供张量核心查询。最后，将每个张量块的多维索引映射到二进制线性空间中，以降低多维匹配的复杂性，并减少计算过程中内存指令的访问次数。

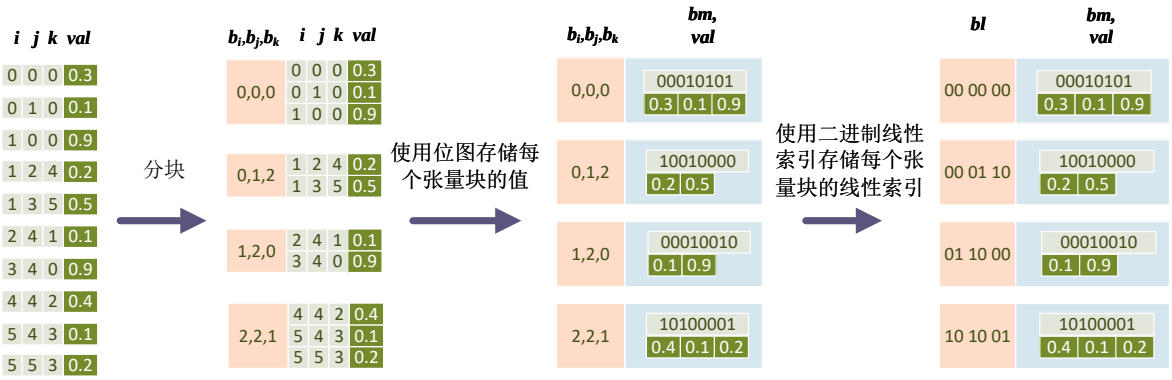


图 5.1 从 COO 到 BCB 的格式转换过程的示意图

5.2.2 分块技术

格式转换的第一步是采用分块技术将稀疏张量划分成多个张量块。分块技术是提高稠密张量计算效率的常见方法。在 BCB 格式中，稀疏张量以稠密张量的划分方式分块处理，得到的张量块由 b_i，b_j 和 b_k 记录其索引位置。

高维张量的稀疏性导致了稀疏张量分块后存在大量的不包含非零值的空张量块。这些空张量块对于 SpTC 的结果没有任何影响，因此分块后只需要记录包含非零值的非空张量块即可。通过对稀疏张量分块处理，使得每个张量块可以被单

中国知网 https://www.cnki.net

独处理，从而提高了张量收缩过程的效率。

总之，张量收缩过程需要快速匹配两个张量之间具有相同收缩模态的索引的非零值。分块技术是一种有效的策略，它通过利用数据的局部性，减小搜索空间的大小并提高搜索效率。通过减小搜索空间，可以显著提高计算效率。

5.2.3 位图存储技术

格式转换的第二步是利用位图存储的技术保存每个非空张量块中的非零元。具体而言使用位图索引 bm 存储每个张量块中的非零元的位置，和使用位图值数组 val 连续存储稀疏张量中的非零值。

位图索引是一组二进制数，每个二进制都对应着张量块中的一个位置，如果张量块内的某个位置上是零值，则位图索引中相应位置被置为 0，反之则置为 1。因此，通过检查位图索引中相应位置的状态，可以判断该位置上是否是非零值，并且可以通过位操作快速得到该张量块中包含非零值的数量。使用位图索引有助于减少张量块内记录非零值索引的冗余，从而提高 SpTC 的计算强度。

5.2.4 位压缩技术

格式转换的第三步是使用位压缩技术减少非空张量块索引的冗余。位压缩技术的核心思想在于将每个张量块的多维索引映射为二进制线性空间，以此来降低多维匹配过程的复杂性并减少内存访问次数。

该方法能够根据张量各维度的大小来动态地调整存储张量块索引的二进制数的数量，而不必采用固定大小的格式来存储张量不同维度的索引值。因此，通过利用位压缩技术将所有非空张量块的索引映射为二进制线性索引 bl，能够有效地减少存储索引所需的开销。

5.2.5 存储空间分析

对于包含 N 个非零值的稀疏张量 $\mathcal{X} \in \mathbb{R}^{S_1 \times S_2 \times \cdots \times S_K}$，假设张量块的大小为 $\mathcal{B} \in \mathbb{R}^{H_1 \times H_2 \times \cdots \times H_K}$。因此，经过位压缩后每个张量块的二进制线性索引占用的位空间大小 W_{bc} 为：

$$W_{bc} = \sum_{k=1}^{K} \left\lceil \log_2 \frac{S_k}{H_k} \right\rceil \tag{5.1}$$

此外，假设 $\mathcal{X}$ 有 M 个非空张量块，则存储所有张量块索引所需的存储空间大小 D_{bc} 为：

$$D_{bc} = W_{bc} \times M \tag{5.2}$$

中国知网 https://www.cnki.net

此外，张量块内的元素以位图格式存储，每个张量块的位图索引占用的位空间大小 W_{bm} 为：

$$W_{bm} = \prod_{k=1}^{K} H_k \tag{5.3}$$

则存储所有非空张量块的位图索引所需的位空间大小 D_{bm} 为：

$$D_{bm} = W_{bm} \times M \tag{5.4}$$

假设非零值的数据类型是单精度浮点型，即每个非零值占用 16 位的存储空间，存储非零值所需的存储空间大小 D_{val} 为：

$$D_{val} = 16 \times N \quad . \tag{5.5}$$

综上所述，以 BCB 格式存储的稀疏张量 $\mathcal{X}$ 所需的总存储空间大小 D_B 为：

$$D_B = D_{bc} + D_{bm} + D_{val} \quad = \left(\sum_{k=1}^{K} \left\lceil \log_2 \frac{S_k}{H_k} \right\rceil + \prod_{k=1}^{K} H_k \right) \times M \ + 16 \times N \quad . \tag{5.6}$$

5.3 BCB-SpTC 并行算法

基于 BCB 格式的 SpTC 算法设计面临一些挑战，其中包括多维匹配问题以及由维度和稀疏性引起的高内存开销。特别是，高维性质使得在高维空间中定位具有相同索引的非零值变得困难。

5.3.1 框架设计

基于 BCB 格式的实现的 SpTC 算法称为 BCB-SpTC，图5.2展示了该算法的概述，它主要分为四个部分。

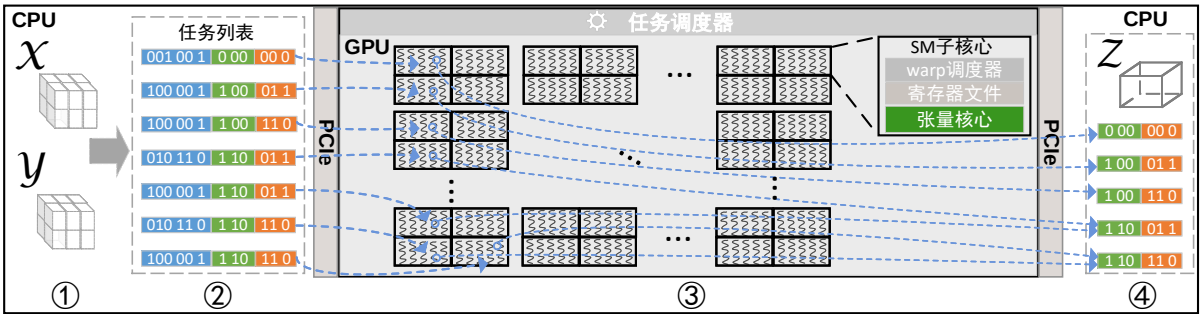


图 5.2 BCB-SpTC 并行计算框架的示意图

- 第①部分采用分块技术将张量划分为多个张量块，并以 BCB 存储格式存

中国知网 https://www.cnki.net

储。这一步骤的目的是过滤掉空的张量块，并通过将非空张量块的多维索引映射为二进制线性索引，以位图形式存储非空张量块中非零值的稀疏分布，从而压缩稀疏张量中多维索引的冗余。

- 第②部分负责生成任务列表，以捕获张量块之间的匹配关系，从而避免计算与无关的张量块。BCB 格式通过过滤空张量块和采用位压缩技术对多维索引处理，不仅将搜索空间从多维映射到一维，而且有效地缩小了搜索范围。
- 第③部分将由一对待计算的非空张量块组成的计算任务加载到 GPU 的张量核心上。这意味着张量核心以特定的数据加载方案读取非空张量块中的数据。
- 第④部分利用 GPU 上的一个线程束来计算一对张量块，线程束调度程序安排另一个活动线程束计算，以实现更高的吞吐量并解决负载不平衡的问题。最后，将具有相同自由模态的索引的结果从生成的张量块中累加，以确保返回的结果张量是正确的。

BCB-SpTC 的详细过程总结在算法5.1中。

算法 5.1 BCB-SpTC 算法

输入:

张量 $\mathcal{X} \in \mathbb{R}^{S_1^{\mathcal{X}} \times S_2^{\mathcal{X}} \times \cdots \times S_{d_{\mathcal{X}}}^{\mathcal{X}}}$;

张量 $\mathcal{Y} \in \mathbb{R}^{S_1^{\mathcal{Y}} \times S_2^{\mathcal{Y}} \times \cdots \times S_{d_{\mathcal{Y}}}^{\mathcal{Y}}}$;

输出:

张量 $\mathcal{Z} \in \mathbb{R}^{S_1^{\mathcal{Z}} \times S_2^{\mathcal{Z}} \times \cdots \times S_{d_{\mathcal{Z}}}^{\mathcal{Z}}}$;

/* 将张量的存储格式从 COO 转换为 BCB*/

1: $BCB_{\mathcal{X}} \leftarrow COO_TO_BCB(\mathcal{X})$;

2: $BCB_{\mathcal{Y}} \leftarrow COO_TO_BCB(\mathcal{Y})$;

/* 生成输入张量之间的任务列表，用于张量块之间的匹配 */

3: $T \leftarrow Task_List(BCB_{\mathcal{X}}, BCB_{\mathcal{Y}})$;

/* 执行张量收缩 */

4: **for** $(\hat{\mathcal{X}}, \hat{\mathcal{Y}})$ in T **do**

5: $\quad \hat{\mathcal{Z}} \leftarrow TC_TCUs(\hat{\mathcal{X}}, \hat{\mathcal{Y}})$;

6: **end for**

/* 删除结果张量中的零值 */

7: Compact_Zero($\mathcal{Z}$);

8: **return** $\mathcal{Z}$;

SpTC 的一个关键步骤是匹配两个张量中有相同收缩模态的索引的张量块，受扩展-排序-压缩（Expansion-Sorting-Compression，ESC）方法的启发[145]，

中国知网 https://www.cnki.net

BCB-SpTC 将任务列表生成过程分为三个阶段。

- 第一阶段被称为配对阶段，主要工作是具有相同收缩模态的索引的张量块之间相互配对。给定结果张量块的一个二进制线性索引，输入张量分别由两个自由模态的索引 $F_{\mathcal{X}}$ 和 $F_{\mathcal{Y}}$ 组成，在 SpTC 的计算过程中只需匹配收缩模态的索引 $C_{\mathcal{X}}$ 和 $C_{\mathcal{Y}}$。
- 第二阶段被称为排序阶段，主要工作是根据收缩过程中所需的配对的张量块排序，使彼此相邻的张量块与相对应的输入张量块能被快速匹配，即 $(F_{\mathcal{X}}, C_{\mathcal{X}})$ 和 $(F_{\mathcal{Y}}, C_{\mathcal{Y}})$，其中 $C_{\mathcal{X}}$ 等于 $C_{\mathcal{Y}}$。
- 第三阶段被称为压缩阶段，主要工作是由任务调度程序将所有已配对的张量块组合 $(F_{\mathcal{X}}, F_{\mathcal{Y}})$ 负载到相应的张量核心上计算，以获得结果张量中非空张量块的近似数量和大小。

在创建任务列表之前，BCB-SpTC 对收缩模态的索引计数，记录与这些索引相对应的张量块的数量。任务列表包括一组已配对的张量块组合，用于表示张量块之间的配对关系，排序方法被应用于这些已配对的张量块组合中的自由模态的索引 $(F_{\mathcal{X}}, F_{\mathcal{Y}})$ 上。整个任务列表生成过程的目的是为了高效地生成一个任务列表，从而捕获张量块之间的匹配关系，以避免了不必要的张量块计算，并使得收缩模态的索引之间保持一致性。

5.3.2 张量核心上的数据填充策略

张量核心是 GPU 上专门用于执行矩形计算 $D = A \times B + C$ 的定制计算单元，其在处理高维计算型 SpTC 时的潜力值得进一步研究。BCB-SPTC 以张量块的形式组织和处理张量，这为利用张量核心来加速 SpTC 提供了机会。

因此，本节探索了将 SpTC 的计算任务负载到张量核心的方法，图5.3示范了张量核心如何加载张量块数据。此外，在张量核心完成计算后，会从任务列表中提取新任务以执行张量收缩操作，并重复此过程，直到遍历了任务列表中的所有任务，从而完成了所有的计算过程。

张量核心上的数据加载方案取决于输入张量的自由模态的索引和收缩模态的索引。张量 $\mathcal{X}$ 的自由模态的索引被组织成矩形 A 的行索引，而张量 $\mathcal{Y}$ 的自由模态的索引被组织成矩形 B 的列索引。位图索引数组则被用来指明张量核心中非零值的位置。

NVIDIA 提供的 WMMA 接口每次仅计算一个线程束中的 256 个元素。64 位的无符号长整数数据类型被用于存储位图索引中的二进制数组，这说明使用一个无符号长整数的值只能表示 64 个元素。如果一次只计算 64 个，则不能利用张量核心的矩阵乘法的全部性能。因此，WMMA 都要将元素从全局内存加载到每个线程的专用寄存器中，这些专用寄存器负责加载和存储部分矩阵。

图5.4展示了在收缩一个模态和收缩两个模态的 SpTC 中，使用张量核心上的

中国知网 https://www.cnki.net

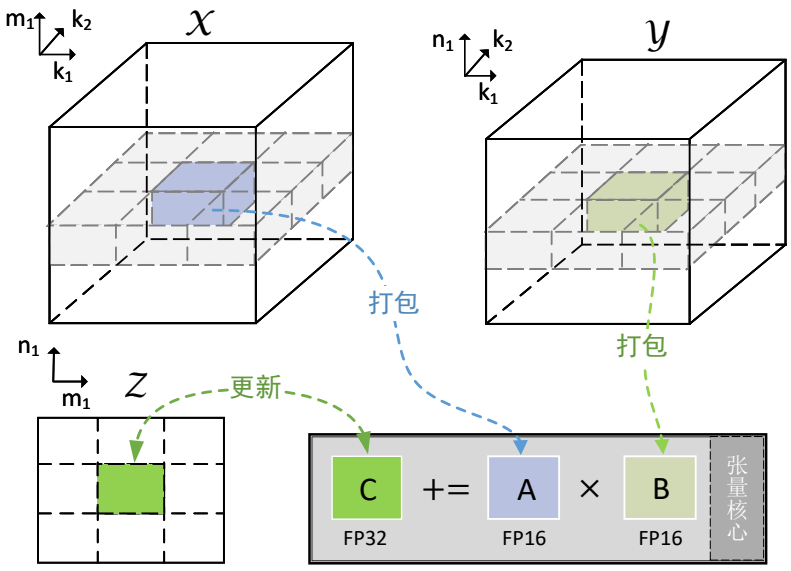
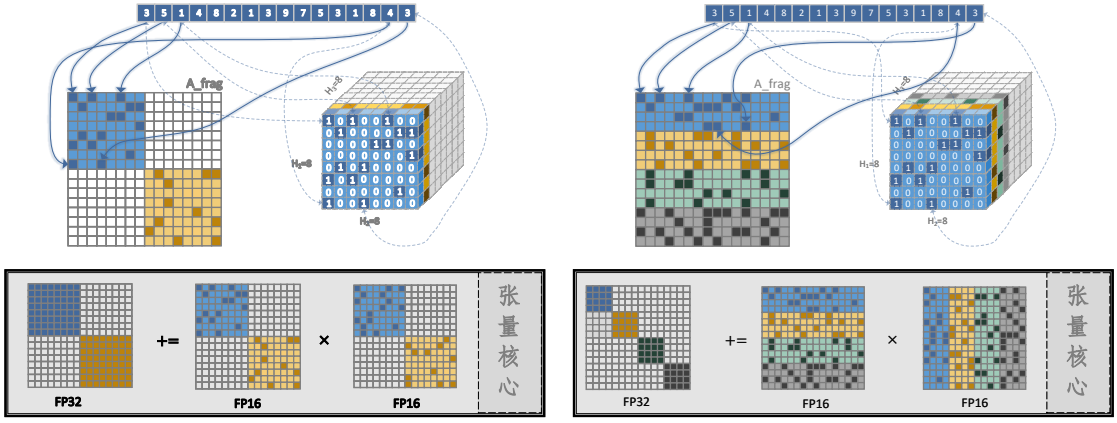


图 5.3　张量核心上数据加载方法的示意图



(a) 一个模态的张量收缩　　(b) 两个模态的张量收缩

图 5.4　张量核心上数据填充方法的示意图

专用寄存器 *Afrag* 或 *Bfrag* 加载大小为 $8 \times 8 \times 8$ 的张量块的加载方法。具体而言，图中主要描述了张量块的第一个切片以及切片中首尾非零值的加载位置，并以一个张量块单位 $\mathcal{B} \in \mathbb{R}^{8\times8\times8}$ 的稀疏张量的自身收缩过程作为示例。

在收缩一个模态 SpTC 的计算过程中，对于结果张量 $\mathcal{Z}$ 中的每个元素，可以得到：

$$\mathcal{Z}(h_1, h_2, h_3, h_1) = \mathcal{B}(h_1, h_2, :) \times \mathcal{B}(:, h_2, h_1) \tag{5.7}$$

其中 $I_m = \{H_1, H_2\}$ 和 $I_k = \{H_3\}$。扩展为切片形式表示为：

$$\mathcal{Z}(h_1, :, :, h_1) = \mathcal{B}(h_1, :, :) \times \mathcal{B}(:, :, h_1) \tag{5.8}$$

中国知网　https://www.cnki.net

如图5.4(a) 所示，在 H_2 上实现收缩一个模态的 SpTC 仅需要将每个切片放入片段的对角矩阵中, 并且两个切片的收缩可以通过执行单个 WMMA 操作完成。对于一个 $8 \times 8 \times 8$ 的张量块，需要四次 WMMA 操作, 其中结果矩阵中的两个对角矩阵被用于生成收缩一个模态的 SpTC 的结果。

在图5.4(b) 中，类似的切片和片段放置方法被采用。对于收缩两个模态的 SpTC，$I_m = \{H_1, H_2\}$ 和 $I_k = \{H_3\}$，计算过程可以表示为：

$$\mathcal{Z}(h_3, h_3) = \sum_{h_1=1}^{8} \sum_{h_2=1}^{8} \mathcal{B}(h_3, :, :) \times \mathcal{B}(:, :, h_3) \tag{5.9}$$

对于收缩两个模态的 SpTC，原始的 8×8 切片被调整为 4×16。一次 WMMA 操作足以对四个切片收缩，因此，对于一个 $8 \times 8 \times 8$ 的张量块，需要两次 WMMA 操作。此外，在使用张量核心计算收缩两个模态的 SpTC 后，结果矩阵中仅位于对角线上的四个小矩阵被累加。因此，一次 WMMA 操作可以获得结果中的四个值。

算法 5.2 收缩一个模态的 BCB-SpTC 中张量块相乘算法

输入:

输入数组: $\mathcal{X}_ele, \mathcal{Y}_ele$;

任务列表相关数组: $task_list_ptr, task_list$;

输出:

结果数组: $\mathcal{Z}_ele$;

1: int $WID \leftarrow threadIdx.x/32$;
2: wmma::fragment A_frag, B_frag, C_frag;
3: int $t_1 \leftarrow task_list_ptr[WID]$;
4: int $t_2 \leftarrow task_list_ptr[WID + 1]$;
5: $bmp_3 \leftarrow 0$;
6: **for** $i = t_1$ **to** t_2 **do**
7: $(bmp_1, bmp_2) \leftarrow task_list(i)$;
8: $slice_num \leftarrow 2$;
9: **for** $s = 0$ **to** 8 **do**
10: Load2Frag($s, slice_num, bmp_1, \mathcal{X}_ele, A_frag$);
11: Load2Frag($s, slice_num, bmp_2, \mathcal{Y}_ele, B_frag$);
12: wmma::mma_sync($C_frag, A_frag, B_frag, C_frag$);
13: Load2Bmp(C_frag, bmp_3);
14: $s+ = slice_num$;
15: **end for**
16: **end for**
17: Load2Tensor($bmp_3, \mathcal{Z}_ele$);

中国知网 https://www.cnki.net

18: **return** $\mathcal{Z}_ele$;

算法 5.2 展示了收缩一个模态的 SpTC 中使用 WMMA 操作张量块时 GPU 线程的执行过程。算法的第 1 行初始化的变量 *WID* 表示每个线程所属的线程束。第 2 行初始化张量核心上的专用寄存器，第 3 和 4 行得到该线程束需要执行的计算任务。第 5 行初始化一个临时变量 bmp_3，用于保存后续计算中的中间结果。第 6 到 16 行介绍了每个线程束通过迭代遍历该线程束中所有的计算任务，逐一在张量核心上计算得到每对张量块对应的结果。第 7 行获得了匹配的张量块对 (bmp_1, bmp_2)。在计算过程中，每个线程束根据第 10 和 11 行中的索引信息从 $slice_num$ 个切片中读取非零值，并将它们存储在寄存器 A_frag 和 B_frag 中。随后，通过在第 12 行执行矩阵矩阵乘法，产生了 C_frag。C_frag 中的两个对角子矩阵在第 13 行被存储。在处理分配给该线程束的所有匹配对之后，结果张量块被传输到结果张量 $\mathcal{Z}$ 中。

收缩两个模态的 SpTC 计算过程与收缩一个模态的 SpTC 相似，主要区别在于 $Load2Frag$ 和 $Load2Bmp$ 函数。在收缩两个模态的 SpTC 中，A_frag 和 B_frag 同时加载了 4 个切片的数据到张量核心上，即算法5.2的第 10 行至第 11 行，其中变量 $slice_{num}$ 被赋值为 4。

通过提取结果矩阵中的四个对角子矩阵，并对这些子矩阵内的所有元素求和得到四个结果值。每个求和得到的结果值等价于收缩一个模态的 SpTC 算法5.2第13行中所得张量内的单个值。因此，对于四维张量等更高维的张量收缩，只需修改数据填充方法即可推广该过程。

5.4 实验评估

本节在 GPU 上的执行了一系列的实验，旨在评估 SCB-SpTC 的性能。首先，分析 BCB 格式的存储空间大小，并评估了其在提高 SpTC 计算性能方面的有效性。其次，通过对 SpTC 计算过程中各阶段的时间分析，验证了提出的张量核心加载方法在不同的缩并模态下高效利用张量核心的可行性。最后，通过与基线方法的比较，展示了 BCB-SpTC 在总体性能方面的有效性与先进性。

5.4.1 实验设置

1、 平台

用于实验的 GPU 为 NVIDIA Ampere 架构的 A100 GPU，它安装在一台搭载 Intel Xeon Gold 5120 CPU 的平台上。实验所使用的 CPU 内存容量为 128GB，而 GPU 的全局内存容量为 40GB，并具备 6912 个 CUDA 核心以及 432 个张量核心。实验中所采用的编译器版本为 GCC 7.5.0 和 NVCC 11.2。

中国知网 https://www.cnki.net

2、 数据集

实验所使用的四个稀疏张量源自具有不同特征的现实场景数据集，分别来自 GroupLens Research[①]、成都出租车数据[②] 和与尿嘧啶相关的化学数据[146]。

- **Ratings：** 由 GroupLens Research 发布。它描述了来自电影推荐服务 MovieLens 会员的匿名电影评分。维度大小为 $6040 \times 3952 \times 5$，数据集中共 1000209 条数据。
- **Art：** 由 GroupLens Research 发布。它描述了来自电影推荐服务 MovieLens 会员的匿名艺术品评分。维度大小为 $669 \times 379 \times 251$，数据集中共 63064 条数据。
- **Taxi：** 这是 2014 年 8 月收集的来自成都市区的交通流量数据。它获得了超过 1.4 亿辆 14000 多辆出租车的 GPS 轨迹数据。Taxi 的维度大小为 $7471 \times 1080 \times 28$，数据集中共 688758 条数据。
- **Uracil：** 来自气相和水溶液中尿嘧啶的化学属性组成的数据集。Uracil 的维度大小为 $90 \times 90 \times 174 \times 174$，数据集中共 1034755 条数据。

3、 参数设置和基线方法

本实验采用了稀疏张量自身收缩的方法来结果分析。实验考虑了三维张量的情况，包括收缩单一模态的计算任务以及收缩两个模态的计算任务。此外，对于四维张量，还额外考虑了收缩三个模态的计算任务。在考虑设备内存限制的情况下，选择了相对较长的模态作为收缩模态的索引。

为了充分展示 BCB-SpTC 性能的优势，本实验选择了当前 GPU 上最新的研究成果，即 DBCSR[147] 和 GSpTC[43] 作为基准方法对比。此外，还选取了最初用于稀疏矩阵乘法的 tSparse[38] 作为额外的对比方法。

具体而言，在对比实验中，tSparse 首先将张量转化为矩阵，再执行与张量收缩等价的矩阵乘法。BCB-SpTC 和 tSparse 都使用了基于位图的非零值存储方法，同时也利用了带有张量核心的分块技术。然而，在分块方式和索引方法方面，它们存在显著差异。BCB-SpTC 采用了多维分块策略，采用了二进制线性索引方案来记录张量块在张量中的位置。而 tSparse 的分块方式是二维的，采用 COO 格式来记录张量块的索引。

5.4.2 BCB 格式的内存使用量分析

表 5.1 汇总了 Ratings、Art、Taxi 和 Uracil 这四个数据集在不同方法下的内存使用情况，以兆字节（Megabyte，MB）作为数值单位，BCB-SpTC 方法在内存利用方面表现出显著的优势。对于所有数据集，相对于 tSparse、GSpTC 和 DBCSR

[①] https://grouplens.org

[②] https://challenge.datacastle.cn

中国知网 https://www.cnki.net

方法，BCB-SpTC 的内存消耗分别减少了 1.75 MB、33.80 MB 和 21.76 MB。这些结果表明了 BCB-SpTC 方法具有出色的存储效率。

表 5.1　数据集的内存使用量（MB）的统计表

张量数据集	BCB-SpTC	tSparse	GSpTC	DBCSR
Ratings	5.14	9.13	11.45	7.89
Art	0.12	0.21	0.74	0.83
Taxi	2.90	5.16	7.88	5.59
Uracil	0.82	1.46	142.11	81.72

在 Uracil 数据集上，BCB-SpTC 显著的节省了内存使用量，即相较于 GSpTC 和 DBCSR 分别减少了 141.29 MB 和 80.90 MB 的存储开销。这一结果突显了 BCB-SpTC 方法在处理大规模数据集方面的显著优势，也验证了分块、位图存储和位压缩等技术的有效性。

具体而言，分块技术的运用降低了内存开销，通过将大型稀疏张量分解为较小的张量块，减少了计算之间的依赖关系，同时也利用了数据的局部性来提高 SpTC 的计算速度。张量块的位图存储技术高效地记录了非零值的分布信息，实现了内存的紧凑使用。此外，位压缩采用了二进制线性索引，大幅减少了对张量块索引数据的访问次数，进一步优化了内存使用。

总而言之，实验结果表明，在存储效率方面，BCB-SpTC 明显领先于其他方法，使其成为在 GPU 上 SpTC 的极具潜力的方法，特别是对于处理大规模数据集的情形。

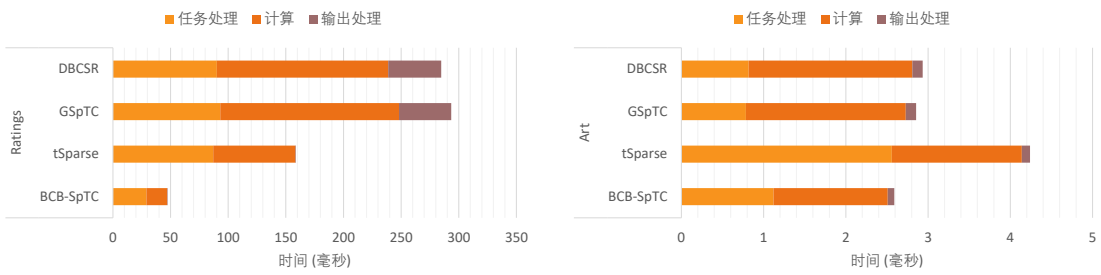
5.4.3　BCB 格式的效率

1、　各阶段的时间分析

尽管不同的方法在实现 SpTC 处理步骤上各有不同，但执行时间可统一分为三个阶段，即任务处理阶段、计算阶段和输出处理阶段。在 BCB-SPTC 中，任务处理阶段包括任务列表的生成，计算阶段包括张量块之间的收缩计算，输出处理阶段包括对计算任务结果的累加和写回。

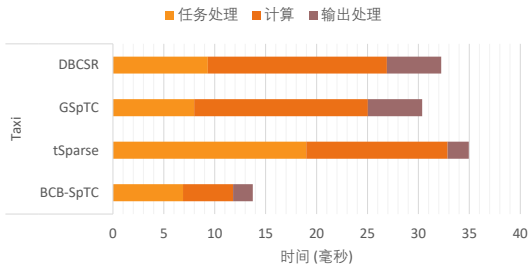
图5.5 表示了各种方法在四个数据集上执行收缩两个模态的计算任务时，三个阶段的时间。在 Ratings、Taxi 和 Uracil 数据集上，BCB-SPTC 的整体执行时间显著低于其他方法。具体而言，在 Ratings 上，相对于其他方法，BCB-SPTC 平均加速比达到了 5.18×；在 Uracil 上，BCB-SPTC 明显快于 DBCSR 和 GSpTC，并且 BCB-SpTC 的执行时间仅为 tSparse 的一半。此外，由于 Art 数据集的稀疏度较高，计算过程中的数据局部性相对较差，因此 BCB-SPTC 的优势不太明显，但仍然略有优势。

中国知网　https://www.cnki.net

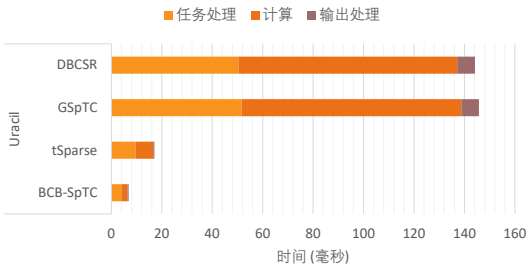


(a) Ratings 数据集

(b) Art 数据集



(c) Taxi 数据集



(d) Uracil 数据集

图 5.5　收缩两个模态 SpTC 的总执行时间结果图

通过比较计算过程中各阶段的执行时间可以看出，BCB-SPTC 在所有数据集上的计算阶段时间开销均低于其他方法。

对于任务处理阶段的时间开销，在 Ratings、Taxi 和 Uracil 数据集上，BCB-SPTC 明显低于其他方法。而在 Art 数据集上，BCB-SPTC 的任务处理阶段相对较长，仅略短于 tSparse，原因是 Art 数据集的维度较小且非零值数量有限，降低了分块等优化技术的必要性。

对于输出处理阶段的时间，BCB-SPTC 在所有数据集上与 tSparse 相近，且优于 DBCSR 和 GSpTC，这是因为分块技术带来的数据局部性极大减少了输出阶段的累加操作。

综上所述，本实验的结果表明，BCB-SPTC 在整体执行过程中明显优于其他方法，特别是在处理大规模和相对稠密的数据集时，BCB-SPTC 在每个阶段的时间开销都明显低于其他方法。

2、　张量核心上数据填充策略的有效性分析

为了验证数据填充策略在张量核心上的有效性，比较了 BCB-SpTC 和 tSparse 的计算时间，实验结果见图5.6。

对于所有收缩一个模态的 SpTC，可以观察到 BCB-SpTC 和 tSparse 在计算时间几乎没有差别，原因在于当收缩一个模态时，BCB-SpTC 与 tSparse 对张量核心有相同的数据填充方式，即它们都是使用张量块中两个切片的数据填充张量核

中国知网　https://www.cnki.net

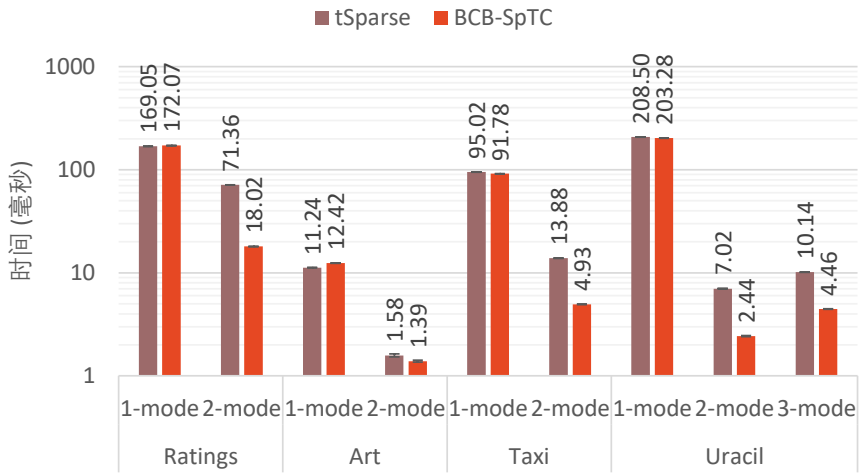


图 5.6　与 tSparse 对比计算阶段时间开销的结果图

心。

对于收缩两个模态和收缩三个模态的 SpTC，可以看到 BCB-SpTC 相对于 tSparse 有较大的性能提升。原因在于 tSparse 仍然使用张量块中两个切片的数据来填充张量核心，而 BCB-SpTC 进一步根据收缩多个模态的计算特点，使用张量块中四个切片的数据来填充张量核心。

从实验结果来看，收缩两个模态时，BCB-SpTC 相对于 tSparse 的可以获得的平均加速比为 2.70×。在执行收缩三个模态的计算任务时，获得的平均加速比为 2.27×。收缩三个模态的 SpTC 性能提升收益略低于收缩两个模态的原因是收缩三个模态的计算过程中加载数据的开销更大，这稍微降低了张量核心的利用率。

3、　误差分析

由于在张量核心上执行的计算采用了 FP16 作为输入数据类型和 FP32 作为输出数据类型，这可能导致数值误差的产生。因此，为了验证 BCB-SpTC 在张量核心上的有效性，将 BCB-SpTC 和 tSparse 的计算结果与使用 32 位浮点数类型作为输入的计算结果比较，它们之间的差异表示为误差 R，计算方式如下：

$$R = \sqrt{\frac{1}{|\Omega|}\sum_{i\in\Omega}(\mathcal{Z}_{FP32}(i) - \mathcal{Z}_{FP16}(i))^2}, \quad |\Omega| = \prod_{j=1}^{d_{\mathcal{Z}}} S_j^{\mathcal{Z}} \tag{5.10}$$

其中 $\mathcal{Z}_{FP32}$ 代表使用 32 位浮点数精度计算得到的结果，$\mathcal{Z}_{FP16}$ 代表使用 16 位浮点数精度计算得到的结果。

如图5.7所示，展示了 BCB-SpTC 和 tSparse 计算的数值误差。BCB-SpTC 和 tSparse 在收缩一个模态的计算过程中在张量核心上采用的数据填充方法相同，导致它们的误差相近。

中国知网　https://www.cnki.net

此外，收缩两个模态的 SpTC 误差大于收缩一个模态的 SpTC，但小于收缩三个模态的 SpTC，其原因在于随着收缩模态数量的增加，使用张量核心计算引入的误差也随之增加。整体而言，tSparse 和 BCB-SpTC 的数值误差都在 1.52% 以下。

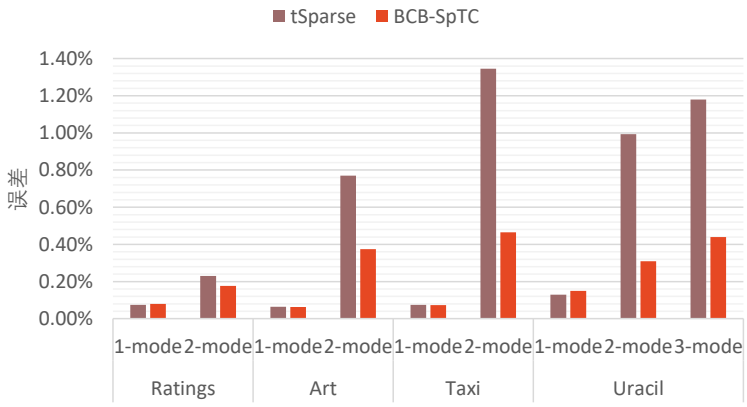


图 5.7　张量核心上数值误差的结果图

5.4.4　与其他实现的比较

BCB-SpTC 和其他三种基线方法执行多种模态收缩的 SpTC 的比较结果如图5.8所示，该图中还包括在 A100 GPU 上每种方法执行收缩一个模态、收缩两个模态和收缩三个模态的 SpTC 时的性能表现。

首先对比 BCB-SpTC 和 tSparse 的计算速度，在执行收缩一个模态的 SpTC 时，BCB-SpTC 始终优于 tSparse，获得的加速比范围介于 1.41× 到 1.69× 之间。而在执行收缩两个模态的 SpTC 时，BCB-SpTC 的加速比范围更广，为 1.62× 到 3.28×，并且在 Uracil 数据集上执行收缩三个模态的 SpTC 实验中也获得了性能提升。这进一步突显了 BCB-SpTC 在处理高维数据时相对于 tSparse 的优势。这些发现验证了 BCB-SpTC 在 SpTC 运算中相对于其他方法的高效性，其性能差异可以归因于 BCB-SpTC 的高效利用张量核心和优化的索引策略。

然后，对 BCB-SpTC 与 GSpTC、DBCSR 两种方法的时间开销进行了对比。在 GSpTC 中，采用了基于 COO 格式的并行优化策略，而 DBCSR 则使用了基于 CSR 格式的并行优化策略。从图 5.8 所呈现的实验结果中，明显可以观察到 BCB-SpTC 的计算时间开销明显更低。其主要原因在于 BCB-SpTC 采用了分块技术、位图存储和位压缩技术，这些优势在内存利用方面表现出色，并且还能够有效地受益于张量核心的加速。

综上所述，BCB-SpTC 在计算速度和效率方面明显优于 tSparse、GSpTC 和 DBCSR，平均获得的加速比为 4.63×，获得的加速比大小范围从 1.10× 到 21.27×。

中国知网　https://www.cnki.net

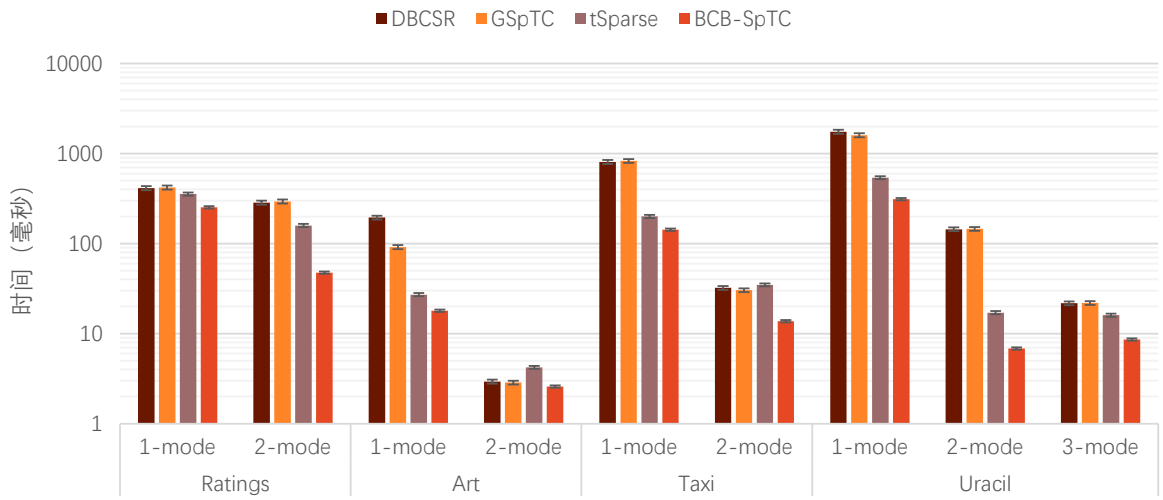


图 5.8　比较不同方法实现 SpTC 性能的结果图

5.5　本章小结

本章以 SpTC 为例，基于 HASTensor 中的优化技术提出了一种基于位坐标位图格式的 SpTC 方法，即 BCB-SpTC，展示了 HASTensor 库在高维计算型稀疏张量运算性能优化方面的有效支持。BCB-SpTC 采用了分块技术，将张量分割成多个张量块，并利用位图存储技术来记录张量块内非零元的稀疏分布，同时使用位压缩技术来最小化张量块索引中的冗余信息。它还将多维索引映射为二进制线性索引，以进一步加速多维度索引匹配过程。通过设计一种新的数据填充策略，BCB-SpTC 能够利用张量核心加速计算。实验结果表明，在真实场景的张量数据集上，相对于当前现有的方法，BCB-SpTC 在张量核心的加速下能够实现显著的性能提升。

中国知网　https://www.cnki.net

第 6 章　链乘计算型稀疏张量运算的并行算法设计和性能优化

本章以 SpTMCM 为例，主要介绍 HASTensor 中针对链乘计算型稀疏张量运算的并行算法设计和性能优化方法。多维数据的分析，尤其是在提取潜在信息方面采用的张量分解，受到了广泛关注。尽管多维稀疏数据通常在多核处理器上进行处理，但要开发出高度优化的链乘计算型张量运算仍然是一项具有挑战性的任务。

为了克服这些挑战，本章提出了一种全新的方法，命名为 HLSTO，并深入研究了 SpTMCM 与 GPU 上新兴的张量核心单元相结合的发展趋势。首先，该方法相较于以前的工作，为 SpTMCM 提供了统一的存储格式和优化方案。然后，设计了一种基于张量块的混合张量格式，用以根据特定阈值对张量块进行分类，以解决稀疏张量因不规则的非零值分布而导致的内存访问低效问题。随后，基于张量核心的特性开发了一种新的 SpTMCM 并行算法，以提高内存带宽利用效率。最后，在大量真实场景的稀疏张量数据集上进行了广泛实验验证，结果显示相较于现有方法，HLSTO 能够实现显著的性能加速。

6.1　引言

张量通常被定义为多维数组，用于表示多维数据。张量分解在多个领域有着广泛的应用，包括数据挖掘[96,148]、推荐系统[149]和异常检测[150]等领域。SpTMCM 是 CP 分解[67]和 Tucker 分解[30]等张量算法和应用的基本内核。在实际应用中，大多数张量都是稀疏的，这意味着它们的大部分值都是零。因此，为了优化在当代架构上的 SpTMCM，设计和开发并行算法变得至关重要。

SpTMCM 在各种硬件平台上的优化问题一直是一个重要的研究课题[151]，也对 SpTMCM 进行了大量的研究。SpTMCM 在共享内存和分布式内存系统[99,133]上受到繁琐的并行执行模式的限制，因此，在 GPU 上优化 SpTMCM 已经成为一项亟需的研究任务[59]。

此外，GPU 正在经历快速发展，最新 GPU 上提供的张量核心单元为优化 SpTMCM 提供了新的机会。张量核心提供高性能和低功耗，但其唯一功能是矩形乘法[152,153]。GPU 本质上具有从特定领域计算到通用计算的可扩展性，这使得将特定于矩形计算的张量核心扩展到 SpTMCM 的研究变得引人入胜且具有重要意义。

中国知网　https://www.cnki.net

以往的 SpTMCM 优化研究都是独立的，如 SpMTTKRP 和 SpTTMChain，通常需要一系列繁琐的预设来优化单个 SpTMCM 的性能，这些繁杂的优化过程往往会给使用者带来很多麻烦[154]。然而，这些不同的 SpTMCM 在数据分布和优化类型上有相似之处，有研究者多次尝试探索各种 SpTMCM 统一计算模式的新方法[24,57,59,110,155,156]。

这些研究的重点主要集中在非零值粒度下乘法运算的通用性上，这不利于基于张量核心的以矩形计算为重点的并行算法的设计。鉴于此，本章通过研究 SpTMCM 的计算模型，提出了一种新的基于矩形计算的 SpMTTKRP 和 SpTTMChain 表示方法。

由于稀疏张量的不规则非零分布，这导致频繁的内存访问[57,157]。具有高带宽内存（High Bandwidth Memory，HBM）的 GPU 成为一个可行的选择。尽管 GPU 的高带宽内存可以提高 SpTMCM 的内存访问带宽，但仍然存在内存容量有限、内存访问延迟高等限制。设计一种适合 GPU 的稀疏张量格式是解决上述问题的可行途径。

在以往的研究中，压缩张量格式可以减少内存占用和组依赖性，但受到模态特异性[28,158] 的限制。分块技术将张量划分成张量块，并利用数据的局部性来减少计算过程中的内存访问开销，但对于不规则非零分布的张量用处不大[49,58,159]。

因此，本章构造了一种基于张量块的混合张量格式，其中具有良好数据局部性的非零值被聚合成一个张量块并保存，而离散分布非零值仍以单个的形式存储。该方法有效地利用了张量数据的局部性，又减少了离散非零值聚合成张量块造成的额外负担。

张量核心是 GPU 上最新的计算组件，具有成为通用计算关键组件的潜力[34,160,161]。其设计旨在容纳小型稠密矩阵，这与传统的张量存储格式不相符。为适应这一硬件特性，本章引入了分块技术，将张量分解为小型矩形张量块，以与张量核心的设计相契合。然后进一步深入研究了在张量核心上加速 SpTMCM 问题，利用了张量核心的快速矩形计算能力，以提升张量块的计算效率，并较好地发挥了张量核心上的乘法累积（Multiply-Accumulate, MAC）操作的优势。

总而言之，为了解决上述问题，本章的主要贡献如下：

- 本章提出了一种新颖的方法，将 SpTMCM 的乘法模式从非零值粒度粗化到切片粒度，从而方便了基于张量核心的并行算法的设计。
- 本章构建了一种基于张量块的混合张量格式，为基于张量核心的并行算法提供了更高的存储效率。
- 本章开发了一种基于张量核心的 SpTMCM 并行算法，通过高效的内存访问模式和利用张量核心的 MAC 操作，使新一代 GPU 的性能优势最大化。
- 本章在两个最新的 GPU 上测试了本章提出的方法的性能，实验结果表明本章的方法比基线方法具有显著的性能优势。

中国知网 https://www.cnki.net

6.2 SpTMCM 的数学符号表示

张量核心是最新的 NVIDIA GPU 中包含的一种新型功能单元，相对于标准的通用 CUDA 内核，它可以显着加速矩形乘法运算。然而，由于张量的高维度，直接利用张量核心来加速 SpTMCM 运算具有相当的困难。考虑到 SpTMCM 计算的复杂性，探索 SpTMCM 中 SpTTMChain 和 SpMTTKRP 统一优化的方法是当前一个重要的研究课题[24,59,110,155,156]。本节将详细介绍一种专为利用张量核心而设计的新型 SpTMCM 统一优化方法。

6.2.1 常见的符号表示

SpTTMChain 和 SpMTTKRP 是张量分解中的关键操作，这些操作可以用一些数学符号来描述。数学符号表示的推导促使着 SpTMCM[59,110] 的统一优化方法的提出。以三阶张量为例，SpTTMChain 和 SpMTTKRP 可以统一表示为逐值计算形式：

$$Y_{(1)}(i,:) \mathrel{+}= \mathcal{X}(i,j,k)\,(B(j,:) \otimes C(k,:)) \tag{6.1}$$

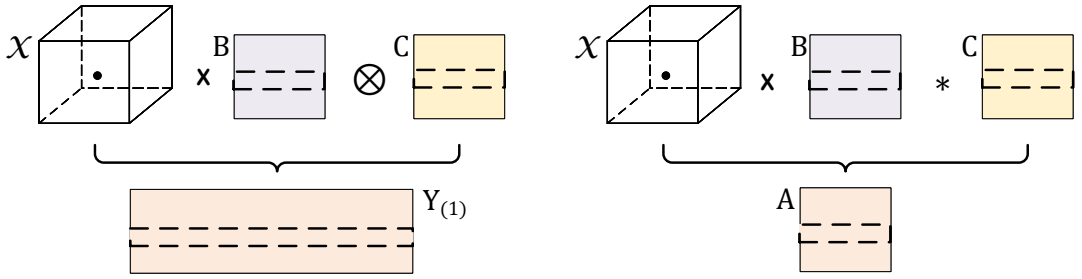
$$A(i,:) \mathrel{+}= \mathcal{X}(i,j,k)\,(B(j,:) * C(k,:)) \tag{6.2}$$

公式（6.1）描述了 SpTTMChain 的逐值计算形式，其中 $\mathcal{X}(i,j,k)$ 是张量 $\mathcal{X}$ 的非零，$B(j,:)$ 和 $C(k,:)$ 是因子矩阵的行向量，$Y_{(1)}(i,:)$ 是模一矩阵化的张量 $\mathcal{Y}$ 的行向量。

与公式（6.1）类似，公式（6.2）描述了 SpMTTKRP 的逐值计算形式。与公式（6.1）的不同之处在于公式（6.2）的结果是因子矩阵 A 的行向量。从计算过程来看，式（6.2）为因子矩阵 B 与 C 的行向量之间的 Hadamard 积，式（6.1）为 Kronecker 积。

如图6.1所示，公式（6.1）和（6.2）的逐值计算形式是以张量的非零值作为遍历目标，这不利于设计高效的 SpTMCM。因此，本章设计了一种新的基于矩形计算的计算方法，以便更有效地利用张量核心，充分发挥张量核心在矩形计算中的性能优势。

中国知网 https://www.cnki.net



$Y_{(1)}(i,:)+=\mathcal{X}(i,j,k)(B(j,:)\otimes C(k,:))$

(a) SpTTMChain

$A(i,:)+=\mathcal{X}(i,j,k)(B(j,:)*C(k,:))$

(b) SpMTTKRP

图 6.1　常用计算方法的示意图

6.2.2　基于矩形计算的符号表示

将 SpTMCM 统一优化方法的用符号表示，首先给出 SpTTMChain 结果中每个值的计算公式，该公式表示为：

$$\mathcal{Y}(i,r_1,r_2)=\sum_{j=1}^{J}\left(\sum_{k=1}^{K}\mathcal{X}(i,j,k)C(k,r_2)\right)B(j,r_1) \tag{6.3}$$

其中 $B(j,r_1)$ 和 $C(k,r_2)$ 分别是相应因子矩阵的标量值，$\mathcal{Y}(i,r_1,r_2)$ 和 $\mathcal{X}(i,j,k)$ 分别是相应张量的非零值。

其次，以向量内积的作为计算形式，SpTTMChain 的计算过程可以表示为：

$$\begin{aligned}\mathcal{Y}(i,r_1,r_2)&=\sum_{j=1}^{J}\left(\mathcal{X}(i,j,:)C(:,r_2)\right)B(j,r_1)\\&=B(:,r_1)^{\mathsf{T}}\left(\mathcal{X}(i,:,:)C(:,r_2)\right)\end{aligned} \tag{6.4}$$

其中 $B(:,r_1)\in\mathbb{R}^{J\times 1}$ and $C(:,r_2)\in\mathbb{R}^{K\times 1}$ 是因子矩阵的列，$\mathcal{X}(i,j,:)\in\mathbb{R}^{1\times K}$ 是是张量 $\mathcal{X}$ 的一个纤维，$\mathcal{X}(i,:,:)\in\mathbb{R}^{J\times K}$ 是张量 $\mathcal{X}$ 的一个切片。

然后，继续调整计算形式，得到 SpTTMChain 的逐切片计算形式，其表示为：

$$\mathcal{Y}(i,:,:)=B^{\mathsf{T}}\left(\mathcal{X}(i,:,:)C\right) \tag{6.5}$$

其中 $B\in\mathbb{R}^{J\times R_1}$ 和 $C\in\mathbb{R}^{K\times R_2}$ 是因子矩阵，$\mathcal{Y}(i::)\in\mathbb{R}^{R_1\times R_2}$ 是张量 $\mathcal{Y}$ 的一个切片。

与 SpTTMChain 的符号推导过程相同，SpMTTKRP 结果中各值的公式可推导

中国知网　https://www.cnki.net

为：

$$A(i,r)=\sum_{j=1}^{J}\left(\sum_{k=1}^{K}\mathcal{X}(i,j,k)C(k,r)\right)B(j,r) \tag{6.6}$$

其中 $A(i,r)$、$B(j,r)$、$C(k,r)$ 为因子矩阵的值。调整计算形式后可表示为：

$$\begin{aligned}A(i,r)&=\sum_{j=1}^{J}\left(\mathcal{X}(i,j,:)C(:,r)\right)B(j,r)\\&=B(:,r)^{\mathsf{T}}\left(\mathcal{X}(i,:,:)C(:,r)\right)\end{aligned} \tag{6.7}$$

其中 $B(:,r)$ 与 $C(:,r)$ 是对应因子矩阵的第 r 列，这正是与公式（6.4）的区别。此外，由于 SpMTTKRP 的结果是一个列数为 R 的因子矩阵 A，与因子矩阵 B 和 C 的列数相同。SpMTTKRP 的逐切片计算形式表示为：

$$A(i,:)=diag(B^{\mathsf{T}}(\mathcal{X}(i,:,:)C)) \tag{6.8}$$

其中 $diag()$ 函数用于提取矩阵的对角线元素，它返回一个向量 $A(i,:)\in\mathbb{R}^{R\times 1}$。通过比较公式（6.5）和（6.8），可以观察到，提取公式（6.5）的结果矩阵的对角线元素会产生公式（6.8）的结果向量，这正是统一优化方法的核心思路。

总之，公式（6.5）和（6.8）给出了一种统一优化的新方法，如图 6.2 所示。这种方法的主要计算部分涉及矩形乘法，这为后续构建基于张量核心的并行算法提供了便利。

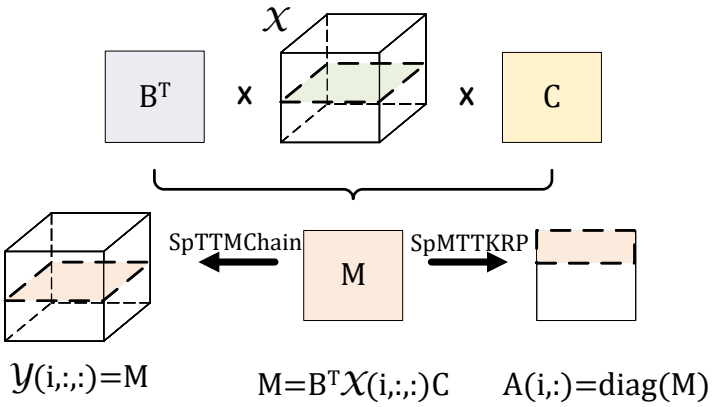


图 6.2　基于矩形计算方法的示意图

6.3　混合线性存储格式

由于不存储零值，与稠密存储格式相比，稀疏存储格式大大节省了存储空间。使用单一的稀疏存储格式来存储稀疏张量是一种常见的做法。然而，由于稀疏格

中国知网　https://www.cnki.net

式中非零值素的存储位置是不连续的，这种情况会导致较高的内存访问延迟和性能降低。为了解决上述问题，本章提出了混合线性存储（Hybrid Linear Storage，HLS）格式，它由混合线性存储稠密（Hybrid Linear Storage-dense，HLS-D）格式和混合线性存储稀疏（Hybrid Linear Storage-sparse，HLS-S）格式组成。

6.3.1 格式转换

为了解决上述问题，将稀疏张量被分为稠密部分和稀疏部分，这将提高前者的数据局部性，并方便使用 TCU 来加速 SpTMCM。此外，HLS 中的整数索引被二进制线性索引取代，以缓解高内存访问延迟的问题并加速数据索引。

稀疏张量通常以 COO 格式存储。稀疏张量 $\mathcal{X} \in \mathbf{R}^{6\times4\times4}$ 从 COO 到 HLS 格式的转换过程如图6.3所示。整个过程可分为四个阶段。

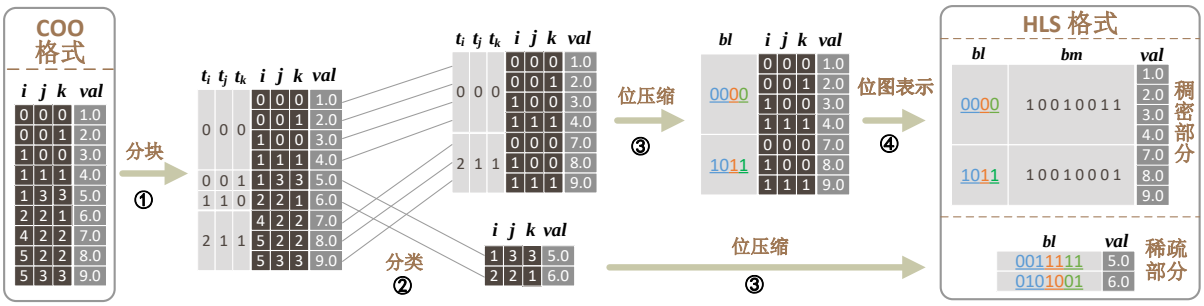


图 6.3　张量从 COO 到 HLS 格式转换过程的示例图

分块：稀疏张量 $\mathcal{X}$ 被分成许多大小为 $S_I \times S_J \times S_K$ 的张量块，图6.3中的大小为 $2 \times 2 \times 2$，并且 t_i、t_j、t_k 表示张量块的索引。由于稀疏张量的不规则非零分布，分块技术会产生非空张量块和空张量块。由于空的张量块与 SpTMCM 的结果无关，因此 HLS 仅记录非空张量块。

分类：经过分块技术划分后，由于稀疏张量的不规则非零分布，一些非空张量块是偏稠密的，而另一些则是偏稀疏的。视非零值个数大于或等于 T 的张量块为稠密块，则这些张量块根据阈值 T 被分类为稠密和稀疏的张量块。如果稀疏的张量块以稠密的方式存储，会造成大量的存储空间浪费。此外，稀疏张量块在张量核心上的性能甚至比在 CUDA 核心上的性能还要差。对于稀疏张量块，解除块结构，只记录稀疏张量块中的非零是必要的。因此，HLS 格式中以张量块形式存储的记为稠密部分，以离散非零值存储的记为稀疏部分。

位压缩：对于稠密部分，将张量块的多维整数类型索引映射为二进制线性索引，即将图6.3中的 t_i、t_j 和 t_k 数组转换为二进制数组 bl，从而降低内存访问的复杂性。在张量块内的索引中，只记录非零值在该张量块中的位置，从而减小了块内索引的维度大小。对于稀疏部分，与稠密部分类似，也是将非零值的多维整数类型索引映射为二进制线性索引。

位图表示：虽然使用二进制线性索引可以减少存储使用量，但可以进一步优

中国知网 https://www.cnki.net

化张量块内值索引的存储方式。位图是一种用于表示每个张量块内值的稀疏分布的存储方式，它连续储存非零值，并通过一组二进制值来跟踪这些非零值对应的位置。如果某个位置上的值非零，则位图索引相应的比特位记为 1，否则为 0。如图6.3所示，i、j、k 被转换为位图形式。位图表示方式可以进一步降低内存使用量，并有助于利用张量核心。

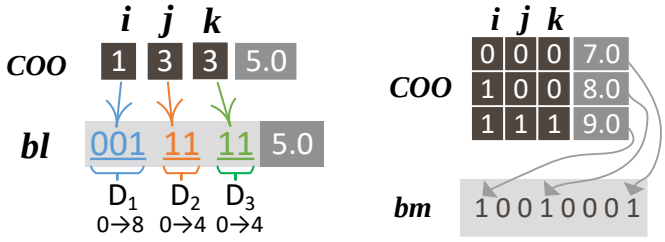
6.3.2 位优化技术

如图6.3所示，**分类**后，张量块和非零值分别存储在 HLS-D 和 HLS-S 两部分。HLS-S 以单独的形式存储每个非零值。它们的值保存在数组 *nnz_vals* 中，总个数用变量 *nnz* 记录。非零值的整数索引映射到二进制线性索引的过程中，每个模态使用的位数存储在数组 *nnz_linear_bits* 中。

对于 HLS-S，每种模态下的非零索引被压缩成单个值，即图6.3中的 bl，该值存储在 *nnz_indeds* 数组中。非零值的位压缩过程如图6.4（a）所示。在本例中，稀疏张量的模态大小为 $6, 4, 4$，所以 *nnz_linear_bits* 设置为 $\{3, 2, 2\}$。此非零的整数索引 $1, 3, 3$ 转换为二进制线性索引为 $\{001, 11, 11\}$，即索引值 bl 为 0011111。HLS-D 中的位压缩过程与 HLS-S 类似，只是压缩对象是张量块的索引，而不是非零值的索引。

对于稠密的部分，张量块每个维度的大小都保存在数组 *b_shapes* 中，并且张量块的数量保存在变量 *nnb* 中。张量块的二进制线性索引被保存在数组 *b_indeds* 中，每个张量块相对于第一个张量块的偏移量保存在 *b_vals_ptr* 中。每个张量块的第一个值的起始位置存储在数组 *b_vals* 中，张量块阈值存储在变量 *b_threshold* 中，而且数组 *b_linear_bits* 的表示与 *nnz_linear_bits* 类似。

对于张量块的位图表示，HLS-D 格式中，位图索引 bm 存储在数组 *b_bmp* 中，其中 *b_bmp* 的每个元素对应于每个张量块的位图索引。如图6.4（b）所示，图中展示了一个包含 3 个非零值的 $2 \times 2 \times 2$ 张量块。通过索引，可以计算出每个非零值在张量块中的相对位置。因此，示例中位图索引 bm 的值记为 $\{10010001\}$。



(a) 位压缩　　(b) 位图表示

图 6.4　两位优化技术的示例图

中国知网　https://www.cnki.net

6.3.3 空间复杂度分析

考虑一个具有 N 个非零值的稀疏张量 $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$，其稀疏部分的非零值数量为 N_s。因此，存储稀疏部分的二进制线性索引所需的空间大小 M_S^i 为：

$$M_S^i = (\lceil \log_2 I \rceil + \lceil \log_2 J \rceil + \lceil \log_2 K \rceil) \times N_s \tag{6.9}$$

在张量分解应用场景中，16 位半精度浮点类型（half）足以满足精度标准，因此假设非零值的数据类型为 half，则稀疏部分的非零值所需的空间大小 M_S^v 为：

$$M_S^v = 16 \times N_s \tag{6.10}$$

假设 $\mathcal{X}$ 有 T 张量块 $(S_I \times S_J \times S_K)$，张量块的二进制线性索引占用的空间大小 M_D^i 为：

$$M_D^i = \left(\left\lceil \log_2 \frac{I}{S_I} \right\rceil + \left\lceil \log_2 \frac{J}{S_J} \right\rceil + \left\lceil \log_2 \frac{K}{S_K} \right\rceil \right) \times T \tag{6.11}$$

则所有张量块存储索引所需的空间大小 M_{bm} 为：

$$M_{bm} = (S_I \times S_J \times S_K) \times T \tag{6.12}$$

稠密部分存储非零值所需的空间大小 M_D^v 为：

$$M_D^v = 16 \times (N - N_s) \tag{6.13}$$

因此，在 HLS 中存储 $\mathcal{X}$ 所需的总空间大小 M_H 为：

$$\begin{aligned} M_H =& M_S^i + M_S^v + M_D^i + M_{bm} + M_D^v \\ =& (\lceil \log_2 I \rceil + \lceil \log_2 J \rceil + \lceil \log_2 K \rceil) \times N_s \\ &+ \left(\left\lceil \log_2 \frac{I}{S_I} \right\rceil + \left\lceil \log_2 \frac{J}{S_J} \right\rceil + \left\lceil \log_2 \frac{K}{S_K} \right\rceil \right) \times M \\ &+ (S_I \times S_J \times S_K) \times M + 16 \times N \end{aligned} \tag{6.14}$$

稠密格式，无论数值是否为零，均采用单精度浮点类型（float）存储稀疏张量的所有数值，每个值都以 32 位的 float 类型存储。因此，根据上述推导，可得出 HLS 格式所需的存储空间远小于 Dense 格式。

COO 格式仅存储非零数值。对于三阶张量，它采用三个 32 位整数数组存储

中国知网 https://www.cnki.net

索引，以及一个浮点数组存储非零数值。在 HLS-S 中，由于采用了二进制线性索引，其存储空间远小于 COO 格式。至于 HLS-D，由于应用了位图表示技术，其存储空间不会随着张量块内非零数值的增加而增加，是保持恒定的。然而，对于 COO 格式而言，随着非零数值数量的增加，其所需存储空间也随之增加。因此，当 COO 格式的非零数值个数超过某一阈值时，HLS-D 格式所需的存储空间将更少，即 HLS 格式所占用的空间更小。

6.4 基于 HLS 的 SpTMCM 并行算法

本节演示了基于 HLS 的 SpTMCM 在 GPU 上的实现。首先，给出了利用张量核心加速 SpTMCM 的并行算法。然后，讨论了该算法的阈值设置和任务负载方案。

6.4.1 使用张量核心加速 SpTMCM

基于 HLS 的 SpTMCM 并行算法主要包括对稠密部分计算的稠密核函数和对稀疏部分计算的稀疏核函数。在稠密核函数中，张量块以 HLS-D 格式存储，并且经过一种合理设计的数据填充方案被加载到张量核心上进行计算。而在稀疏核函数中，张量块以 HLS-S 格式存储，并被加载到 CUDA 核心上进行计算。

对于以 HLS-D 格式存储的非零值，SpTTMChain 通过对输入切片和两个因子矩阵进行两次乘法，计算得到一个结果切片，公式（6.5）详细描述了这一过程。相比之下，SpMTTKRP 对应的公式（6.8）在公式（6.5）的基础上增加了一个取对角线的操作。

执行 SpTMCM 的过程以张量块为单位，如图6.5所示，其工作流程可分为四个步骤。首先，通过分块技术将输入张量 $\mathcal{X}$ 划分为多个张量块，并将这些张量块以及两个因子矩阵通过 PCIe 传输到全局内存中。其次，按照第一个模态遍历每个张量块中的所有切片，以执行逐切片计算的 SpTMCM 形式。因子矩阵在首次使用时被加载到寄存器中，并在接下来的循环中被反复利用。第三，SpTMCM 的结果需要在输出之前根据特定运算进行进一步处理。具体而言，在 SpTTMChain 中，该乘积被累积到结果张量 $\mathcal{Y}$ 中对应的切片中。而在 SpMTTKRP 中，该乘积只对结果矩阵 A 的对角元素有贡献，因此这些对角元素被累加到结果矩阵 A 的相应行中。最后，结果通过 PCIe 总线传输回 CPU，以完成计算过程。

张量核心上的 HLS-D 格式对应的计算过程如算法6.1所示。输入张量通过分块技术以及阈值 *b_threshold* 的设定划分得到为多个大小相同的张量块，并由数组 *b_shape* 记录每个张量块的大小。变量 *nnb* 表示总张量块的数量。每个线程块负责计算一个张量块，线程块内的最小操作单元是线程束，每个线程束由 32 个线程组成。因此，线程块可以进一步划分为大小为 N_w 的线程束，这些线程束用于更

中国知网 https://www.cnki.net

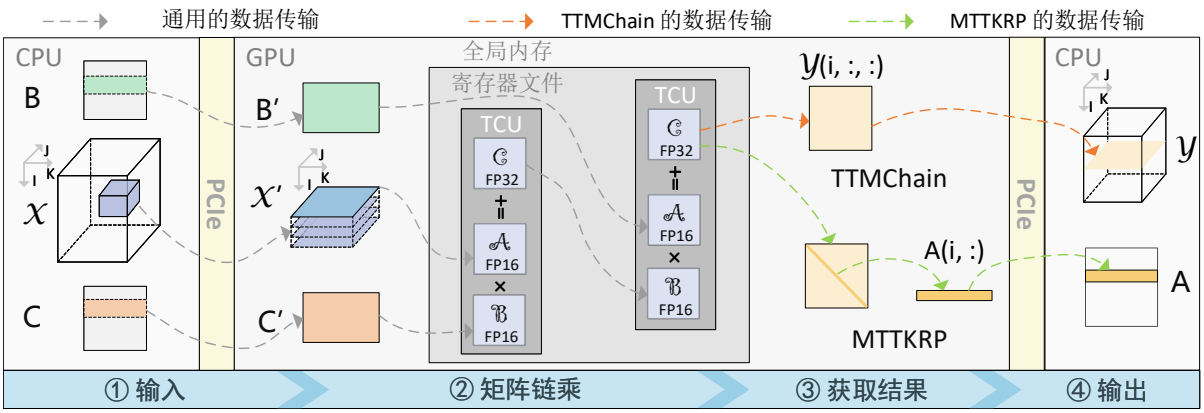


图 6.5　张量核心加速 SpTMCM 的流程图

新结果张量的一个切片或结果矩阵的一行。

算法 6.1 HLS-D 格式的三阶张量在第一模态上的 SpTTMChain 内核函数

输入:

$b_inds[nnb]$, $b_bmp[64 \times nnb]$, $b_vals[nnz_D]$,
$b_val_ptr[nnb+1]$, $b_linear_bits[3]$,
$B[J][R_1]$, $C[K][R_2]$;

输出:

$\mathcal{Y} \in \mathbb{R}^{I \times R_1 \times R_2}$;

using namespace nvcuda::wMMA;
$id_w \leftarrow threadIdx.x/32$;
__shared__ uint $pre_shm[64]$;
__shared__ half $xcs[N_w \times 256]$;
fragment x_frag, c_frag, m_frag, bT_frag,
xc_frag, y_frag;
if $blockIdx.x < nnb$ **then**
  // 获取张量块的坐标索引
  $b_index \leftarrow b_inds[blockIdx.x]$;
  $b_COO \leftarrow$ Binary2COO(b_linear_bits, b_index);
  // 得到相应的矩阵位置
  $B_t \leftarrow B + b_COO[1] \times 16 \times R_1$,
  $C_t \leftarrow C + b_COO[2] \times 16 \times R_2$;
  load_matrix_sync(bT_frag, B_t, R_1), load_matrix_sync(C_frag, C_t, R_2);
  // 获取当前张量块的位图位置
  $bmp_t \leftarrow b_bmp + 64 \times blockIdx.x$;
  $vals_t \leftarrow b_vals + b_val_ptr[blockIdx.x]$;
  // 计算前缀和

中国知网 https://www.cnki.net

Get_Prefix_Sum(pre_shm, bmp_t);
$off_w = id_w \times 16/N_w$;
for $i \leftarrow off_w$ **to** $(id_w + 1) \times 16/N_w$ **do**
  fill_fragment($xc_frag, 0.0f$);
  fill_fragment($x_frag, 0.0f$);
  Fill2Frag($x_frag, vals_t, pre_shm, bmp_t$);
  MMA_sync($xc_frag, x_frag, c_frag, xc_frag$);
  store_matrix_sync($xcs + id_w \times 256$, xc_frag, 16, mem_row_major);
  load_matrix_sync(m_frag, $xcs + id_w \times 256$, 16);
  MMA_sync($y_frag, bT_frag, m_frag, y_frag$);
  $\mathcal{Y}(i, :, :) += y_frag$;
end for
end if
return $\mathcal{Y}$;

为了更有效地利用张量核心，*b_shape* 的设定为 $16, 16, 16$，这意味着一个张量块由 16 个切片组成，每个切片的大小为 16×16。这种设置方式确保了张量块中的所有模态的维度大小都相同，有助于减少稀疏张量在不同模态下计算时的模态转换开销。

在图6.4（a）中，由于张量块的索引被映射为 HLS 格式的二进制线性索引，因此要获得当前张量块的相应矩阵位置，必须将二进制线性索引解压缩为坐标。算法6.2表示了二进制线性索引到 COO 整数索引的转换函数 *Binary2COO*，该函数在算法6.1的第9行被调用。

算法 6.2 Binary2COO() 函数

输入:

$b_linear_bits[3], b_index$;

输出:

$b_COO[3]$;

$mask \leftarrow$0xFFFFFFFFFFFFFFFF;
$b0 \leftarrow b_linear_bits[0]$;
$b1 \leftarrow b_linear_bits[1]$;
$b2 \leftarrow b_linear_bits[2]$;
$b_COO[0] \leftarrow b_index \gg (b1 + b2)$,
$b_COO[1] \leftarrow (b_index \gg b1)\& \sim (mask \ll b1)$,
$b_COO[2] \leftarrow b_index\& \sim (mask \ll b2)$;
return $b_COO[3]$;

中国知网 https://www.cnki.net

在公式 (6.5) 中，同一模态下的所有切片都共享相同的因子矩阵。因此，在因子矩阵 B 和 C 首次通过线程束加载后，它们将一直保留在寄存器中，即算法6.1中的第12行。这种方式利用了张量分块带来的数据局部性，为后续切片计算提供了数据重用。

多个线程束并行计算张量块中的所有切片，每个线程束处理 *16/N_w* 切片，即算法6.1中的第19行。然后，将 $\mathcal{X}(i,:,:)$ 与 C 相乘，得到中间结果 $\mathcal{X}(i,:,:)C$。在 WMMA 接口中，输入和输出都使用了专用寄存器结构体 *Afrag*、*Bfrag* 和 *Cfrag*，这些寄存器结构体之间不能直接相互赋值。

因此，在算法6.1中的第21行，将第 i 个切片按照图6.4（b）所示的方式填充到输入结构体中。第一次矩形乘法计算后，输出的中间结果 $\mathcal{X}(i,:,:)C$ 需要作为第二次矩形乘法的输入，即算法6.1中的第24和第25行。在第 i 个切片对应的第二次矩形乘法中，*Afrag* 载入的是 B^T，而 *Bfrag* 载入的是 $\mathcal{X}(i,:,:)C$。最后，将所有第 i 个切片对应的计算结果累加起来，以得到结果张量中的第 i 个切片。

与 SpTTMChain 相比，SpMTTKRP 中需要额外的一个步骤，即取最后一次矩形计算得到的结果矩阵上的对角线，并将该对角线组成因子矩阵 A 的第 i 行。因此，与传统的优化工作遵循的逐值计算方式不同，逐切片计算方式更适用于张量核心上的 SpTTMChain 和 SpMTTKRP。受益于分块技术，一个张量块中所有切片涉及的因子矩阵是不变的，它们可以驻留在寄存器中，以便后续计算重用。这种重用性有效地减少了计算过程中对全局内存访问的依赖，从而提高了整体的计算效率。

对于以 HLS-S 格式存储的非零值，每个非零值都以逐值的方式计算其对应的结果。所有非零值被均匀地分配给每个线程块进行计算，每个非零值由一个 CUDA 线程处理。首先，为每个非零值分配一个线程，并且通过算法6.2将经过位压缩的二进制线性索引解压缩为多维整数索引，以确定结果值的位置。最后，每个线程计算出相应的结果值。在 SpMTTKRP 中，这些结果值通过公式（6.2）来更新结果矩阵 Y 中的对应值；而在 SpTTMChain 中，这些结果值通过公式（6.1）来更新结果张量 $\mathcal{Y}$。

基于 HLS 格式的 SpTMCM 算法展示了对张量核心的适应性，但还存在两个关键问题需要解决：1、阈值大小的设置问题，这会影响 HLS 格式中稠密部分和稀疏部分的数据存储量，以及对张量核心和 CUDA 核心的有效利用。2、任务负载方案的设计问题，这涉及到张量核心与 CUDA 核心之间的协作方式，以及提高 GPU 利用率的方法。

6.4.2 HLS 的张量块阈值分析

不同的阈值大小会影响 HLS-D 和 HLS-S 中的计算量。当阈值增加时，HLS-D 的计算量减少，HLS-S 的计算量增加。相反，当阈值减小时，HLS-D 的计算量增

中国知网 https://www.cnki.net

加，HLS-S 的计算量减少。如果某一种格式的计算量过多或过少，都难以有效利用张量核心。因此，需要找到一个最优的阈值，以利用张量核心，同时避免不必要的计算。

由于以 HLS-D 格式存储的计算量不受块内非零值数量的影响，而以 HLS-S 格式存储的计算量会随着非零值数量的线性增加而变化。因此，当 HLS-S 的最小计算量低于 HLS-D 的计算量，且 HLS-S 的最大计算量高于 HLS-D 的计算量时，HLS-D 和 HLS-S 的计算量曲线将会交汇于一点。因此，该交点就是理论上最佳的张量块阈值。

6.4.3 任务负载方案

合适的任务调度策略有利于提高 GPU 整体的利用率[162]。GPU 上混合格式常用的两种任务负载策略是顺序负载和融合负载。

顺序负载方案，如 CUSP[163]，通过逐个启动混合格式的各个部分，将其作为单独的网格在 GPU 上执行，实现了整体的计算。这种方法容易实现，但存在一个缺点，即如果混合格式的计算网格中任何部分的线程块数量太少，就会导致一些流多处理器闲置，即 GPU 占用率降低。

融合负载方案，如 TileSpGEMM[107]，统一调度混合格式各个部分的计算任务。如果来自混合格式的任何部分的线程块无法占用整个 GPU，则融合负载方案会提前调度来自混合格式其他部分的线程块进入 GPU 执行。与顺序负载方案相比，该方法的优点是提高了 GPU 的利用率，但实现难度较高。为了充分发挥 GPU 的潜力，本章提出了一种基于 HLS 格式的融合负载方案，其中的张量计算任务被加载到 GPU 的流多处理器上。该过程如图6.6所示。

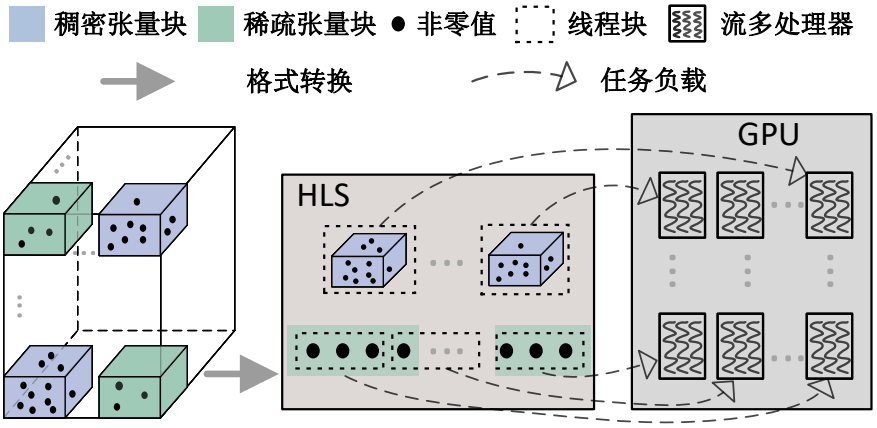


图 6.6 HLS 格式计算任务加载到 GPU 的流多处理器上的过程示意图

HLS 格式的张量由稠密张量块和离散非零值组成，它们被组织成不同的计算任务，以负载到流多处理器上进行处理。在 HLS-D 中，每个稠密张量块的计算任务被组织到一个线程块中，并以基于矩形计算的方式负载到张量核心上进行处

中国知网 https://www.cnki.net

理。而在 HLS-S 中，离散非零值不适合矩形计算，因此每个线程块使用 CUDA 核心处理一批数量相等的非零值，以确保负载在各个线程块之间平衡。

6.5 实验评估

本节通过大量的实验对本章提出的名为 HLSTO 的基于 HLS 格式的 SpTMCM 方法进行了充分的评估。首先介绍了 HLSTO 的参数设置，并验证了融合任务负载方案的有效性。其次进行了对 HLS 格式的存储空间和计算带宽的分析，以阐明 HLS 格式的优点和不足之处。最后将 HLSTO 与当前最先进的 SpTMCM 实现进行了比较，以证明本章方法在整体性能上的优势和先进性。

6.5.1 实验设置

1、 平台

本实验使用了两种不同架构的 GPU：A100 GPU 采用了 Ampere 架构，并安装在 Intel Xeon Gold 5120 处理器上；NVIDIA TITAN RTX GPU（简称 TITAN）采用了 Volta 架构，并安装在 Intel Xeon Silver 4110 处理器上。详细的硬件配置如表6.1所示。

表 6.1 SpTMCM 实验平台配置总结表

参数	A100 平台		TITAN 平台	
	Intel(R) Xeon(R) Gold 5120 CPU	NVIDIA A100 GPU	Intel(R) Xeon(R) Silver 4110 CPU	NVIDIA TITAN RTX GPU
微架构	Skylake	Ampere	Skylake	Turing
CUDA 核心数	-	6912	-	4608
张量核心数	-	432	-	576
频率	2.20 GHz	1.41 GHz	2.10 GHz	1.77GHz
内存大小	64 G	40 G	132 G	24 G
最后一级缓存大小	19.25 M	80 M	11 M	6 M
编译器	GCC 7.5.0	NVCC 11.2	GCC 5.4.0	NVCC 10.1

2、 数据集

本实验使用六个从真实数据集中获得的三阶稀疏张量，这六个真实场景的张量数据集分别来自 GroupLens Research [①] 和滴滴出行 GAIA 倡议[②]。表6.2是六个数据集的总结，其中展示的前三个数据集均由 GroupLens Research 发布。

[①] https://grouplens.org

[②] https://gaia.didichuxing.com

中国知网 https://www.cnki.net

电影推荐服务 MovieLens 成员的匿名电影评分描述在数据集 Movies[③] 中。Art 数据集[④] 包括来自互联网音乐服务 Last.fm 的 2000 名用户的社交网络、标签和音乐家收听数据。Book 数据集包括来自 Delicious 社交书签系统的 2000 个用户的社交网络、书签和标记信息。

后三个数据集 G1、G2、和 G3 来自同一场景下不同日期的车辆轨迹数据，它们反映了张量的稀疏分布在实际使用场景中的相似性。

表 6.2　SpTMCM 实验中数据集的总结表

数据集名称	I	J	K	非零值数	稀疏度
Movies	1,238	5,665	1,078	11,440,996	0.152%
Art	669	379	251	63,066	0.991%
Book	364	104	227	93,866	1.092%
G1	480	875	749	10,685,223	3.397%
G2	480	875	749	10,806,566	3.435%
G3	480	875	749	10,638,476	3.381%

3、 张量块阈值设置

在多维分块技术中，张量块的大小设置为 $16 \times 16 \times 16$。通过生成大量具有不同张量块密度的张量作为测试数据集来确定张量块阈值，张量块密度是张量块中非零值所占的比例。在生成这些数据集时，通过设置每个模态的稀疏度来控制稀疏张量的整体稀疏度，通过设置张量块内密度来控制张量块内阈值的测试。在每个模态下保持稀疏张量的稀疏度相同，并调整每个实验的阈值，从而找到最优阈值。

实验结果如图6.7所示，SpMTTKRP 和 SpTTMChain 中张量块的最优密度在 A100 上分别为 1.90% 和 2.00%，在 TITAN 上分别为 1.50% 和 0.90%。因此，张量块的最优阈值在 A100 上分别被设置为 78 和 61，在 TITAN 上分别被设置为 82 和 41。在接下来的实验中，将默认使用上述阈值作为 HLSTO 的实验参数。

4、 融合负载策略的有效性

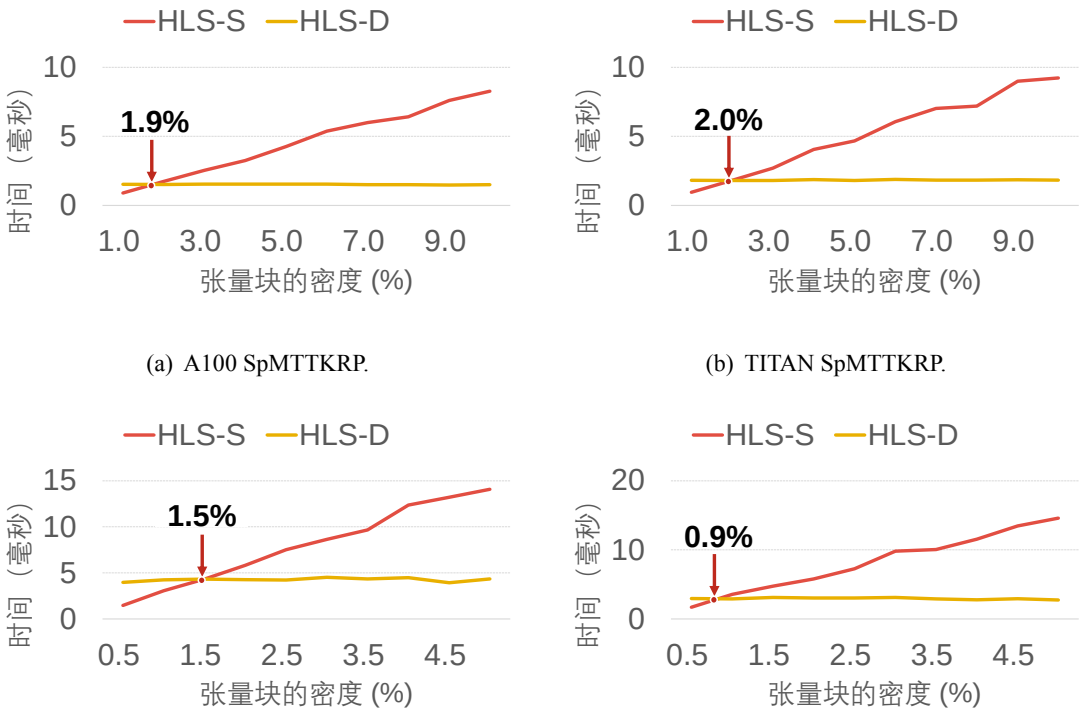
由于融合方案在 GPU 占用率较低的情况下能发挥更大的作用，这意味着在具有稠密张量块或离散非零值的 HLS 格式张量不能占用整个 GPU 的场景下，融合负载方案将获得更显著的性能提升。在实验使用的数据集中，只有 Art 和 Book 属于这种情况，所以本节重点展示 HLSTO 在这两个数据集上的实验结果。

图6.8显示了执行融合负载方案和顺序负载方案的时间开销比较。从图中可以

[③] https://grouplens.org/datasets/movielens/

[④] https://grouplens.org/datasets/hetrec-2011/

中国知网 https://www.cnki.net



(a) A100 SpMTTKRP. (b) TITAN SpMTTKRP.

(c) A100 SpTTMChain. (d) TITAN SpTTMChain.

图 6.7　不同张量块密度对应的结果图

观察到，在 Art 张量和 Book 张量上，融合负载方案的性能优于顺序负载方案。具体来说，对于 SpMTTKRP，融合负载方案相对于顺序负载方案分别实现了 8.30× 和 7.00× 的加速比；而对于 SpTTMChain，融合负载方案在 Art 张量和 Book 张量上分别实现了 5.00× 和 1.38× 的加速比。

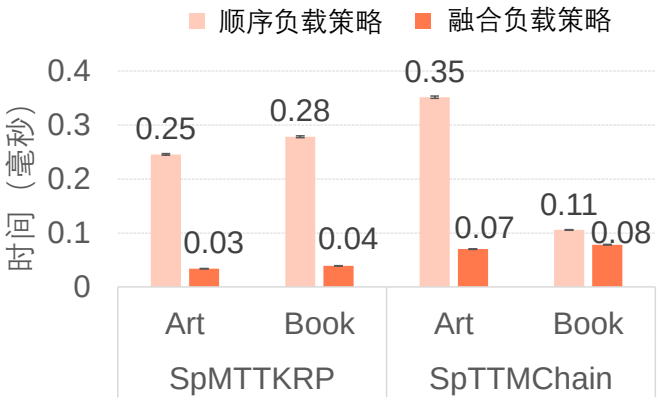


图 6.8　A100 上执行融合任务负载方案前后的时间开销比较的结果图

此外，在 Book 数据集上，SpTTMChain 的融合负载策略与顺序负载策略之间差距很小，其原因在于 Book 数据集比 Art 数据集更稠密，并且 SpMTTKRP 比 SpTTMChain 计算步骤更多，计算强度更大，更易使得 GPU 满负载。

中国知网 https://www.cnki.net

6.5.2 HLS 的有效性

1、 内存使用分析

表6.3提供了 A100 GPU 上使用 HLS 格式的稀疏张量的内存使用情况的详细信息。不同的格式记录值所需的浮点数位数不同，HLS 格式中的 HLS-D 和 HLS-S 都使用了 16 位的浮点数类型，而 Dense 和 COO 格式使用了 32 位的浮点数类型。在 HLS 格式存储的数据集中，SpMTTKRP 和 SpTTMChain 的内存使用情况也有所不同，这是因为它们使用了不同的阈值条件，从而导致 HLS-D 和 HLS-S 两部分的数据量也不同。表格中的 HLS (SpMTTKRP) 和 HLS (SpTTMChain) 分别表示在两种阈值条件下 HLS 所需的存储空间，而 HLS-D、HLS-S、COO 和 Dense 则表示使用相应格式存储整个张量所需的存储空间。

从实验结果可以看出，稠密格式存储稀疏张量的存储空间开销最大，其次是 COO 格式。此外，在 SpMTTKRP 和 SpTTMChain 的两种阈值条件下，HLS 格式的存储空间都小于 HLS-D 和 HLS-S。对于 G1、G2 和 G3 张量，HLS-D 的存储空间小于 HLS-S，这是因为轨迹数据在张量块中具有局部聚集的稀疏性，即张量中的非零值总是分布在少数张量块中。

表 6.3 数据集的内存使用情况（MB）的总结表

张量	HLS				HLS-S	HLS-D	COO	Dense
	SpMTTKRP		SpTTMChain		-	-	-	-
	A100	TITAN	A100	TITAN	-	-	-	-
Movies	49.88	49.63	51.58	55.48	65.47	175.25	174.58	28840.28
Art	0.25	0.25	0.27	0.29	0.30	0.87	0.96	242.77
Book	0.39	0.39	0.41	0.45	0.45	1.04	1.43	32.78
G1	38.11	37.95	38.98	40.45	50.95	48.23	163.04	1200.03
G2	38.26	38.09	39.17	40.61	50.95	48.35	164.90	1200.03
G3	38.00	37.83	38.87	40.40	50.73	48.23	162.33	1200.03

实验结果表明，HLS 比 HLS-S 平均减少 19.95% 的存储空间，比 HLS-D 平均减少 43.83% 的存储空间。因此，混合格式相对于单一格式而言，更有利于节省存储空间。HLS 比 COO 平均减少 74.17% 的存储空间，比 Dense 格式平均减少 97.85% 的存储空间。因此，与其他存储格式相比，HLS 格式需要的存储空间更少。

中国知网 https://www.cnki.net

2、 计算带宽与时间开销分析

图6.9描述了 SpMTTKRP 和 SpTTMChain 在全稀疏的 HLS-S、全稠密的 HLS-D 和混合类型的 HLS 存储格式下的内存带宽平均值。由于这些不同的存储格式只压缩张量块和非零值的索引数据，所以本节以索引占用的内存空间作为数据传输的总量进行比较。

实验结果显示采用 HLS 格式的 SpTMCM 比采用 HLS 格式的 SpTMCM 带宽更高。G1、G2 和 G3 数据集使用 HLS 格式时带宽可达 14 GB/s，使用 HLS-S 格式时带宽仅为 13 GB/s。而 HLS-D 格式下的带宽更高，达到 16 GB/s 左右。由于 HLS 将稀疏度高的张量块重新打散成非零值，因此 HLS 的内存带宽比 HLS-D 稍低，但 HLS-D 所需的存储空间却比 HLS 高得多。总之，相对其他格式，HLS 格式可以有效地提升内存带宽。

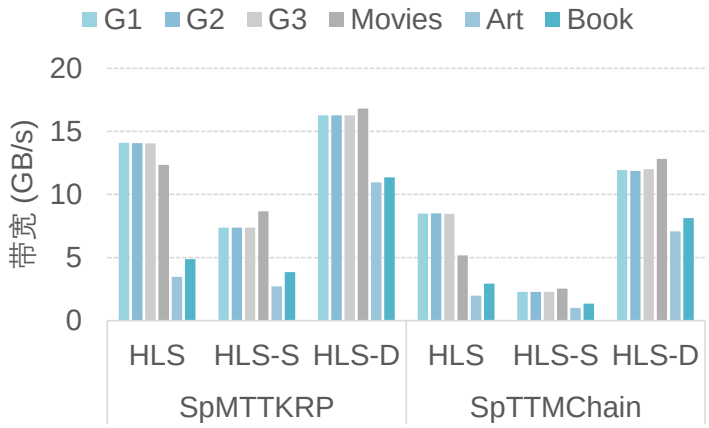


图 6.9　A100 上 SpTMCM 内存带宽的结果图

图6.10显示了 A100 上 HLS、HLS-S 和 HLS-D 的 SpMTTKRP 和 SpTTMChain 的时间开销。在 SpMTTKRP 实验中，对于非零较少数据集，如 Art 和 Book，HLS 的性能略低于 HLS-S 和 HLS-D。对于大多数具有大量非零值的数据集，如 Movies、G1、G2 和 G3，HLS 的性能明显优于 HLS-S 和 HLS-D。因此，在 SpMTTKRP 中，HLS 实现了 1.35× 到 3.98× 的加速。对于 SpTTMChain，HLS 在所有数据集上的性能都优于 HLS-S 和 HLS-D，实现了 1.04× 到 6.00× 的加速。

总体来看，计算带宽在 HLS 下平均提高了 136.00%。尽管与 HLS-D 相比，HLS 的计算带宽平均减少了 36.41%，但它相对于 HLS-D 节省了 43.83% 的存储空间。因此，HLS 在存储空间和计算带宽之间取得了最佳的平衡。

3、 误差分析

由于 HLSTO 使用了张量核心参与计算，而张量核心要求数据精度是 16 位的浮点数类型，因此使用对称平均绝对百分比误差作为误差指标，来衡量 HLSTO 的有效性。令 x_i 是在张量核心上的计算结果，$\hat{x}_i$ 是与 CPU 上使用 32 位浮点数的

中国知网 https://www.cnki.net

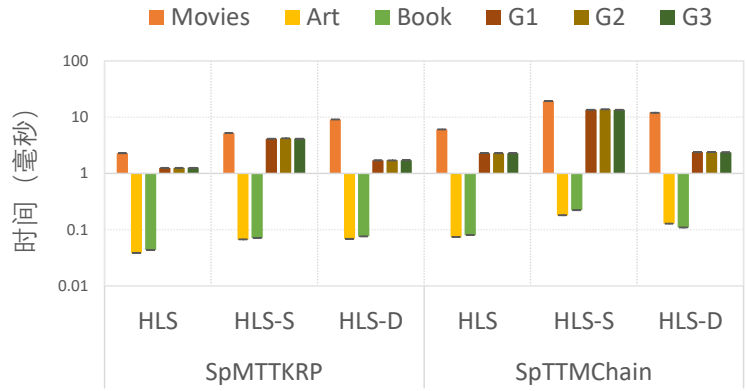


图 6.10　A100 上 SpTMCM 的时间开销的结果图

计算结果，误差的计算方式如下：

$$Error = \frac{100\%}{n}\sum_{i=1}^{n}\frac{|x_i - \hat{x}_i|}{|x_i|+|\hat{x}_i|} \tag{6.15}$$

由于 SpTTMChain 和 SpMTTKRP 返回不同的结果在形式上存在区别，因此它们计算误差的方式略有不同。具体而言，SpTTMChain 是计算两个张量之间的误差，SpMTTKRP 是计算两个因子矩阵之间的误差。从图6.11可以观察到，HLSTO 在 A100 和 TITAN GPU 上的误差都在 0.17% 以下。这意味着 HLSTO 在不同的 GPU 架构上具有较低的相对误差，即 HLSTO 具有较好的可扩展性。

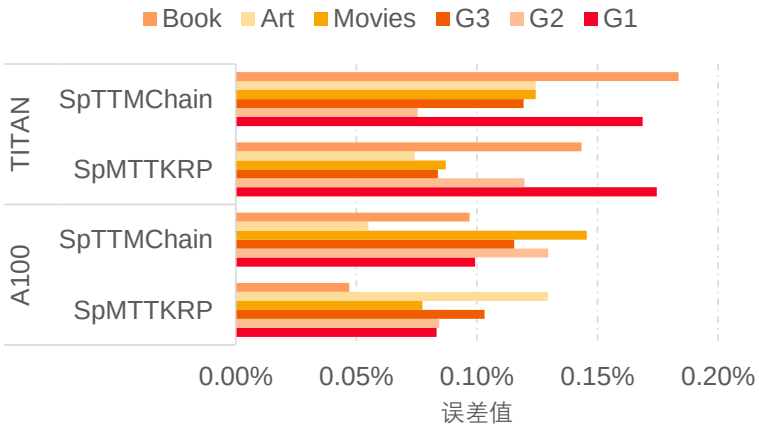


图 6.11　SpTMCM 的计算误差的结果图

6.5.3　与其他实现的比较

1、　运行时开销

在实验中，HLSTO 方法与其他最近的研究方法进行了性能比较，如 Hi-COO[58], BLCO[49], MM-CSF[48], F-COO[59], FWC[46], B-CSF[28]。这些方法都是来

中国知网　https://www.cnki.net

自于一些基础矩阵存储格式的扩展。具体来说，Hi-COO、BLCO、F-COO 和 HLSTO 是 COO 格式的扩展，而 MM-CSF、FWC 和 B-CSF 则是 CSR 存储格式的扩展。

在 SpMTTKRP 任务上，图6.12展示了这些方法的性能比较结果。大多数数据集中，HLSTO 表现出了更好的性能，尤其是在非零值较多的数据集（如 G1、G2 和 G3）上。对于相对非零值较少的数据集（如 Art 和 Book），HLSTO 也表现出较低的时间开销。总体而言，HLSTO 在 SpMTTKRP 任务上实现了显著的性能提升，加速比为 1.16× 到 24.12×。

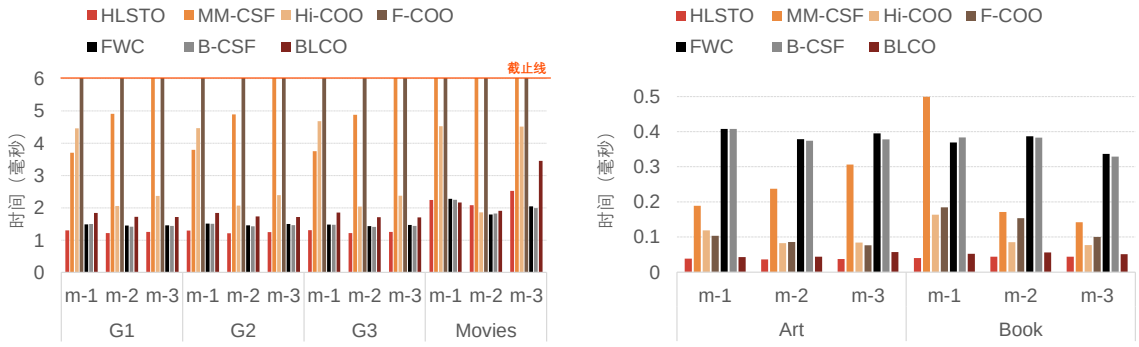


图 6.12　SpMTTKRP 不同方法在 A100 上的性能的结果图

在 SpTTMChain 任务上，图6.13展示了 HLSTO 与 Hi-COO 和 MM-CSF 的性能比较结果。HLSTO 在 SpTTMChain 任务上也表现出了较好的性能，实现了 5.07× 到 7.15× 的加速比。这表明 HLSTO 在处理不同类型的稀疏张量计算任务时都具有显著的性能优势。

总体而言，HLSTO 方法在不同类型的稀疏张量计算任务中都表现出了优越的性能，与现有方法相比取得了显著的加速效果。这证明了 HLSTO 方法在链乘计算型稀疏张量运算领域的先进性和有效性。

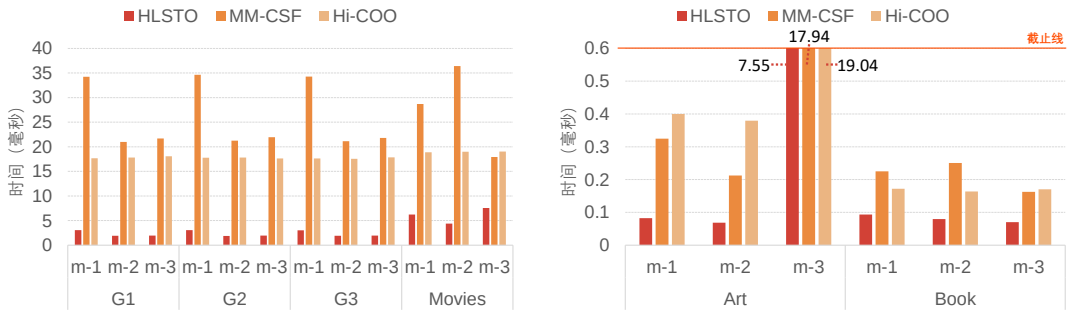


图 6.13　SpTTMChain 不同方法在 A100 上的性能的结果图

2、 格式构建开销

从标准稀疏格式 COO 到其他格式的转换是在实际计算之前进行的预处理过程。该过程的时间开销通常比单次 SpTMCM 的运行时间要大得多，因为它涉及

中国知网 https://www.cnki.net

到重组和转换数据结构。在实际应用中，SpTMCM 通常需要进行数百甚至数千次迭代，因此预处理的额外开销会分摊到多个计算中。与其他方法不同，HLSTO 需要为每个张量块生成一个位图索引来利用张量核心，并且使用细粒度的数据结构来避免原子操作。

图6.14显示了从标准稀疏格式 COO 到其他格式的转换所需的时间。可以看出，所有高级格式的转换时间都以秒为单位，这表明转换过程相对耗时。与其他方法相比，HLSTO 所需的转换时间比大多数方法要少，这意味着 HLSTO 具有很强的竞争力。

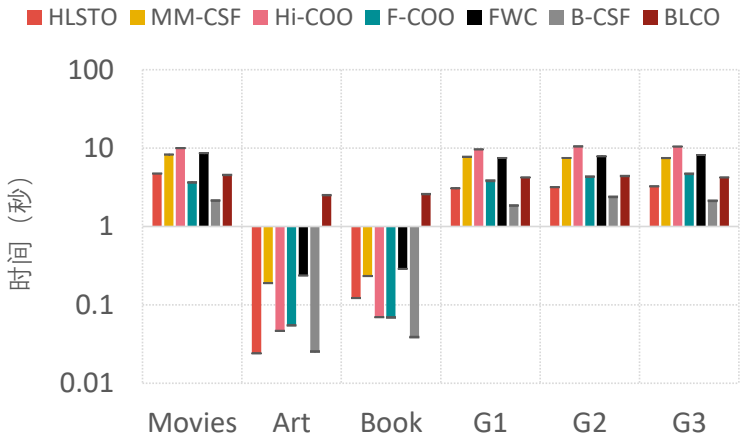


图 6.14　从标准稀疏格式 COO 到其他格式的转换时间的结果图

6.6　本章小结

本章以 SpTMCM 为例，基于 HASTensor 中的优化技术提出了一种基于混合线性格式的张量矩阵链乘算子方法，即 HLSTO，展示了 HASTensor 库在链乘计算型稀疏张量运算性能优化方面的有效支持。HLSTO 采用混合格式 HLS 来存储稀疏张量，通过分块技术将张量分解成多个张量块，以改善数据局部性。随后，深入分析了混合格式 HLS 中的 HLS-D 和 HLS-S 两部分的计算量，并通过实验确定了 HLS 的最佳阈值。接下来，基于 HLS 设计了 SpTMCM 的并行算法，并引入了融合负载策略，以实现张量核心和 CUDA 核心的协同计算，以提高 GPU 的利用率。实验结果表明，在真实场景的张量数据集上，本章提出的 HLSTO 方法在 GPU 上胜过其他先进的 SpTMCM 实现。

中国知网　https://www.cnki.net

结论与展望

稀疏张量，由于其高效表达稀疏高维数据结构的潜力，逐渐在数据科学、人工智能和工程领域崭露头角，其重要性逐渐凸显。因此，对稀疏张量运算库的需求正在各行各业中迅速增加。与此同时，随着处理器架构由单一结构向异构结构的演进，也为稀疏张量运算的优化提供了新的机遇。

本文主要关注在 CPU-GPU 平台上稀疏张量运算库的设计与性能优化。稀疏张量运算依据其乘数维度以及乘数数量被总结为四种类型，包括访存受限类型的 SpTV、批量计算类型的 SpTTM、高维计算类型的 SpTC 以及序贯计算类型的 SpTMCM。本文以这四种典型的稀疏张量运算作为研究对象，对其进行深入的研究和探索。

首先，本文从稀疏张量在大数据时代的应用背景出发，结合处理器架构的发展趋势，阐述了在 CPU-GPU 异构平台上进行稀疏张量运算的并行算法设计和性能优化的必要性和紧迫性。接着，对四种类型的稀疏张量运算进行了总结，并以 SpTV、SpTTM、SpTC 和 SpTMCM 为代表，详细讨论了稀疏张量运算库的设计和性能优化的研究价值和意义，同时深入探讨了每种类型的稀疏张量运算所面临的主要挑战。随后，本文全面梳理了国内外稀疏张量运算并行算法设计和性能优化的研究现状，并总结了早期研究工作的不足之处。

针对 CPU-GPU 异构计算平台，本文设计了一款名为 HASTensor 的稀疏张量运算库。该库采用切片粒度和张量块粒度两种张量格式作为高效存储格式，借助性能建模、深度学习和最新的硬件张量核心等优化技术，从四个方面展开深入研究。

1、以 SpTV 为例，详细描述了访存受限型稀疏张量运算的并行算法设计和性能优化；

2、以 SpTTM 为例，详细描述了批量计算型稀疏张量运算的并行算法设计和性能优化；

3、以 SpTC 为例，详细描述了高维计算型稀疏张量运算的并行算法设计和性能优化；

4、以 SpTMCM 为例，详细描述序贯计算型稀疏张量运算的并行算法设计和性能优化。

本文各部分所完成的主要研究内容和贡献总结如下：

针对 SpTV 中由于访存受限特征导致的计算效率问题，本文的第一个研究提出了一种名为 IAP-SpTV 的创新性方法。该方法通过融合混合张量格式和流水线技术，动态选择最佳的存储格式和流水线数量，显著提升了稀疏张量与向量的乘

中国知网 https://www.cnki.net

法运算效率。特别的是，IAP-SpTV 具有很好的适应性，不仅适用于各类数据类型和硬件环境，还能够巧妙地较好地重叠计算和数据传输，从而极大增强了计算性能和效率。

针对 SpTTM 中批量处理的计算过程，本文的第二个研究以 CPU-GPU 异构平台为焦点，为 SpTTM 设计了一系列专门的异构并行算法，以适应不同硬件架构的独特特性。通过合理规划的任务映射策略、稀疏格式选择模型和内存访问模式的优化，实现了稀疏张量乘以矩阵运算的高效并行计算。这项创新性的设计不仅显著提高了计算速度，同时也优化了计算资源的高效利用。

针对 SpTC 中高维计算的特性，本文的第三个研究采用了位压缩和位图技术，以降低高维张量的存储开销，从而显著提高内存访问效率。此外，引入了冲突解决方法，通过构建任务列表和消除任务之间的依赖关系，有效提高了线程块的利用率。通过设计高效的张量核心填充策略，确保在处理各类张量缩并任务时，张量核心得到较好地利用。这些策略不仅在充分发挥了新一代 GPU 性能优势的同时，也显著加速了高维稀疏数据的并行计算。

针对 SpTMCM 中由于序贯计算导致的计算依赖问题，本文的第四个研究引入了一项创新性的统一计算方式，将计算粒度从非零元素粗化到切片粒度，以适应张量核心的矩形计算方式。通过采用多维张量块的混合张量格式和高效的内存访问模式，降低连续计算过程中的计算依赖，进一步实现 SpTMCM 的高效并行算法。这一方法能够利用 GPU 上不同计算核心的优势，从而实现对连续张量乘法运算的性能提升。

本文在稀疏张量运算的并行算法设计和性能优化领域取得了较好的研究成果，充分地补充完善了该领域的前沿探索研究。然而，由于时间和技术的限制，本文的工作仍然存在着一定的局限性，值得进一步深入研究。综合国内外最新的研究成果，虽然高维稀疏张量运算的并行算法设计和性能优化已经取得了较大的进步，还存在很多具有挑战性且意义重大的问题尚待解决，未来的研究可以在以下几个方面深入探讨：

1、面向人工智能应用场景的新型稀疏张量运算的并行算法设计和性能优化。随着人工智能领域的快速发展，不断涌现出新的深度学习模型。新模型的提出对稀疏张量运算带来了全新的需求，需要设计新的并行算法和性能优化技术，以满足模型高效运行的要求。

2、面向人工智能算子融合后的复杂稀疏张量运算的并行算法设计和性能优化。在人工智能应用中，张量数据的组织和处理变得更加复杂，而算子融合技术则进一步增加了稀疏张量运算的复杂性。这涉及到访存密集型和计算密集型算子的融合，需要研究高效地处理这种复杂的稀疏张量运算的方法，以满足人工智能应用的性能需求。

3、面向大数据处理应用场景的分布式张量运算的并行算法设计和性能优化。

中国知网 https://www.cnki.net

大数据处理常常在分布式集群环境中进行，这带来了新的挑战，包括设备间的通信和任务调度。稀疏张量运算在这种环境下需要考虑更复杂的问题，如任务划分和集群配置，因此需要设计高性能的分布式张量运算算法和灵活的任务调度策略。

4）面向时序数据挖掘应用场景的流式张量运算的并行算法设计和性能优化。时序数据通常以流式张量的形式组织，其中的数据在某些维度上具有相似性。利用这种相似性可以优化稀疏张量运算的性能。例如，可以设计专门的算法来处理流式张量中连续切片之间的相似结构，从而提高计算效率。因此，利用流式张量的特点设计适用的并行算法是未来研究的一个重要方向。

中国知网 https://www.cnki.net

参考文献

[1] Zhang J Z, Xu W, Hu P. Tightly Integrated Multiomics-Based Deep Tensor Survival Model for Time-to-Event Prediction. Bioinformatics, 2022, 38(12):3259–3266

[2] Xu X, Zhang T, Xu C, et al. Spatial–Temporal Tensor Graph Convolutional Network for Traffic Speed Prediction. IEEE Transactions on Intelligent Transportation Systems, 2022, 24(1):92–103

[3] Sofuoglu S E, Aviyente S. GLOSS: Tensor-Based Anomaly Detection in Spatiotemporal Urban Traffic Data. Signal Processing, 2022, 192:108370

[4] Lauriola I, Lavelli A, Aiolli F. An Introduction to Deep Learning in Natural Language Processing: Models, Techniques, and Tools. Neurocomputing, 2022, 470:443–456

[5] Wang D, Wu B, Zhao G, et al. Kronecker CP Decomposition With Fast Multiplication for Compressing RNNs. IEEE Transactions on Neural Networks and Learning Systems, 2023, 34(5):2205–2219

[6] Tahmasbi H, Jalali M, Shakeri H. Modeling User Preference Dynamics With Coupled Tensor Factorization for Social Media Recommendation. Journal of Ambient Intelligence and Humanized Computing, 2021, 12:9693–9712

[7] 郑皎凌, 乔少杰, 舒红平, et al. 面向多维稀疏数据仓库的欺诈销售行为挖掘. 软件学报, 2020, 31(03):710–725

[8] Li Q, Zeng C, Xu W, et al. A Social Rumor and Anti-Rumor Game Diffusion Model Based on Sparse Representation and Tensor Completion. Journal of Network and Computer Applications, 2022, 201:103343

[9] Gillet A, Leclercq É, Cullot N. Multi-Level Optimization of the Canonical Polyadic Tensor Decomposition at Large-Scale: Application to the Stratification of Social Networks Through Deflation. Information Systems, 2023, 112:102142

[10] 杨克宇, 高云君, 陈璐, et al. 分布式的增量式张量 Tucker 分解方法. 计算机学报, 2021, 44(08):1696–1713

[11] Wang J, Han H, Li H, et al. Multiple Strategies Differential Privacy on Sparse Tensor Factorization for Network Traffic Analysis in 5G. IEEE Transactions on Industrial Informatics, 2021, 18(3):1939–1948

[12] Senanayake R, Hong C, Wang Z, et al. A Sparse Iteration Space Transformation Framework for Sparse Tensor Algebra. In: Proc of The ACM on Programming Languages, volume 4. Virtual Event, USA, 2020, 1–30

[13] Ahrens P, Kjolstad F, Amarasinghe S. Autoscheduling for Sparse Tensor Algebra

中国知网 https://www.cnki.net

With an Asymptotic Cost Model. In: Proc of The 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation. San Diego, California, USA, 2022, 269–285

[14] Chou S, Kjolstad F, Amarasinghe S. Automatic Generation of Efficient Sparse Tensor Format Conversion Routines. In: Proc of The 41st ACM SIGPLAN Conference on Programming Language Design and Implementation. London, UK, 2020, 823–838

[15] Karsavuran M O, Acer S, Aykanat C. Partitioning Models for General Medium-Grain Parallel Sparse Tensor Decomposition. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(1):147–159

[16] Yu J, Zhou G, Sun W, et al. Robust to Rank Selection: Low-Rank Sparse Tensor-Ring Completion. IEEE Transactions on Neural Networks and Learning Systems, 2023, 34(5):2451–2465

[17] 杨礼吉, 王家祺, 景丽萍, et al. 基于张量计算的卷积神经网络语义表示学习. 计算机学报, 2023, 46(03):568–578

[18] Bik A, Koanantakool P, Shpeisman T, et al. Compiler Support for Sparse Tensor Computations in MLIR. ACM Transactions on Architecture and Code Optimization, 2022, 19(4):1–25

[19] 宋冰冰, 张浩, 吴子锋, et al. 自动化张量分解加速卷积神经网络. 软件学报, 2021, 32(11):3468–3481

[20] Wang J, Ding D, Li Z, et al. Sparse Tensor-Based Multiscale Representation for Point Cloud Geometry Compression. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023, 45(7):9055–9071

[21] Chen Y, Xiao G, Özsu M T, et al. aeSpTV: An Adaptive and Efficient Framework for Sparse Tensor-Vector Product Kernel on a High-Performance Computing Platform. IEEE Transactions on Parallel and Distributed Systems, 2020, 31(10):2329–2345

[22] Yang W, Li K, Li K. A Pipeline Computing Method of SpTV for Three-Order Tensors on CPU and GPU. ACM Transactions on Knowledge Discovery from Data, 2019, 13(6):1–27

[23] Li J, Ma Y, Yan C, et al. Optimizing Sparse Tensor Times Matrix on Multi-core and Many-Core Architectures. In: Proc of The 2016 6th Workshop on Irregular Applications: Architecture and Algorithms. Salt Lake City, UT, USA, 2016, 26–33

[24] Ma Y, Li J, Wu X, et al. Optimizing Sparse Tensor Times Matrix on GPUs. Journal of Parallel and Distributed Computing, 2019, 129:99–109

[25] Liu J, Ren J, Gioiosa R, et al. Sparta: High-performance, element-wise sparse tensor contraction on heterogeneous memory. In: Proc of The 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. Virtual Event, Republic

中国知网 https://www.cnki.net

of Korea, 2021, 318–333

[26] Hérault T, Robert Y, Bosilca G, et al. Distributed-Memory Multi-GPU Block-Sparse Tensor Contraction for Electronic Structure. In: Proc of The IEEE International Parallel and Distributed Processing Symposium. Portland, Oregon, USA, 2021, 537–546

[27] Wijeratne S, Wang T Y, Kannan R, et al. Accelerating Sparse MTTKRP for Tensor Decomposition on FPGA. In: Proc of Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays. Monterey, California, USA, 2023, 259–269

[28] Nisa I, Li J, Sukumaran-Rajam A, et al. Load-Balanced Sparse MTTKRP on GPUs. In: Proc of The IEEE International Parallel and Distributed Processing Symposium. Rio de Janeiro, Brazil, 2019, IEEE, 123–133

[29] Baert W, Vannieuwenhoven N. Accelerating Sparse Tensor-Times-Matrix-Chains with All-Accumulation. In: Proc of The 20th International Conference of Numerical Analysis and Applied Mathematics. Heraklion, Greece, 2022, AIP Publishing

[30] Smith S, Karypis G. Accelerating the Tucker Decomposition with Compressed Sparse Tensors. In: Proc of The European Conference on Parallel Processing. Santiago de Compostela, Spain, 2017, Springer, 653–668

[31] 陈国良, 孙广中, 徐云, et al. 并行计算的一体化研究现状与发展趋势. 科学通报, 2009, 54(08):1043–1049

[32] Valiant L G. A Bridging Model for Multi-Core Computing. Journal of Computer and System Sciences, 2011, 77(1):154–166

[33] Kalaiselvi T, Sriramakrishnan P, Somasundaram K. Survey of Using GPU CUDA Programming Model in Medical Image Analysis. Informatics in Medicine Unlocked, 2017, 9:133–144

[34] Choquette J, Gandhi W, Giroux O, et al. NVIDIA A100 Tensor Core GPU: Performance and Innovation. IEEE Micro, 2021, 41(2):29–35

[35] Liu Y, Du C, Chen J, et al. Scheduling Energy-Conscious Tasks in Distributed Heterogeneous Computing Systems. Concurrency and Computation: Practice and Experience, 2022, 34(1):e6520

[36] Mittal S, Vetter J S. A Survey of CPU-GPU Heterogeneous Computing Techniques. ACM Computing Surveys, 2015, 47(4):1–35

[37] Afif M, Said Y, Atri M. Computer Vision Algorithms Acceleration Using Graphic Processors NVIDIA CUDA. Cluster Computing, 2020, 23:3335–3347

[38] Zachariadis O, Satpute N, Gómez-Luna J, et al. Accelerating Sparse Matrix-Matrix Multiplication with GPU Tensor Cores. Computers & Electrical Engineering, 2020, 88:106848

中国知网 https://www.cnki.net

[39] Xiao G, Li K, Chen Y, et al. CASPMV: A Customized and Accelerated SpMV Framework for the Sunway TaihuLight. IEEE Transactions on Parallel and Distributed Systems, 2019, 32(1):131–146

[40] Pawłowski F, Uçar B, Yzelman A J. A Multi-Dimensional Morton-Ordered Block Storage for Mode-Oblivious Tensor Computations. Journal of Computational Science, 2019, 33:34–44

[41] Smith S, Karypis G. Tensor-Matrix Products with a Compressed Sparse Tensor. In: Proc of The 5th Workshop on Irregular Applications: Architectures and Algorithms. New York, NY, USA: Association for Computing Machinery, 2015

[42] Ozog D, Hammond J R, Dinan J, et al. Inspector-Executor Load Balancing Algorithms for Block-Sparse Tensor Contractions. In: Proc of The 42nd International Conference on Parallel Processing. Lyon, France, 2013, IEEE, 30–39

[43] Xiao G, Yin C, Chen Y, et al. GSpTC: High-Performance Sparse Tensor Contraction on CPU-GPU Heterogeneous Systems. In: Proc of The IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys). Hainan, China, 2022, IEEE, 380–387

[44] Zhao T, Popoola T, Hall M, et al. Polyhedral Specification and Code Generation of Sparse Tensor Contraction with Co-Iteration. ACM Transactions on Architecture and Code Optimization, 2022, 20(1):1–26

[45] Li J, Choi J, Perros I, et al. Model-Driven Sparse CP Decomposition for Higher-Order Tensors. In: Proc of The IEEE International Parallel and Distributed Processing Symposium. Orlando, Florida, USA, 2017, 1048–1057

[46] Hu R, Yang W, Zhou X, et al. Performance Analysis and Optimization for MTTKRP of Sparse Tensor on CPU and GPU. The IEEE 22nd International Conference on High Performance Computing and Communications; IEEE 18th International Conference on Smart City; IEEE 6th International Conference on Data Science and Systems, 2020. 545–550

[47] Kurt S E, Raje S, Sukumaran-Rajam A, et al. Sparsity-Aware Tensor Decomposition. In: Proc of The IEEE International Parallel and Distributed Processing Symposium. Lyon, France, 2022, IEEE, 952–962

[48] Nisa I, Li J, Sukumaran-Rajam A, et al. An Efficient Mixed-Mode Representation of Sparse Tensors. In: Proc of The International Conference for High Performance Computing, Networking, Storage and Analysis. Denver, Colorado, USA, 2019, 1–25

[49] Nguyen A K, Helal A, Checconi F, et al. Efficient, Out-of-Memory Sparse MTTKRP

中国知网 https://www.cnki.net

on Massively Parallel Architectures. In: Proc of The 36th ACM International Conference on Supercomputing. Virtual Event, 2022, 1–13

[50] Hayashi K, Ballard G, Jiang Y, et al. Shared-Memory Parallelization of MTTKRP for Dense Tensors. In: Proc of The 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. Vienna, Austria, 2018, 393–394

[51] Nagasaka Y, Fukumoto N. Efficient Collision-Free MTTKRP Algorithm for Multicore CPUs with Less Memory Usage. In: Proc of The 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing. Taormina, Italy, 2022, IEEE, 534–543

[52] Won J, Mendis C, Emer J S, et al. WACO: Learning Workload-Aware Co-optimization of the Format and Schedule of a Sparse Tensor Program. In: Proc of The 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. Vancouver, British Columbia, Canada, 2023, 920–934

[53] Li J, Uçar B, Çatalyürek Ü V, et al. Efficient and Effective Sparse Tensor Reordering. In: Proc of The ACM International Conference on Supercomputing. Phoenix, Arizona, USA, 2019, 227–237

[54] Chen Y, Xiao G, Özsu M T, et al. Exploiting Hierarchical Parallelism and Reusability in Tensor Kernel Processing on Heterogeneous HPC Systems. In: Proc of The IEEE 38th International Conference on Data Engineering. Kuala Lumpur, Malaysia, 2022, 2523–2536

[55] Qin E, Jeong G, Won W, et al. Extending Sparse Tensor Accelerators to Support Multiple Compression Formats. In: Proc of The IEEE International Parallel and Distributed Processing Symposium. Portland, Oregon, USA, 2021, 1014–1024

[56] Wijeratne S, Jaiswal A, Jacob A P, et al. Performance Modeling of Sparse MTTKRP Using Optical Static Random Access Memory on FPGA. In: Proc of The IEEE High Performance Extreme Computing Conference. Waltham, Massachusetts, USA, 2022, IEEE, 1–7

[57] Kaya O, Uçar B. High Performance Parallel Algorithms for the Tucker Decomposition of Sparse Tensors. In: Proc of The 45th International Conference on Parallel Processing. Philadelphia, Pennsylvania, USA, 2016, 103–112

[58] Li J, Sun J, Vuduc R. HiCOO: Hierarchical storage of sparse tensors. In: Proc of The International Conference for High Performance Computing, Networking, Storage and Analysis. Dallas Texas, 2018, IEEE, 238–252

[59] Liu B, Wen C, Sarwate A D, et al. A Unified Optimization Approach for Sparse Tensor Operations on GPUs. In: Proc of The 2017 IEEE International Conference on Cluster Computing. Honolulu, Hawaii, USA, 2017, 47–57

中国知网 https://www.cnki.net

[60] Huang G, Dai G, Wang Y, et al. GE-SpMM: General-purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. In: Proc of The International Conference for High Performance Computing, Networking, Storage and Analysis. Atlanta, Georgia, USA, 2020, 1–12

[61] Dakkak A, Li C, Xiong J, et al. Accelerating reduction and scan using tensor core units. In: Proc of Proceedings of the ACM International Conference on Supercomputing. Phoenix, Arizona, USA, 2019, 46–57

[62] Wang Y, Feng B, Ding Y. TC-GNN: Accelerating Sparse Graph Neural Network Computation Via Dense Tensor Core on GPUs. arXiv e-prints, 2021. arXiv:2112.02052

[63] Feng B, Wang Y, Chen G, et al. EGEMM-TC: Accelerating Scientific Computing on Tensor Cores with Extended Precision. In: Proc of The 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. Virtual Event, Republic of Korea, 2021, 278–291

[64] Xie Z, Tan G, Liu W, et al. IA-SpGEMM: An Input-Aware Auto-Tuning Framework for Parallel Sparse Matrix-Matrix Multiplication. In: Proc of The ACM International Conference on Supercomputing. Phoenix, Arizona, USA, 2019, 94–105

[65] Zhao Y, Li J, Liao C, et al. Bridging the Gap between Deep Learning and Sparse Matrix Format Selection. In: Proc of The 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. New York, NY, USA: Association for Computing Machinery, 2018, 94–108

[66] Sun Q, Liu Y, Dun M, et al. SpTFS: Sparse Tensor Format Selection for MTTKRP via Deep Learning. In: Proc of The International Conference for High Performance Computing, Networking, Storage and Analysis. Atlanta, Georgia, USA, 2020, 1–14

[67] Sun Q, Liu Y, Yang H, et al. Input-Aware Sparse Tensor Storage Format Selection for Optimizing MTTKRP. IEEE Transactions on Computers, 2022, 71(8):1968–1981

[68] Tan G, Liu J, Li J. Design and Implementation of Adaptive SpMV Library for Multicore and Many-Core Architecture. ACM Transactions on Mathematical Software, 2018, 44(4):1 – 25

[69] Benatia A, Ji W, Wang Y, et al. Sparse Matrix Format Selection with Multiclass SVM for SpMV on GPU. In: Proc of The 45th International Conference on Parallel Processing. Philadelphia, Pennsylvania, USA, 2016, 496–505

[70] Sedaghati N, Mu T, Pouchet L N, et al. Automatic Selection of Sparse Matrix Representation on GPUs. In: Proc of The 29th ACM on International Conference on Supercomputing. Newport Beach/Irvine, California, USA, 2015

[71] Niu Y, Lu Z, Dong M, et al. TileSpMV: A Tiled Algorithm for Sparse Matrix-Vector

中国知网 https://www.cnki.net

Multiplication on GPUs. In: Proc of The IEEE International Parallel and Distributed Processing Symposium. Portland, Oregon, USA, 2021, 68–78

[72] 袁娥, 张云泉, 刘芳芳, et al. SpMV 的自动性能优化实现技术及其应用研究. 计算机研究与发展, 2009, 46(7):1117–1126

[73] 李佳佳, 张秀霞, 谭光明, et al. 选择稀疏矩阵乘法最优存储格式的研究. 计算机研究与发展, 2014, 51(4):882–894

[74] Su B Y, Keutzer K. clSpMV: A cross-platform OpenCL SpMV framework on GPUs. In: Proc of The 26th ACM international conference on Supercomputing. Venice, Italy, 2012, 353–364

[75] Popoola T, Zhao T, St. George A, et al. Code Synthesis for Sparse Tensor Format Conversion and Optimization. In: Proc of The 21st ACM/IEEE International Symposium on Code Generation and Optimization. Montréal, Quebec, Canada, 2023, 28–40

[76] Dai G, Huang G, Yang S, et al. Heuristic Adaptability to Input Dynamics for SpMM on GPUs. In: Proc of The 59th ACM/IEEE Design Automation Conference. San Francisco, California, USA, 2022, 595–600

[77] Kim J, Sukumaran-Rajam A, Thumma V, et al. A Code Generator for High-performance Tensor Contractions on GPUs. In: Proc of The IEEE/ACM International Symposium on Code Generation and Optimization. Washington, District of Columbia, USA, 2019, IEEE, 85–95

[78] Du Z, Li J, Wang Y, et al. Alphasparse: Generating High Performance SpMV Codes Directly from Sparse Matrices. In: Proc of The International Conference for High Performance Computing, Networking, Storage and Analysis. Dallas, Texas, USA, 2022, IEEE, 1–15

[79] Vuduc R, Demmel J W, Yelick K A. OSKI: A Library of Automatically Tuned Sparse Matrix Kernels. In: Proc of Journal of Physics: Conference Series. IOP Publishing, 2005, 521

[80] Zheng L, Jia C, Sun M, et al. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In: Proc of The 14th USENIX Symposium on Operating Systems Design and Implementation. Virtual Event, 2020, 863–879

[81] Duan L, Xiao C, Li M, et al. A-Tucker: Fast Input-Adaptive and Matricization-Free Tucker Decomposition of Higher-Order Tensors on GPUs. CCF Transactions on High Performance Computing, 2023, 5(1):12–25

[82] Li J, Battaglino C, Perros I, et al. An Input-Adaptive and in-Place Approach to Dense Tensor-Times-Matrix Multiply. In: Proc of The International Conference for High Performance Computing, Networking, Storage and Analysis. New York, NY, USA:

中国知网 https://www.cnki.net

Association for Computing Machinery, 2015

[83] NGUYEN A. Accelerating Sparse Matricized Tensor Times Khatri Rao Product on Massively Parallel Architectures: [Dissertation]. Eugene, Oregon, USA: University of Oregon, 2023

[84] Cheng S, Liu Z, Du J, et al. ATP: Adaptive Tensor Parallelism for Foundation Models, 2023

[85] Kourtis K, Karakasis V, Goumas G, et al. CSX: An Extended Compression Format for SpMV on Shared Memory Systems. ACM SIGPLAN Notices, 2011, 46(8):247–256

[86] Gibson P, Cano J. Transfer-Tuning: Reusing Auto-Schedules for Efficient Tensor Program Code Generation. In: Proc of The International Conference on Parallel Architectures and Compilation Techniques. Chicago, Illinois, 2022, 28–39

[87] Zheng S, Liang Y, Wang S, et al. FlexTensor: An Automatic Schedule Exploration and Optimization Framework for Tensor Computation on Heterogeneous System. In: Proc of The Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. New York, NY, USA, 2020, 859–873

[88] Akiba T, Sano S, Yanase T, et al. Optuna: A Next-Generation Hyperparameter Optimization Framework. In: Proc of The 25th ACM SIGKDD international conference on knowledge discovery & data mining. Anchorage, Alaska, USA, 2019, 2623–2631

[89] Sreenivasan V, Javali R, Hall M, et al. A Framework for Enabling OpenMP Autotuning. In: Proc of The International Workshop on OpenMP. Auckland, New Zealand, 2019, Springer, 50–60

[90] Verma G, Raskar S, Xie Z, et al. Transfer Learning Across Heterogeneous Features For Efficient Tensor Program Generation, 2023

[91] Zhao Z, Shuai X, Ling N, et al. Moses: Exploiting Cross-Device Transferable Features for on-Device Tensor Program Optimization. In: Proc of The 24th International Workshop on Mobile Computing Systems and Applications. Newport Beach, California, USA, 2023, 22–28

[92] Zhang S, Li M, Yang H, et al. FamilySeer: Towards Optimized Tensor Codes by Exploiting Computation Subgraph Similarity, 2022

[93] Tang S, Zhai J, Wang H, et al. FreeTensor: A Free-Form DSL with Holistic Optimizations for Irregular Tensor Programs. In: Proc of The 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation. San Diego, California, USA, 2022, 872–887

[94] Zheng B, Jiang Z, Yu C H, et al. DietCode: Automatic Optimization for Dynamic Tensor Programs. In: Proc of The Machine Learning and Systems. Santa Clara,

中国知网 https://www.cnki.net

California, USA, 2022, 848–863

[95] Zhai Y, Zhang Y, Liu S, et al. TLP: A Deep Learning-Based Cost Model for Tensor Program Tuning. In: Proc of The 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. Vancouver, British Columbia, Canada, 2023, 833–845

[96] Kolda T G, Bader B W. Tensor Decompositions and Applications. SIAM review, 2009, 51(3):455–500

[97] Kaya O, Uçar B. Scalable Sparse Tensor Decompositions in Distributed Memory Systems. In: Proc of The International Conference for High Performance Computing, Networking, Storage and Analysis. Austin, Texas, USA, 2015, 1–11

[98] Liu S, Trenkler G. Hadamard, Khatri-Rao, Kronecker and Other Matrix Products. International Journal of Information and Systems Sciences, 2008, 4(1):160–177

[99] Weng T, Zhou X, Li K, et al. Efficient Distributed Approaches to Core Maintenance on Large Dynamic Graphs. IEEE Transactions on Parallel and Distributed Systems, 2021, 33(1):129–143

[100] Li K, Yang W, Li K. Performance Analysis and Optimization for SpMV on GPU Using Probabilistic Modeling. IEEE Transactions on Parallel and Distributed Systems, 2014, 26(1):196–205

[101] Lee D, Shin K. Robust Factorization of Real-world Tensor Streams with Patterns, Missing Values, and Outliers. In: Proc of The 2021 IEEE 37th International Conference on Data Engineering. Chania, Greece, 2021, 840–851

[102] Bader B W, Kolda T G. Efficient MATLAB Computations with Sparse and Factored Tensors. SIAM Journal on Scientific Computing, 2008, 30(1):205–231

[103] Li J, Ma Y, Vuduc R. ParTI! : A Parallel Tensor Infrastructure for Multicore CPUs and GPUs, 2018-Oct. Last updated: Jan 2020

[104] Vuduc R W. Automatic Performance Tuning of Sparse Matrix Kernels. Berkeley, California, USA: University of California, Berkeley, 2003

[105] Morris C, Ritzert M, Fey M, et al. Weisfeiler and Leman Go Neural: Higher-order Graph Neural Networks. In: Proc of The Thirty-Third AAAI Conference on Artificial Intelligence. Honolulu, Hawaii, 2019, 4602–4609

[106] Benoit A, Robert Y. Mapping Pipeline Skeletons onto Heterogeneous Platforms. Journal of Parallel and Distributed Computing, 2008, 68:790–808

[107] Niu Y, Lu Z, Ji H, et al. TileSpGEMM: A Tiled Algorithm for Parallel Sparse General Matrix-Matrix Multiplication on GPUs. In: Proc of The 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. Seoul, Republic of Korea, 2022, 90–106

中国知网 https://www.cnki.net

[108] Smith S, Ravindran N, Sidiropoulos N, et al. SPLATT: Efficient and Parallel Sparse Tensor-Matrix Multiplication. The IEEE International Parallel and Distributed Processing Symposium, 2015. 61–70

[109] Garland M, Le Grand S, Nickolls J, et al. Parallel Computing Experiences with CUDA. IEEE Micro, 2008, 28(4):13–27

[110] Srivastava N, Jin H, Smith S, et al. Tensaurus: A Versatile Accelerator for Mixed Sparse-Dense Tensor Computations. In: Proc of The IEEE International Symposium on High Performance Computer Architecture. San Diego, California, USA, 2020, 689–702

[111] Zhang Y, Yang W, Li K, et al. Performance Analysis and Optimization for SpMV Based on Aligned Storage Formats on an ARM Processor. Journal of Parallel and Distributed Computing, 2021, 158:126–137

[112] Bell N, Garland M. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In: Proc of The Conference on High Performance Computing Networking, Storage and Analysis. New York, NY, USA: Association for Computing Machinery, 2009

[113] Hendrickson B, Kolda T G. Partitioning Rectangular and Structurally Unsymmetric Sparse Matrices for Parallel Processing. SIAM Journal on Scientific Computing, 2000, 21(6):2048–2072

[114] Ramchoun H, Chikh M A, Idrissi J, et al. Multilayer Perceptron: Architecture Optimization and Training. International Journal of Interactive Multimedia and Artificial Intelligence, 2016, 4(1):26–30

[115] Fey M, Lenssen J E. Fast Graph Representation Learning with PyTorch Geometric. CoRR, 2019, abs/1903.02428

[116] Kurdila A J, Zabarankin M. Convex functional analysis. Berlin, Germany: Springer Science & Business Media, 2006

[117] Cantador I, Brusilovsky P, Kuflik T. Second Workshop on Information Heterogeneity and Fusion in Recommender Systems. In: Proc of The 5th ACM conference on Recommender systems. New York, NY, USA: ACM, 2011, 387–388

[118] Harper F M, Konstan J A. The movielens datasets: History and context. Acm transactions on interactive intelligent systems (tiis), 2015, 5(4):1–19

[119] Smith S, Choi J W, Li J, et al. FROSTT: The Formidable Repository of Open Sparse Tensors and Tools, 2017

[120] Globerson A, Chechik G, Pereira F, et al. Euclidean Embedding of Co-Occurrence Data. In: Saul L, Weiss Y, Bottou L, (eds.). Proc of The Advances in Neural Information Processing Systems. Vancouver, British Columbia, Canada: MIT Press, 2004,

中国知网 https://www.cnki.net

497–504

[121] Yang W, Li K, Li K. A Parallel Computing Method Using Blocked Format with Optimal Partitioning for SpMV on GPU. Journal of Computer and System Sciences, 2018, 92:152–170

[122] Ho J C, Ghosh J, Steinhubl S R, et al. Limestone: High-Throughput Candidate Phenotype Generation via Tensor Factorization. Journal of Biomedical Informatics, 2014, 52:199–211

[123] Lahat D, Adalỳ T, Jutten C. Challenges in Multimodal Data Fusion. In: Proc of The 22nd European Signal Processing Conference. Lisbon, Portugal, 2014, IEEE, 101–105

[124] Latchoumane C F V, Vialatte F B, Solé-Casals J, et al. Multiway Array Decomposition Analysis of EEGs in Alzheimer's Disease. Journal of neuroscience methods, 2012, 207(1):41–50

[125] Anandkumar A, Ge R, Hsu D, et al. Tensor Decompositions for Learning Latent Variable Models. Journal of Machine Learning Research, 2014, 15:2773–2832

[126] Shivakumar S, Li J, Kannan R, et al. Efficient Parallel Sparse Symmetric Tucker Decomposition for High-Order Tensors. In: Proc of The SIAM Conference on Applied and Computational Discrete Algorithms. Virtual Conference, 2021, SIAM, 193–204

[127] 卢凯, 赖志权, 李笙维, et al. 并行智能训练技术: 挑战与发展. 中国科学: 信息科学, 2023, 53(08):1441–1468

[128] Nisa I, Siegel C, Rajam A S, et al. Effective Machine Learning Based Format Selection and Performance Modeling for SpMV on GPUs. In: Proc of The IEEE International Parallel and Distributed Processing Symposium Workshops. Vancouver, British Columbia, Canada, 2018, IEEE, 1056–1065

[129] Chen Y, Xiao G, Li K, et al. FgSpMSpV: A Fine-grained Parallel SpMSpV Framework on HPC Platforms. ACM Transactions on Parallel Computing, 2022, 9(2):1–29

[130] Anzt H, Cojean T, Yen-Chen C, et al. Load-balancing Sparse Matrix Vector Product Kernels on GPUs. ACM Transactions on Parallel Computing, 2020, 7(1):1–26

[131] Merrill D, Garland M, Grimshaw A. High-Performance and Scalable GPU Graph Traversal. ACM Transactions on Parallel Computing, 2015, 1(2):1–30

[132] Yu Y, Wen D, Zhang Y, et al. Efficient Matrix Factorization on Heterogeneous CPU-GPU Systems. In: Proc of The IEEE 37th International Conference on Data Engineering. Chania, Greece, 2021, 1871–1876

[133] Weng T, Zhou X, Li K, et al. Distributed Approaches to Butterfly Analysis on Large Dynamic Bipartite Graphs. IEEE Transactions on Parallel and Distributed Systems, 2022, 34(2):431–445

中国知网 https://www.cnki.net

[134] Chen C, Li K, Teo S G, et al. Citywide Traffic Flow Prediction Based on Multiple Gated Spatio-temporal Convolutional Neural Networks. ACM Transactions on Knowledge Discovery from Data, 2020, 14(4):1–23

[135] Gilmer J, Schoenholz S S, Riley P F, et al. Neural Message Passing for Quantum Chemistry. In: Proc of The 34th International Conference on Machine Learning. Sydney, NSW, Australia: PMLR, 2017, 1263–1272

[136] Ying Z, You J, Morris C, et al. Hierarchical Graph Representation Learning with Differentiable Pooling. In: Proc of The Advances in Neural Information Processing Systems. Montréal, Canada, 2018

[137] Gray J, Kourtis S. Hyper-Optimized Tensor Network Contraction. Quantum, 2021, 5:410

[138] Kim J, Sukumaran-Rajam A, Hong C, et al. Optimizing Tensor Contractions in CCSD(T) for Efficient Execution on GPUs. In: Proc of The 2018 International Conference on Supercomputing. Beijing, China, 2018, 96–106

[139] Roberts C, Milsted A, Ganahl M, et al. TensorNetwork: A Library for Physics and Machine Learning, 2019

[140] Kossaifi J, Khanna A, Lipton Z, et al. Tensor Contraction Layers for Parsimonious Deep Nets. In: Proc of The IEEE Conference on Computer Vision and Pattern Recognition Workshops. Honolulu, Hawaii, USA, 2017, 26–32

[141] Kossaifi J, Lipton Z C, Kolbeinsson A, et al. Tensor Regression Networks. Journal of Machine Learning Research, 2020, 21(1):4862–4882

[142] Fishman M, White S, Stoudenmire E. The ITensor Software Library for Tensor Network Calculations. SciPost Physics Codebases, 2022. 004

[143] Matthews D A. High-Performance Tensor Contraction without Transposition. SIAM Journal on Scientific Computing, 2018, 40(1):C1–C24

[144] Martinez-Ferrer P J, Yzelman A, Beltran V. A Native Tensor–Vector Multiplication Algorithm for High Performance Computing. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(12):3363–3374

[145] Bell N, Dalton S, Olson L N. Exposing Fine-Grained Parallelism in Algebraic Multigrid Methods. SIAM Journal on Scientific Computing, 2012, 34(4):C123–C152

[146] Epifanovsky E, Kowalski K, Fan P D, et al. On the Electronically Excited States of Uracil. The Journal of Physical Chemistry A, 2008, 112(40):9983–9992

[147] Sivkov I, Seewald P, Lazzaro A, et al. DBCSR: A Blocked Sparse Tensor Algebra Library. CoRR, 2019, abs/1910.13555

[148] Jang J G, Kang U. Fast and Memory-Efficient Tucker Decomposition for Answering Diverse Time Range Queries. In: Proc of The 27th ACM SIGKDD Conference on

中国知网 https://www.cnki.net

Knowledge Discovery & Data Mining. Virtual Event, Singapore, 2021, 725–735

[149] Warlop R, Mary J, Gartrell M. Tensorized Determinantal Point Processes for Recommendation. In: Proc of The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. Anchorage, Alaska, USA, 2019, 1605–1615

[150] Xie K, Li X, Wang X, et al. Fast Tensor Factorization for Accurate Internet Anomaly Detection. IEEE/ACM Transactions on Networking, 2017, 25(6):3794–3807

[151] 武昱, 闫光辉, 王雅斐, et al. 结合 GPU 技术的并行 CP 张量分解算法. 计算机科学, 2018, 45(11):298–303+317

[152] Chowdhury R, Silvestri F, Vella F. A Computational Model for Tensor Core Units. In: Proc of The 32nd ACM Symposium on Parallelism in Algorithms and Architectures. Virtual Event, USA, 2020, 519–521

[153] Li A, Su S. Accelerating Binarized Neural Networks via Bit-Tensor-Cores in Turing GPUs. IEEE Transactions on Parallel and Distributed Systems, 2020, 32(7):1878–1891

[154] Psarras C, Karlsson L, Bientinesi P. The Landscape of Software for Tensor Computations. CoRR, 2021, abs/2103.13756

[155] Zhang T, Liu X Y, Wang X, et al. cuTensor-Tubal: Efficient Primitives for Tubal-Rank Tensor Learning Operations on GPUs. IEEE Transactions on Parallel and Distributed Systems, 2019, 31(3):595–610

[156] Xiao Q, Zheng S, Wu B, et al. HASCO: Towards Agile HArdware and Software CO-design for Tensor Computation. In: Proc of The ACM/IEEE 48th Annual International Symposium on Computer Architecture. Valencia, Spain, 2021, IEEE, 1055–1068

[157] Oh S, Park N, Jang J G, et al. High-Performance Tucker Factorization on Heterogeneous Platforms. IEEE Transactions on Parallel and Distributed Systems, 2019, 30(10):2237–2248

[158] Abubaker N, Acer S, Aykanat C. True Load Balancing for Matricized Tensor Times Khatri-Rao Product. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(8):1974–1986

[159] Choi J, Liu X, Smith S, et al. Blocking Optimization Techniques for Sparse Tensor Computation. In: Proc of The IEEE International Parallel and Distributed Processing Symposium. Vancouver, British Columbia, Canada, 2018, IEEE, 568–577

[160] Navarro C A, Carrasco R, Barrientos R J, et al. GPU Tensor Cores for Fast Arithmetic Reductions. IEEE Transactions on Parallel and Distributed Systems, 2020, 32(1):72–84

[161] Huang H, Liu X Y, Tong W, et al. High Performance Hierarchical Tucker Ten-

中国知网 https://www.cnki.net

sor Learning Using GPU Tensor Cores. IEEE Transactions on Computers, 2022, 72(2):452–465

[162] Li K, Tang X, Veeravalli B, et al. Scheduling Precedence Constrained Stochastic Tasks on Heterogeneous Cluster Systems. IEEE Transactions on Computers, 2013, 64(1):191–204

[163] Dalton S, Bell N, Olson L, et al. Cusp: Generic Parallel Algorithms for Sparse Matrix and Graph Computations, 2014. http://cusplibrary.github.io/. Version 0.5.0

中国知网 https://www.cnki.net